Escola de Engenharia Elétrica, Mecânica e de Computação

CERISE – Centro de Excelência em Redes Inteligentes Sem Fio e Serviços Avançados

Projeto de Implantação do CERISE-Centro de Excelência em Redes Inteligentes Sem fio e Serviços Avançados com Alcance Nacional e Internacional

Relatório Técnico-Científico

Título do Plano de Trabalho Individual: Pesquisa, Desenvolvimento e Inovação em Gêmeos Digitais, Tecnologias Imersivas, Robótica e Navegação Autônoma no escopo do CERISE/EMC/UFG

Bolsista: Eduardo Martins de Carvalho Caixeta

Grupo / Eixo Temático: Gêmeos Digitais / Navegação Autônoma / Realidade Aumentada

Coordenadores dos Grupos / Eixos Temáticos: Prof. Dr. Alisson Assis Cardoso (Navegação Autônoma e Gêmeos Digitais); Prof. Dr. Carlos Galvão Pinheiro Junior (Realidade Aumentada)

Versão: 26/10/2026

1. INTRODUÇÃO E CONTEXTUALIZAÇÃO

Neste relatório, são apresentadas as atividades e os resultados obtidos como bolsista do CERISE ao longo de 2026, referentes ao plano de trabalho individual intitulado “Pesquisa, Desenvolvimento e Inovação em Gêmeos Digitais, Tecnologias Imersivas, Robótica e Navegação Autônoma no escopo do CERISE/EMC/UFG”.

Primeiramente, apresenta-se uma introdução com a contextualização do trabalho. Em seguida, são expostos os objetivos constantes no plano de trabalho, as atividades desenvolvidas, os resultados obtidos e uma comparação entre as atividades previstas e aquelas efetivamente executadas, incluindo as pendências.

A realidade aumentada (RA) vem se destacando como uma tecnologia emergente com potencial para transformar diversas áreas da sociedade. Ao mesclar elementos virtuais ao ambiente físico em tempo real, a RA possibilita interações mais dinâmicas e informativas, com impacto direto em setores como saúde, educação, engenharia e entretenimento [1]. Um dos avanços mais notáveis está relacionado à sua crescente acessibilidade, impulsionada por dispositivos móveis: mapeamento tridimensional, reconhecimento de objetos e processamento em tempo real tornaram a RA mais precisa e adaptável a diferentes contextos [2]. Em aplicações industriais, a RA tem sido empregada em manutenção, treinamento técnico e assistência remota, enquanto no campo educacional enriquece experiências de aprendizado por meio de simulações imersivas [3], e na área da saúde apoia procedimentos e treinamentos médicos [4].

Um gargalo recorrente dessas aplicações é a obtenção de conteúdo tridimensional. Em 2025, foi desenvolvido um aplicativo capaz de capturar fotografias de um objeto e enviá-las a um servidor para a construção do modelo 3D correspondente, etapa que, naquela versão, era apenas simulada pelo servidor. Em 2026, a evolução de redes neurais generativas de imagem para 3D, capazes de produzir malhas texturizadas a partir de uma única fotografia em segundos, motivou a integração de um modelo dessa classe ao aplicativo, com otimização de desempenho e com um servidor capaz de atender múltiplos usuários simultaneamente.

Paralelamente, o conceito de gêmeo digital, uma representação virtual sincronizada com um sistema físico, depende de mecanismos de percepção capazes de estimar o estado do sistema real. No caso de braços robóticos, a estimação de pose a partir de imagens, sem acesso aos estados das juntas, permite construir gêmeos digitais de robôs sem instrumentação adicional. Em 2025, foi treinado um modelo de estimação de pose de braço robótico baseado em visão computacional e profundidade monocular; em 2026, o trabalho

avançou para a geração de um novo conjunto de dados com um braço robótico virtual, a validação do modelo nesse novo cenário e a construção do gêmeo digital do braço.

Por fim, no eixo de Navegação Autônoma, o uso cooperativo de múltiplos drones para monitoramento de áreas, por exemplo para detecção e contagem de pessoas, exige a divisão da área de atuação entre as aeronaves, o planejamento de rotas de cobertura e um enlace de comunicação de longo alcance e baixo consumo, papel para o qual a tecnologia LoRa é candidata natural.

2. OBJETIVOS DO PLANO DE TRABALHO

O objetivo geral deste plano de trabalho é realizar pesquisa, desenvolvimento e inovação em gêmeos digitais, tecnologias imersivas, robótica e navegação autônoma no escopo do CERISE/EMC/UFG. O plano está organizado em dois eixos:

- Desenvolvimento, aprimoramento e validação de soluções baseadas em inteligência artificial e visão computacional, visando à geração de objetos tridimensionais com textura e melhor desempenho no aplicativo desenvolvido em 2025; à utilização de um braço robótico, físico ou virtual, para a geração de novos conjuntos de dados; à validação do modelo de detecção de pose desenvolvido no ano anterior; ao desenvolvimento de um gêmeo digital do braço robótico; e ao desenvolvimento de software para segmentação de áreas em cenários com múltiplos drones.
- Desenvolvimento e consolidação da infraestrutura computacional, dos sistemas embarcados e dos mecanismos de comunicação do projeto, incluindo a otimização do servidor do aplicativo para múltiplas solicitações simultâneas, a publicação do aplicativo na Play Store, a continuidade do projeto de comunicação baseado em LoRa e o desenvolvimento de algoritmos de colaboração entre drones.

Os objetivos específicos para a consecução do plano de trabalho são:

- Objetivo 1 — Inteligência artificial aplicada à geração de modelos tridimensionais: integrar ao aplicativo um modelo de rede neural capaz de gerar objetos 3D com textura, otimizar o processamento e validar a aplicação em uso real.
- Objetivo 2 — Infraestrutura computacional e implantação do aplicativo: otimizar o servidor para múltiplas solicitações, adequar a arquitetura para escalabilidade e confiabilidade e publicar o aplicativo na Play Store.

- Objetivo 3 — Geração de dados e validação de modelos de detecção de pose: utilizar um braço robótico (físico ou virtual) para gerar um novo conjunto de dados, validar o modelo de detecção de pose do ano anterior e analisar seu desempenho e robustez.
- Objetivo 4 — Desenvolvimento de gêmeo digital do braço robótico: modelar o braço em ambiente virtual, integrar o modelo de detecção de pose ao ambiente digital e reproduzir o comportamento do braço com base nos dados capturados.
- Objetivo 5 — Planejamento e adequação de sistema com drones para detecção e contagem de pessoas: planejar o sistema, desenvolver software de segmentação da área de atuação entre os drones, construir os módulos de comunicação definitivos e adequar os drones para recebê-los.

3. ATIVIDADES REALIZADAS

As atividades foram organizadas em seis frentes (geração 3D, servidor e aplicativo, dataset e gêmeo digital do braço, estimação de pose, comunicação LoRa e drones), cada uma com código em repositório privado no GitHub, resultados medidos em arquivos JSON e CSV e evidências gráficas. Os experimentos com GPU, o servidor de geração 3D e as simulações do braço foram executados na estação NVIDIA DGX Spark do laboratório (superchip GB10, com GPU Blackwell e CPU aarch64, e 128 GB de memória unificada), compartilhada com serviços de IA em produção; a compilação iOS usou um Mac mini M4, e as compilações Android e o emulador, um notebook com Windows. Na numeração do plano de trabalho, foram realizadas:

- Atividade 1: comparação de 10 modelos abertos de geração 3D texturizada, com 6 medidos em até 13 objetos do Google Scanned Objects (subconjuntos de 4 e 2 objetos para o Hunyuan3D-2.0 e o 2.1), e seleção do Hunyuan3D-2 Turbo;
- Atividade 2: integração do aplicativo CERISE AR 1.1.0 ao servidor real de geração 3D, validada no emulador Android, e compilação para o Simulador iOS;
- Atividade 3: otimização do modelo, três configurações de qualidade (*presets*) e empacotamento do pipeline;
- Atividade 4: validação dos objetos gerados por testes automatizados, visualizadores e análise qualitativa;
- Atividade 5: análise dos gargalos do servidor de 2025 e implantação de uma arquitetura assíncrona com fila persistente;

- Atividade 6: adequação do aplicativo à Play Store e preparação do pacote de publicação, sem a publicação;
- Atividade 7: dataset sintético do braço virtual servo6dof, com 5.000 poses e 30.000 imagens;
- Atividades 8 e 9: adaptação do modelo de pose de 2025 por transferência de aprendizado e avaliação de precisão e robustez;
- Atividade 10 e itens 3 e 4 do Objetivo 5: software do protótipo LoRa, sem hardware de rádio, e projeto do módulo dos drones;
- Atividades 11 e 12: modelo digital do braço e gêmeo digital com o modelo de pose, em simulação;
- Atividade 13 e algoritmos de colaboração: segmentação da área entre drones e realocação dinâmica, em simulação 2D;
- Item 1 do Objetivo 5: detecção de pessoas em imagens aéreas e agregação das contagens entre drones, com validação da contagem em vídeos reais de drone (VisDrone2019-MOT) e simulação 3D fotorrealista no NVIDIA Isaac Sim, com quatro drones percorrendo a área da UFG e 80 pessoas animadas;
- Atividade 14: elaboração deste relatório e publicação de dois artigos científicos sobre a estimação de pose de braço robótico desenvolvida no projeto em 2025, no MEDER 2026 e no CBA 2026, este último com apresentação de pôster;
- Participação no CERISE Summit 2026 (18 de setembro de 2026), evento de extensão do CERISE cadastrado na PROEX UFG sob o código EV308-2026, como expositor das apresentações de projetos e protótipos e como monitor do evento (10 horas em cada função).

Os códigos implementados estão nos repositórios da organização cerise-ufg do GitHub, listados nos Anexos. Os vídeos dos protótipos estão nas pastas de evidências desses repositórios: a simulação no Isaac Sim com quatro drones contando pessoas e o close-up da simulação, na pasta simulacao/evidencias, e a contagem em vídeos reais de drone, na pasta validacao_videos/evidencias, ambas do repositório cerise-2026-drones-segmentacao; e a varredura de poses do gêmeo digital do braço, na pasta evidencias do repositório cerise-2026-servo6dof-gemeo-digital. Para implementação dos protótipos, pode-se seguir o roteiro disponível na pasta Roteiros do CERISE, no documento Roteiro de Implementação CERISE 2026, que traz, para cada frente, os requisitos, a obtenção do código e dos dados, os comandos de cada etapa, os resultados esperados e os problemas comuns. A parte de cada frente também está no arquivo ROTEIRO.md do repositório de código correspondente.

4. RESULTADOS OBTIDOS

Nesta seção, é apresentado um resumo dos resultados obtidos:

1) Análise comparativa e seleção do modelo de geração 3D: o Hunyuan3D-2 Turbo foi selecionado, com a melhor geometria medida (distância de Chamfer de 0,032 e F-score@0,05 de 0,951);

2) Otimização do modelo integrado e desempenho: redução de cerca de 38 % no tempo de geração e três *presets* com tempos medianos de 23,9 s, 30,4 s e 82,1 s, limites superiores medidos com a GPU compartilhada;

3) Validação dos objetos 3D gerados: 58 de 58 testes automatizados aprovados e 7 de 7 arquivos GLB carregados nos visualizadores;

4) Servidor de geração 3D: taxa de erro reduzida de 90,9 % para zero e latência p95 até 22 % menor com 16 clientes simultâneos;

5) Aplicativo CERISE AR com geração de objetos 3D texturizados: fluxo da foto ao modelo texturizado validado no emulador contra o servidor real, em 13,5 s (*preset* rápido) a 31,5 s (*preset* alto) sem fila;

6) Adequação do aplicativo para a Play Store: pacote 1.1.0 pronto e verificado, com publicação pendente;

7) Dataset do braço robótico: 5.000 poses e 30.000 imagens, verificadas integralmente e documentadas;

8) Aplicação do modelo de pose de 2025 ao novo dataset: acurácia ADD@60 mm de 68,0 % com transferência de aprendizado, contra 32,8 % da linha de base inicializada no ImageNet;

9) Avaliação de precisão e robustez do modelo de pose: robustez a brilho e contraste; sensibilidade a ruído, desfoque e compressão; e latência de 162,5 ms por imagem;

10) Modelo digital do braço robótico: cinemática validada contra o ROS 2 e IoU de silhueta de 0,950 e 0,938 nas duas câmeras, sem calibração de massas e inércias;

11) Gêmeo digital do braço robótico: erro médio de 5,41° nas juntas 1 a 4 e latência mediana de 99,6 ms, em simulação;

12) Comunicação LoRa: software do protótipo verificado sem hardware (51 testes automatizados e 1.130 execuções de simulação) e projeto do módulo dos drones, com o ensaio físico e a fabricação pendentes;

13) Segmentação da área de atuação entre drones: menor *makespan* em 31 de 40 casos com o DARP com refinamento por tempo;

14) Colaboração entre drones e realocação dinâmica: cobertura de pelo menos 99,7 % após o abandono de um drone, com aumento médio de 23,1 % no *makespan*;

15) Detecção e contagem de pessoas em imagens aéreas: mAP50 de pessoa de 0,528 (validação) e 0,316 (test-dev) após ajuste fino;

16) Validação da contagem em vídeos reais de drone: HOTA de 0,328 e MAE de 7,46 pessoas por quadro no VisDrone2019-MOT test-dev, mostrando que a contagem de pessoas distintas deve ser feita pela posição no solo, e não pelos IDs do rastreador;

17) Simulação 3D de drones contando pessoas: voos fotorrealistas no Isaac Sim com quatro drones e 80 pessoas animadas, com AP50 de 0,702 por quadro e contagem única global com erro de −1,3 % a 40 m e −4,1 % a 30 m na configuração calibrada;

18) Produção científica: dois artigos publicados (MEDER 2026 e CBA 2026) e um pôster apresentado no CBA 2026, sobre a estimação de pose com profundidade monocular (acurácia de 75,4 % com limiar de 60 mm no conjunto real DREAM, com cerca de 70 % menos dados sintéticos de treino);

19) Participação no CERISE Summit 2026 como expositor de projetos e protótipos e como monitor do evento.

As subseções a seguir detalham cada um desses tópicos.

4.1. Análise comparativa e seleção do modelo de geração 3D

Esta etapa corresponde à Atividade 1 do Objetivo 1, que prevê a análise comparativa de redes neurais de geração de objetos 3D com textura. O aplicativo de 2025 enviava as fotografias a um servidor que apenas simulava a reconstrução e devolvia sempre o mesmo arquivo GLB; o objetivo foi comparar experimentalmente os modelos abertos de imagem para 3D texturizado e selecionar a rede a ser integrada ao aplicativo.

4.1.1. Metodologia

Os experimentos foram executados na estação NVIDIA DGX Spark (GB10), com GPU de arquitetura Blackwell (sm_121), CPU aarch64 de 20 núcleos e 128 GB de memória unificada, num contêiner derivado de `nvcr.io/nvidia/pytorch:26.05-py3` (PyTorch 2.12, CUDA 13.2), com o alocador limitado a 24 GB. A estação é de produção e foi compartilhada durante todas as medições; por isso, os tempos absolutos devem ser lidos como limites superiores.

O conjunto de avaliação reuniu 13 objetos do Google Scanned Objects (GSO) [5] (licença CC-BY 4.0), com malha escaneada de referência,

renderizados com nvdiffrast [6] numa vista de entrada de 1024×1024 px e em três vistas não observadas pelos modelos. Na geometria, foram calculados a distância de Chamfer e o F-score a τ = 0,05 e 0,1, após alinhamento por ICP com escala uniforme [7]; na aparência, a similaridade de cosseno CLIP ViT-L/14 [8] entre a entrada e a imagem renderizada do resultado (CLIP entrada) e entre resultado e referência nas vistas novas (CLIP vistas novas). Foram medidos ainda a mediana do tempo por objeto, com os modelos já carregados, e o pico de memória. Um autoteste do alinhamento indicou piso de erro com Chamfer médio de 0,0042 e F-score@0,05 mínimo de 0,976; sua saída permanece na estação e não foi copiada para o repositório.

4.1.2. Levantamento da literatura

A tabela a seguir resume os modelos considerados e a situação da instalação no GB10, que combina CPU aarch64, arquitetura sm_121 e CUDA 13.

Tabela 1. Modelos abertos de geração 3D texturizada a partir de imagem e situação da instalação no GB10

Modelo (organização, ano)	Licença	Saída	Situação no GB10
Hunyuan3D-2.0 [9] (Tencent, 2025)	Tencent Hunyuan 3D 2.0 Community	malha e textura UV RGB	instalado e medido
Hunyuan3D-2mini Turbo [9] (Tencent, 2025)	idem	malha e textura UV RGB	instalado e medido
Hunyuan3D-2 Turbo e FlashVDM [10] (Tencent, 2025)	idem	malha e textura UV RGB	instalado e medido
Hunyuan3D-2.1 [11] (Tencent, 2025)	Tencent Hunyuan 3D 2.1 Community	malha e materiais PBR	instalado com adaptações e medido
TRELLIS [12] (Microsoft, 2024)	MIT	gaussianas, campos de radiância ou malha	bloqueado: dependências sem versão aarch64
TRELLIS.2, sucessor do TRELLIS [12] (Microsoft, 2025)	MIT	malha O-Voxel e PBR	bloqueado: componente de acesso restrito
TripoSR [13] (Stability AI e Tripo AI, 2024)	MIT	malha com cor por vértice	instalado e medido
Stable Fast 3D [14] (Stability AI, 2024)	Stability AI Community	malha com UV, textura e material	bloqueado: pesos de acesso restrito
SPAR3D [15] (Stability AI, 2025)	Stability AI Community	nuvem de pontos e malha texturizada	bloqueado: pesos de acesso restrito
InstantMesh [16] (TencentARC, 2024)	Apache-2.0	malha com cor por vértice ou textura	instalado e medido

O TRELLIS depende de `spconv` e `cumm`, sem distribuição para linux-aarch64 no PyPI (`resultados/trellis_bloqueio_pypi.txt`). O TRELLIS.2 usa extensões CUDA próprias e o codificador DINOv3, de acesso restrito, e os pesos do Stable Fast 3D e do SPAR3D também exigem login no Hugging Face e aceite da licença, etapa ainda não realizada.

4.1.3. Avaliação experimental

Foram medidos seis modelos ou variantes. O Hunyuan3D-2.1 exigiu duas adaptações para aarch64 (GLB com materiais PBR escrito com trimesh, pois o Blender não tem pacote para essa plataforma, e rasterizador compilado à parte). Pelo custo de cerca de 4 min por objeto, o Hunyuan3D-2.0 na configuração padrão e o 2.1 foram medidos em subconjuntos de 4 e 2 objetos.

Tabela 2. Avaliação no GB10 com os modelos carregados (n = objetos avaliados; tempo mediano por objeto)

Modelo	n	Tempo (s)	Pico GPU (GB)	Chamfer ↓ / F@0,05 ↑	CLIP entrada / vistas novas ↑
TripoSR	13	0,9	4,2	0,042 / 0,904	0,849 / 0,809
InstantMesh	13	16,5	23,2	0,046 / 0,893	0,844 / 0,836
Hunyuan3D-2mini Turbo	13	65,8	13,6	0,035 / 0,930	0,893 / 0,859
Hunyuan3D-2 Turbo	13	68,8	14,6	**0,032 / 0,951**	0,907 / 0,854
Hunyuan3D-2.0 (padrão)	4	229,5	14,5	0,039 / 0,918	**0,933 / 0,884**
Hunyuan3D-2.1 (PBR)	2	242,9	13,8	0,032 / 0,949	0,883 / 0,878

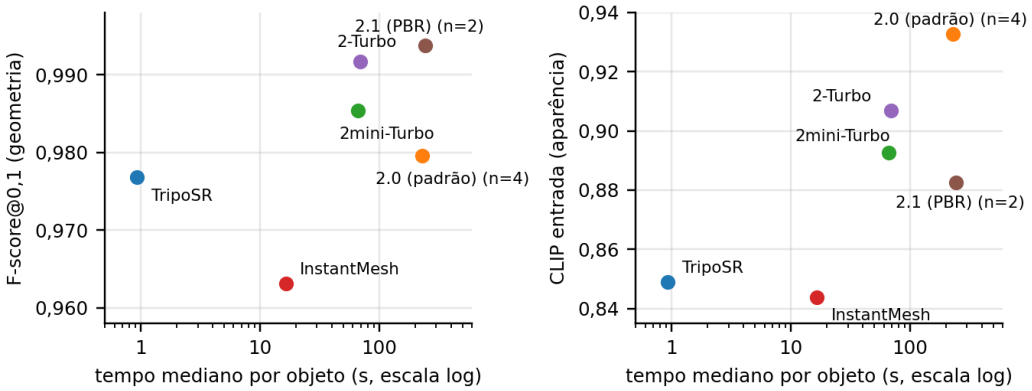


Figura 1. Qualidade em função do tempo mediano por objeto (escala logarítmica): à esquerda, F-score@0,1 da geometria; à direita, similaridade CLIP entre a entrada e a imagem renderizada do resultado; os rótulos 2-Turbo, 2mini-Turbo, 2.0 e 2.1 referem-se às variantes do Hunyuan3D, e n indica o número de objetos dos modelos medidos em subconjuntos



Figura 2. Comparação qualitativa numa vista não observada pelos modelos (azimute 210°): entrada, referência escaneada e resultado de cada modelo alinhado à referência; “não medido” indica objeto fora do subconjunto daquele modelo

4.1.4. Discussão e seleção

O Hunyuan3D-2 Turbo obteve a melhor geometria no conjunto completo (Chamfer 0,032; F-score@0,05 de 0,951 e F-score@0,1 de 0,992), e o Hunyuan3D-2.1 alcançou valores equivalentes nos dois objetos em que foi medido. O TripoSR foi cerca de 75 vezes mais rápido que o Hunyuan3D-2 Turbo (0,9 s contra 68,8 s), mas teve geometria inferior e aparência limitada à cor por vértice. Na configuração avaliada, o InstantMesh também produziu apenas cor por vértice e atingiu 23,2 GB de pico; seu F-score@0,05 caiu para 0,766 no subconjunto que inclui a águia e a tartaruga, contra 0,913 no ônibus e na caneca, queda concentrada na águia, cujas asas finas colapsam. Na aparência, o Hunyuan3D-2.0 com o módulo de textura completo obteve os maiores valores de CLIP (0,933 e 0,884), à custa de 229,5 s por objeto, e o CLIP do 2.1 ficou subestimado, porque o renderizador da avaliação não aplica sombreamento PBR. A comparação de tempos é apenas indicativa: os modelos turbo foram medidos com média de carga da CPU próxima de 39, e o 2.0 e o 2.1, com carga entre 3 e 4; a mesma configuração Turbo levou 28,7 s por objeto na série sequencial de otimização (7 objetos), com a máquina parcialmente livre, e 63,9 s na série intercalada (4 objetos), apresentada na seção seguinte.

Foi selecionado o **Hunyuan3D-2 Turbo** (forma destilada com FlashVDM, textura Paint-v2-0-Turbo e remoção de iluminação), pela melhor geometria medida no conjunto completo; pela textura UV real, requisito para boa aparência em RA; pelo pico de 14,6 GB, compatível com a coexistência com os serviços de produção; e pela licença, que permite o uso no Brasil, embora exclua a União Europeia, o Reino Unido e a Coreia do Sul e exija licença própria acima de 1

milhão de usuários mensais. O TripoSR (MIT) fica como alternativa de licença permissiva, e o Hunyuan3D-2.1 (PBR), como evolução futura com GPU dedicada.

4.2. Otimização do modelo integrado e desempenho

Esta etapa corresponde à Atividade 3 do Objetivo 1 (aplicativo otimizado e relatório de desempenho), na parte relativa ao modelo de geração. O objetivo foi reduzir o tempo de geração do Hunyuan3D-2 Turbo sem perda mensurável de qualidade, definir *presets* para o aplicativo e empacotar o pipeline para o servidor.

4.2.1. Variantes avaliadas

Partiu-se da configuração base (5 passos de difusão, FlashVDM, octree 384, limpeza e decimação para 40 mil faces com pymeshlab, textura de 2048 px e fp16), alterando cada fator isoladamente. Na série intercalada, com 4 objetos, todas as variantes de parâmetros foram executadas em sequência para cada objeto, sob as mesmas condições de ocupação; na série sequencial, com 7 objetos, cada variante teve rodada própria, o que permitiu incluir as variantes que exigem outros pesos.

Tabela 3. Série intercalada (4 objetos; razão = mediana, por objeto, do tempo da variante dividido pelo tempo da base)

Variante	Tempo mediano (s)	Razão vs. base	Pós-proc. da malha (s)	F@0,05 ↑	CLIP entrada / vistas novas ↑
base	63,9	1,00	33,1	0,949	0,917 / 0,837
pós-processamento rápido	**37,5**	**0,62**	**6,2**	0,950	0,908 / 0,833
octree 256	**41,0**	**0,64**	13,1	0,951	0,914 / 0,844
20 mil faces	63,8	0,99	33,3	0,948	0,915 / 0,843
10 mil faces	63,6	0,98	33,6	0,949	0,914 / 0,845
textura 1024 px	63,0	0,99	33,0	0,949	0,912 / 0,843
10 passos de difusão	67,1	1,10	32,9	0,948	0,913 / 0,864

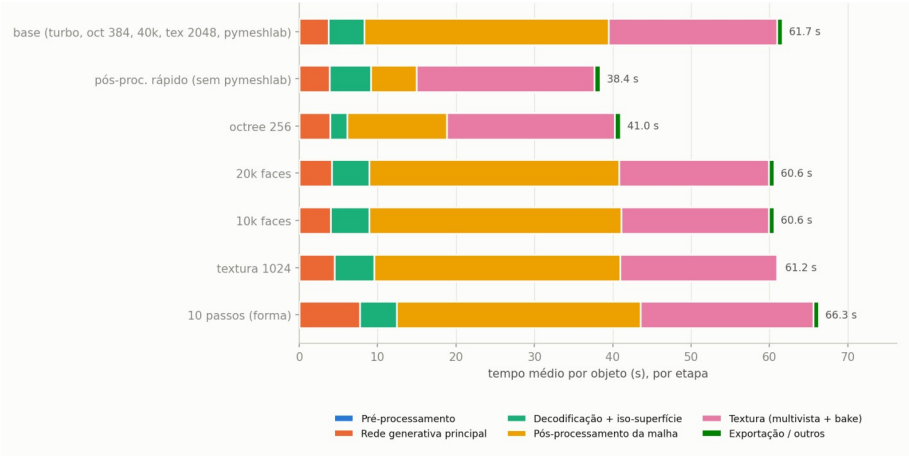


Figura 3. Tempo médio por objeto e por etapa de cada variante do Hunyuan3D-2 Turbo na série intercalada (4 objetos)

Os principais resultados foram os seguintes:

- O **FlashVDM** mostrou-se indispensável: sem ele, a etapa de forma passou de 3,5 s para 115,7 s (medido em 3 objetos), sem ganho de geometria.
- O **pós-processamento rápido** (descarte de componentes com menos de 1 % da área do maior e decimação com `fast-simplification`) trouxe a maior economia: a limpeza caiu de 33,1 s para 6,2 s e o total, cerca de 38 % (razão mediana de 0,62 na série intercalada e de 0,55 na sequencial), com geometria inalterada e CLIP de entrada 0,009 menor.
- A **octree 256** reduziu o total em cerca de 36 % sem perda mensurável de qualidade.
- O orçamento de faces e a textura de 1024 px praticamente não alteraram o tempo, mas reduziram o GLB bruto (de 3,7 MB para 2,8 MB com 10 mil faces e para 1,9 MB com textura de 1024 px).
- **10 passos** custaram 10 % a mais de tempo, sem ganho geométrico, e elevaram o CLIP das vistas novas de 0,837 para 0,864.
- O **módulo de textura completo** elevou o tempo da textura de 12,0 s para 51,7 s na série sequencial, mas foi a variante que mais melhorou as partes ocultas (CLIP vistas novas de 0,846 para 0,893, contra 0,864 com 10 passos na mesma série).
- A **forma em bf16** teve qualidade equivalente e não trouxe ganho de tempo, e o fp16 foi mantido; o `torch.compile` não foi avaliado.

4.2.2. Presets e pipeline empacotado

Foram definidos três *presets* sobre um único modelo de forma (Turbo com FlashVDM e 5 passos) e com o pós-processamento rápido, o que permite manter

os modelos carregados no servidor: **rápido** (octree 256, 10 mil faces e textura de 1024 px), **padrão** (octree 384, 20 mil faces e textura gerada em 2048 px e entregue em 1024 px) e **alto** (octree 384, 40 mil faces, textura de 2048 px e módulo de textura completo). Os *presets* foram medidos de ponta a ponta com o pipeline empacotado (remoção de fundo, geração, pós-processamento e miniatura) nos 13 objetos do GSO e em 6 fotografias reais, com os serviços de produção ativos.

Tabela 4. Presets medidos no pipeline empacotado (19 entradas; GLB com textura PNG, anterior à correção descrita adiante)

Preset	Tempo mediano GSO (s)	Tempo mediano fotos reais (s)	Faixa (s)	Pico GPU (GB)	GLB mediano (MB)
rápido	23,9	24,0	21,9–27,3	14,5	1,65
padrão	30,4	29,8	26,8–76,6	14,5	2,01
alto	82,1	56,7	53,5–104,5	14,5	6,14

A qualidade geométrica foi a mesma nos três *presets* (F-score@0,05 de 0,951 no GSO); o CLIP de entrada foi de 0,890, 0,894 e 0,922 e o das vistas novas, de 0,840, 0,851 e 0,876 (rápido, padrão e alto). A carga a frio do pipeline levou 115 s e o aquecimento, 79 s (107 s e 118 s no *preset* alto), o que justifica manter os modelos residentes no servidor. A dispersão dos tempos reflete a variação de ocupação da GPU, como mostra a decomposição por etapa a seguir.

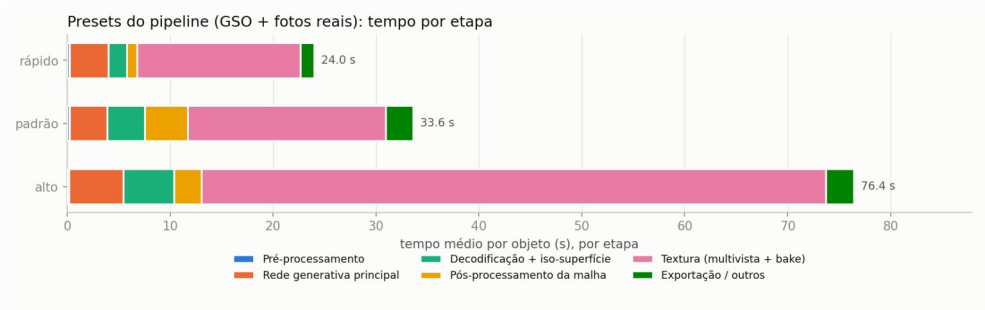


Figura 4. Tempo médio por objeto e por etapa dos três presets no pipeline empacotado; no preset alto predomina a difusão multivista do módulo de textura completo

O pós-processamento para o celular assenta o objeto em y = 0 e o centraliza em x e z, corrigindo o script de ajuste de 2025, que deslocava o eixo Z pelo centro do eixo Y; escala a maior dimensão para 0,30 m, reduz a textura com glTF-Transform e valida o arquivo com o Khronos glTF-Validator. Por requisito do aplicativo, o GLB não usa Draco, meshopt nem KTX2. O pipeline foi empacotado na classe `Gen3DPipeline`, que carrega os modelos uma única vez, e na imagem Docker `cerise26_gen3d:latest`.

4.2.3. Correção da compressão da textura e limitações

O teste ponta a ponta do servidor, em 26/10/2026, revelou que a recodificação da textura em JPEG não ocorreu nas rodadas da subseção anterior: o pós-processamento omitia a opção `--formats` ao chamar `gltf-transform jpeg`, e, na versão 4.5 da ferramenta, o padrão recomprime apenas texturas que já são JPEG. Os tamanhos de GLB da tabela de *presets* estão, portanto, superestimados; as conclusões de tempo e de geometria não se alteram. Com a correção (`--formats "*"`), aplicada no servidor e no repositório, a mesma fotografia de teste passou de 2,97 MB para 0,72 MB (−76 %) no *preset* padrão e de 2,60 MB para 0,44 MB (−83 %) no rápido, sem alteração mensurável no tempo de geração. Falta reconstruir a imagem Docker com a correção e refazer a rodada de *presets*, de preferência numa janela ociosa da GPU.

4.3. Validação dos objetos 3D gerados

Esta etapa corresponde à Atividade 4 do Objetivo 1 (relatório de validação e resultados experimentais). Além das métricas já apresentadas, o pipeline otimizado foi validado por testes funcionais automatizados, pelo carregamento nos visualizadores usados pelo aplicativo e por análise qualitativa, incluindo os casos de falha.

4.3.1. Testes funcionais automatizados

A suíte `validation/test_pipeline.py` (pytest) gera modelos a partir de três entradas em cada *preset* (duas imagens do GSO e uma fotografia real com fundo, que exercita a remoção de fundo) e verifica cada GLB final segundo os critérios da tabela a seguir, além de dois testes sem GPU.

Tabela 5. Verificações automatizadas do pipeline empacotado (9 GLBs: 3 entradas em 3 presets)

Verificação	Critério	Resultado
Validade do arquivo	Khronos glTF-Validator sem erros	aprovado
Material	material glTF (`pbrMetallicRoughness`) com `baseColorTexture`	aprovado
Compatibilidade com o app	sem Draco, meshopt ou KTX2; textura JPEG ou PNG	aprovado
Orçamento de faces	até 10 mil, 20 mil e 40 mil faces conforme o preset	aprovado
Escala e posição	maior dimensão de 0,30 m, base em y = 0, centrado em x e z	aprovado
Tempo e tamanho	até 60, 90 e 180 s; até 3, 4 e 10 MB	aprovado
Total	rápido e padrão: 38 testes em 189,7 s; alto: 20 testes em 135,2 s	58 de 58 aprovados

Como aceitava textura JPEG ou PNG e teve os limites de tamanho calibrados com a rodada de *presets*, a suíte não detectou o defeito de recodificação da textura; recomenda-se exigir textura JPEG e recalibrar os limites após a nova rodada.

4.3.2. Carregamento nos visualizadores

Sete GLBs finais (cinco do *preset* padrão, um do rápido e um do alto) foram abertos em Chromium sem interface gráfica (Playwright) com o `<model-viewer>` e com o `GLTFLoader` do three.js. Os sete carregaram nos dois visualizadores, sem erros e sem extensões glTF, com uma malha, um material com mapa de cor, a contagem de triângulos do *preset*, maior dimensão de 0,30 m e base em y = 0 (`resultados/viewer/viewer_report.json`). Esses arquivos são anteriores à correção da textura; o GLB corrigido foi validado da mesma forma no teste do servidor.



Figura 5. Quatro dos GLBs finais carregados no model-viewer em Chromium sem interface gráfica: ônibus e pato no preset padrão, caneca no rápido e robô no alto

4.3.3. Análise qualitativa e casos de falha

A figura a seguir mostra, no *preset* padrão, seis fotografias reais com licença CC0 obtidas no Wikimedia Commons (quatro peças do acervo aberto do Metropolitan Museum, um bule fotografado no Victoria and Albert Museum e um pato de borracha sobre fundo complexo) e os modelos gerados. A remoção de fundo funcionou inclusive sobre a grade metálica atrás do pato, e o pipeline produziu formas plausíveis para estatuetas cerâmicas, bem diferentes dos objetos domésticos do GSO; na grade equivalente com oito objetos do GSO, mantida no repositório, a textura reproduz detalhes finos e a geometria preserva estruturas finas, como as asas da águia.

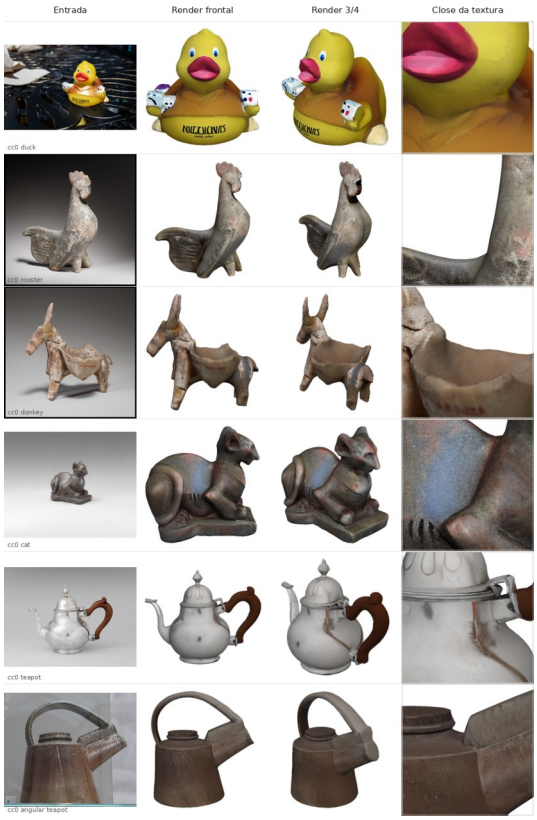


Figura 6. Grade qualitativa do preset padrão nas seis fotografias reais com licença CC0: entrada com fundo, vista frontal, vista de três quartos e detalhe da textura

Foram identificados os seguintes casos de falha e limitações:

- **Partes ocultas**: as faces não visíveis na foto tendem a sair mais lisas e acinzentadas, e regiões escuras ou finas podem sair pretas ou com lacunas; o módulo de textura completo do *preset* alto atenua o problema (CLIP vistas novas de 0,876, contra 0,851 no padrão).
- **Materiais metálicos**: no bule de prata, os reflexos ficam incorporados ao albedo, porque o Hunyuan3D-2 não gera materiais PBR.
- **Objetos compostos**: os dados de jogo segurados pelo pato saem como peças parcialmente destacadas, e a limpeza de componentes pequenos pode remover partes legítimas.
- **Limitações da avaliação**: o CLIP mede semelhança visual, e não exatidão métrica, e não houve teste com fotografias capturadas pela câmera do celular.

4.4. Servidor de geração 3D para múltiplas requisições

Esta etapa corresponde à Atividade 5 do Objetivo 2 (análise da arquitetura do servidor e identificação de gargalos no atendimento a múltiplas requisições) e às metas de otimização para solicitações simultâneas e de escalabilidade e

confiabilidade. O servidor executa o modelo selecionado (Hunyuan3D-2 Turbo [9]) na estação DGX Spark, compartilhada com os serviços de IA em produção.

4.4.1. Arquitetura de 2025 e gargalos identificados

O servidor de 2025 era um aplicativo Flask em modo de depuração, cuja rota `POST /` executava `time.sleep(67)` para simular o processamento e devolvia sempre o mesmo GLB. Os gargalos identificados foram o processamento dentro da requisição, com a conexão presa durante a geração; a ausência de fila e de limite de concorrência; a falta de estado persistente, com arquivos de nomes fixos; e a ausência de validação de entrada, autenticação, TLS, métricas e registros. Como a espera simulada não exercita a GPU, a arquitetura foi reproduzida com o modelo real em dois modos: sem trava, como no código original, e com trava global, a correção mínima que serializa o uso da GPU, mas mantém as conexões presas. Sem trava, a partir de dois clientes simultâneos todos os pedidos falharam com HTTP 500, porque as chamadas concorrentes compartilham o *scheduler* da difusão, cujo índice de passo é estado mutável.

4.4.2. Arquitetura implementada em 2026

A nova arquitetura separa a recepção dos pedidos do processamento na GPU, conforme a figura a seguir. A API, em FastAPI [17], nunca acessa a GPU: valida e normaliza a foto (até 15 MB, com metadados e GPS removidos), registra o *job* e responde `202` em milissegundos, com identificador, posição e estimativa de tempo. Os *jobs* ficam numa fila persistente em SQLite no modo WAL [18], com reserva atômica e ordem FIFO com prioridade por *preset* e envelhecimento (deslocamentos de 0, 30 e 90 s para os *presets* rápido, padrão e alto). Um processo de GPU carrega o modelo uma vez e processa um *job* por vez (configuração de produção) ou dois *slots* com pesos compartilhados e travas por componente; aplica tempo limite, devolve à fila o *job* interrompido por reinício ou falta de memória e só aceita novo *job* com ao menos 10 GB livres. Completam a arquitetura o cache por *hash* SHA-256 da imagem, das opções e da versão do pipeline, a coalescência de pedidos idênticos, os limites de fila (32 *jobs*, com resposta 429), por cliente e de taxa, a retenção máxima de 7 dias e a implantação em Docker Compose com reinício automático, acessível apenas pela interface local da estação.

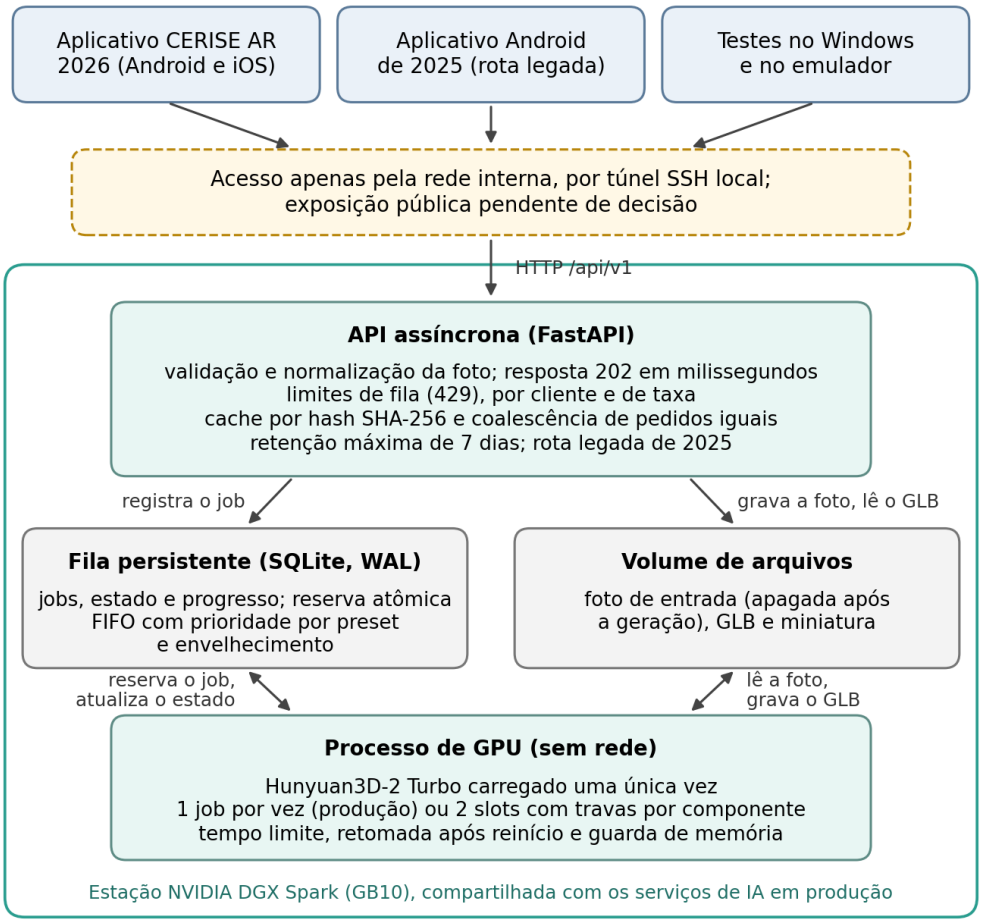


Figura 7. Arquitetura de 2026: API assíncrona, fila persistente em SQLite, volume de arquivos e processo de GPU com o modelo carregado; acesso apenas pela rede interna, com a exposição pública pendente de decisão

4.4.3. Testes de carga e resultados

Os testes foram executados em 26/10/2026 com os serviços de produção ativos na mesma GPU. O cliente de carga (asyncio e httpx) rodou no próprio hospedeiro, sem latência de rede, com 64 imagens sem repetição dentro de cada cenário. O cenário principal consistiu em três *jobs* em sequência com 1 cliente e em rajadas de 2, 4, 8 e 16 clientes simultâneos, cada um enviando um *job*, totalizando 33 *jobs* do *preset* rápido por configuração, com sondas de leitura paralelas para medir a responsividade.

Tabela 6. Resumo por configuração (33 jobs do preset rápido: 3 em sequência com 1 cliente e rajadas de 2 a 16 clientes; latência do envio ao GLB baixado)

Configuração	Atendidos	Latência p50 / p95 (s)	p95 com 16 clientes (s)	Vazão com 16 clientes (jobs/min)	Leituras com GPU ocupada, p95 / p99 (ms)
2025, sem trava	3/33	12,3 / 12,5 (só 1 cliente)	sem sucesso	0	102,4 / 213,0
2025, com trava	33/33	52,1 / 199,0	210,2	4,37	123,4 / 228,6
2026, fila e 1 *worker*	33/33	49,6 / 174,1	184,1	4,99	6,6–6,7 / 8,3–8,6
2026, fila e 2 *slots*	33/33	46,5 / 155,6	164,4	5,63	6,6–6,7 / 9,0–9,2

A taxa de erro caiu de 90,9 % (30 de 33 pedidos na arquitetura de 2025 sem trava) para zero. Com 16 clientes, a fila com 1 *worker* reduziu a latência p95 em 12 % e elevou a vazão em 14 % em relação à versão síncrona com trava; com 2 *slots*, os ganhos foram de 22 % e 29 %. A espera passou a ocorrer na fila, sem conexão presa, e as rotas de leitura responderam de 15 a 19 vezes mais rápido que o Flask no p95, inclusive sob *polling* de cerca de 50 req/s (9.569 sondas, sem erros).

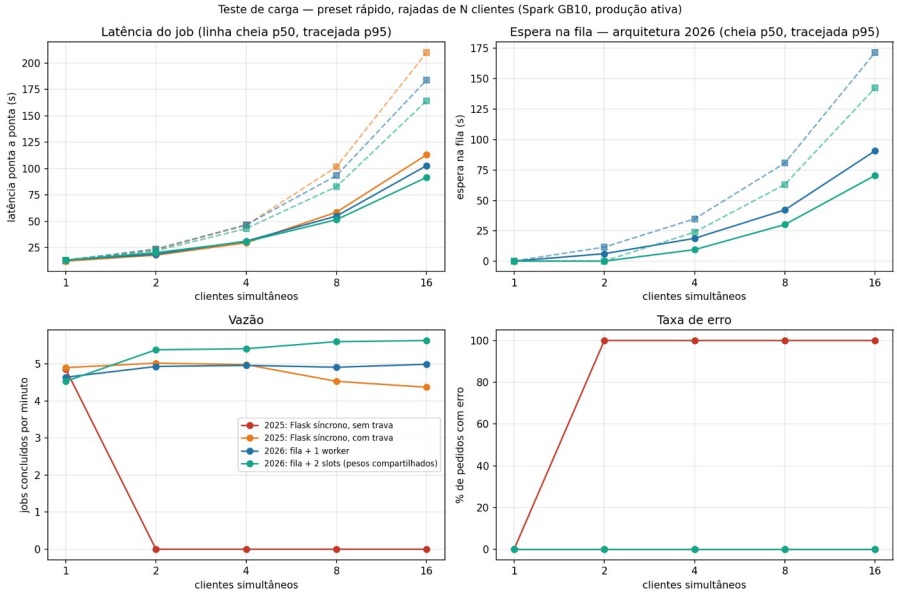


Figura 8. Latência ponta a ponta (p50 e p95), espera na fila, vazão e taxa de erro em função do número de clientes simultâneos para as quatro configurações

Tabela 7. Cenários adicionais na configuração de produção (1 *worker*)

Cenário	Pedidos	Atendidos	Execuções na GPU	Resultado
Reenvio de imagens já processadas (cache)	10	10	0	0,02 s por pedido
8 clientes com a mesma imagem nova (coalescência)	8	8	1	12,31 s (p50)
Misto com prioridade por *preset* (12 rápidos, 4 padrão)	16	16	16	p50: rápido 66,0 s; padrão 131,9 s
Misto em FIFO pura (mesmos pedidos)	16	16	16	p50: rápido 99,7 s; padrão 85,1 s
16 clientes com fila máxima de 8	16	9	9	7 recusados com 429 em 0,32 s

A prioridade reduziu em 34 % a latência mediana dos *jobs* rápidos, com média geral praticamente inalterada (86,0 s para 82,9 s); a espera máxima de um *job* padrão aumentou 29,9 s, dentro do envelhecimento de 30 s configurado. O limite de fila converteu esperas de até 192 s em recusas imediatas que o aplicativo pode exibir ao usuário.

4.4.4. Discussão e limitações

O gargalo dominante é a própria GPU: com 2 *slots*, a execução mediana de cada *job* subiu de 11,9 s para 21,0 s, enquanto a espera nas travas foi de 0,05 a 0,2 s por componente, razão pela qual dois *slots* rendem só 13 % mais vazão. A capacidade sustentada medida é de cerca de 5 *jobs*/min no *preset* rápido, e um objetivo de p95 de 1 min comporta rajadas de 4 a 5 pedidos (p95 de 46,7 s com 4 clientes e de 93,5 s com 8). O pico por *job* foi de 18,9 GB alocados, abaixo do teto de 24 GB. Os 18 testes automatizados do servidor, com motor simulado, foram aprovados, e, em dois desligamentos não planejados da estação, os contêineres retornaram automaticamente com a fila preservada. Como limitações, as configurações foram medidas em sequência numa máquina compartilhada; o cenário misto não incluiu o *preset* alto; a escala para várias GPUs foi apenas projetada; o monitoramento das métricas não foi ligado; e o servidor não foi exposto publicamente, havendo uma proposta de exposição documentada no repositório, que aguarda decisão.

Código e evidências da geração 3D e do servidor: repositório cerise-2026-gen3d (privado)

4.5. Aplicativo CERISE AR com geração de objetos 3D texturizados

Esta etapa corresponde à Atividade 2 do Objetivo 1: integrar o modelo selecionado ao aplicativo e entregar uma versão funcional com geração de objetos 3D texturizados. O aplicativo CERISE AR, reescrito em 2026 em React Native com Expo [19] para Android e iOS, passou a permitir que o usuário fotografe um objeto, envie a imagem ao servidor, acompanhe a fila e o

processamento, baixe o modelo GLB texturizado e o visualize em 3D e em realidade aumentada.

4.5.1. Implementação

Aplicativo e servidor foram desenvolvidos em paralelo sobre um contrato REST fixado previamente (`/api/v1`), com um servidor simulado reproduzindo o contrato enquanto o real não estava disponível. No aplicativo, destacam-se o cliente da API com envio multipart, progresso, tempo limite e cancelamento; o gerenciador de *jobs* com *polling*, recuo exponencial em falhas de rede e retomada ao reabrir o aplicativo; a leitura da caixa envolvente do GLB para normalizar a escala na RA; o visualizador 3D sem rede, com o `<model-viewer>` [20] numa WebView; e a RA com ViroReact [21], aberta somente após a confirmação de suporte do ARCore/ARKit. No servidor, o motor entrega o GLB no formato exigido pelo aplicativo (glTF 2.0 [22], apoiado em y = 0, maior dimensão de 0,30 m, textura JPEG e sem Draco, meshopt ou KTX2), e a rota legada de 2025 passou a devolver um modelo efetivamente gerado. Num teste a partir de um computador externo, por túnel SSH, uma foto CC0 no *preset* padrão produziu o GLB em 16,1 s (15,3 s de GPU), aprovado no Khronos glTF-Validator sem erros nem avisos; esse teste revelou o defeito da textura descrito na subseção de otimização do modelo.

4.5.2. Validação ponta a ponta com o servidor real

A integração foi validada no emulador Android 16 (API 36, x86_64 com tradução ARM), que alcançou o servidor por encaminhamento SSH local, sem exposição pública. Foram usadas fotografias reais CC0 de objetos (pato de borracha, bule de prata e estatuetas de gato e de burro), escolhidas pelo seletor de fotos do sistema, e não fotografias tiradas pela câmera de um celular. Para observar a fila, um segundo cliente enviou um *job* do *preset* alto imediatamente antes de um pedido do aplicativo, e a foto do pato foi reenviada para exercitar o cache.

Tabela 8. Tempos medidos pelo aplicativo no emulador contra o servidor real (com textura e remoção de fundo; entre parênteses, o tamanho do GLB)

Objeto e *preset*	Envio (s)	Fila: app / servidor (s)	Geração no app / execução no servidor (s)	*Download* (s)	Total (s)
Pato, rápido (340 KiB)	0,41	1,25 / 0,24	11,19 / 11,20	0,67	13,5
Burro, rápido (398 KiB)	0,49	1,35 / 0,20	11,24 / 11,26	0,45	13,5
Bule, padrão (622 KiB)	0,66	1,36 / 0,09	14,42 / 14,73	0,61	17,1
Gato, alto (1,44 MiB)	0,39	1,21 / 0,22	29,04 / 30,17	0,90	31,5
Gato, padrão, atrás de 1 *job* alto	0,40	28,25 / 27,85	15,81 / 15,88	0,46	44,9
Pato, rápido, reenvio (cache)	0,29	0,28 / 0	0 / 0	0,43	1,0

Sem concorrência, o tempo percebido variou de 13,5 s (rápido) a 31,5 s (alto), dominado pela execução na GPU (83 % a 96 % do total); envio e *download* somaram no máximo 1,3 s. Com os mesmos parâmetros, as etapas na GPU levaram de um terço a metade do tempo medido na rodada de *presets*, feita horas antes e sob outra ocupação da GPU compartilhada (no *preset* alto, a difusão multivista levou 15,9 s, contra 33 a 54 s), o que confirma que os tempos da tabela de *presets* são limites superiores. A fila de 1,2 a 1,4 s registrada no aplicativo é artefato da resolução do *polling*, pois o servidor registrou de 0,09 a 0,24 s. Com um *job* alto à frente, a espera (27,85 s) praticamente igualou a execução desse *job* (28,2 s), como esperado para um único processo de GPU. Todos os GLBs entregues (seis gerados e o reenvio atendido pelo cache) tinham textura JPEG, maior dimensão de 0,30 m e nenhuma extensão de compressão. O teste motivou ajustes em JavaScript: o *download* de um GLB de 398 KiB levava 5,0 s e, após deixar de gravar a lista de modelos a cada evento de progresso e baixar a miniatura em paralelo, caiu para 0,45 s, com o total passando de 18,0 s para 13,5 s (mudanças aplicadas juntas); além disso, a exclusão de um modelo no aplicativo passou a apagar a cópia no servidor.

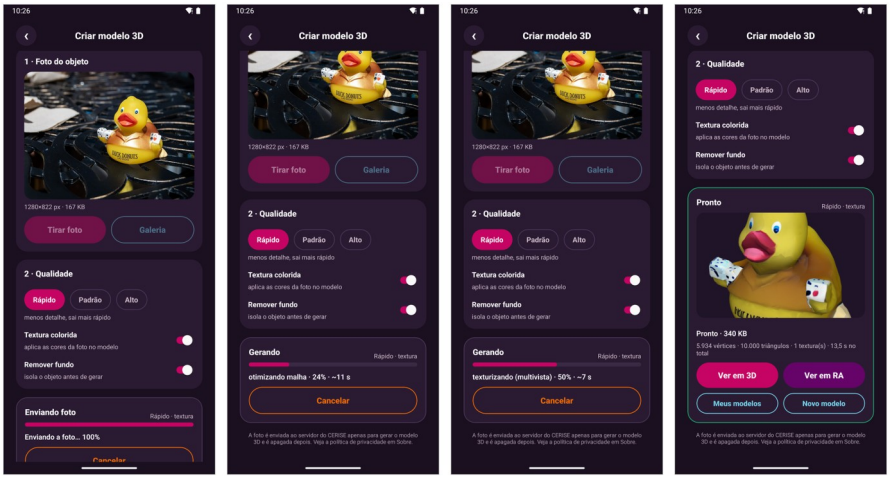
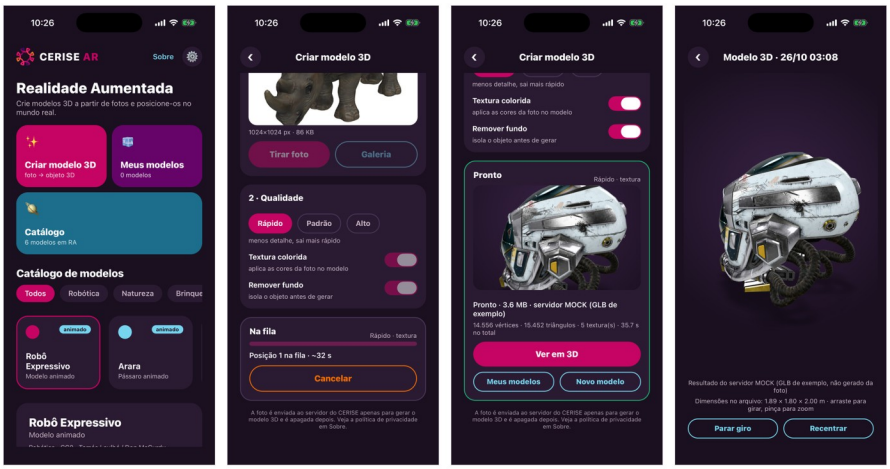


Figura 9. Fluxo de geração no Android com o servidor real: envio da foto, geração da malha, texturização multivista e resultado com o GLB texturizado baixado em 13,5 s no total

4.5.3. iOS e limitações

O *build* Release para o Simulador iOS foi gerado num Mac mini M4 (Xcode 26.6), com pacote de 41 MB, executado no simulador do iPhone 17 Pro. Como a biblioteca de RA não tem versão para simulador, esse *build* exclui o módulo de RA, e o fluxo foi validado apenas contra o servidor simulado, com um GLB de exemplo, como ilustrado abaixo. O iOS não foi retestado contra o servidor real, pois o Mac utilizado não alcança a estação; as mudanças da integração são apenas de JavaScript e valem para as duas plataformas.



(a) Home (b) Na fila (c) Pronto (d) Visualizador 3D

Figura 10. Simulador iOS no iPhone 17 Pro: tela inicial, pedido na fila, resultado e visualizador 3D com o GLB de exemplo devolvido pelo servidor simulado

A validação com o servidor real ocorreu apenas no emulador e pela rede interna, sem aparelho físico, rede móvel ou fotografias da câmera. A RA dos modelos gerados não foi validada, porque o ARCore não carrega no emulador x86_64 com bibliotecas ARM traduzidas e o simulador iOS não executa a biblioteca de RA; essa validação exige aparelho físico. A estimativa de espera exibida na fila foi superestimada para o *preset* alto (1 min 21 s para 28 s reais), por depender de um valor inicial ainda não recalibrado. Por fim, o uso fora da rede interna depende da exposição pública do servidor, ainda não decidida.

4.6. Adequação do aplicativo para publicação na Play Store

Esta etapa corresponde à Atividade 6 do Objetivo 2, que prevê a publicação do aplicativo na Play Store. O trabalho concentrou-se na adequação técnica e normativa e na preparação e verificação do pacote de publicação. A publicação não foi realizada: as etapas na conta de desenvolvedor do Play Console ainda não foram executadas, e dois pré-requisitos, a publicação da política de privacidade e a exposição pública do servidor, aguardam decisão.

4.6.1. Adequação técnica e normativa

A configuração nativa foi declarada nos arquivos do Expo e em *plugins* próprios, de modo que os projetos Android e iOS são gerados de forma reprodutível. A versão 1.1.0 (código 110) declara apenas as permissões de câmera e de internet e bloqueia no manifesto as permissões sensíveis acrescentadas por bibliotecas; além das duas declaradas, restam no APK apenas a de estado da rede e uma permissão interna do AndroidX, ambas de bibliotecas; a galeria usa o seletor de fotos do sistema, que dispensa permissão de mídia. As arquiteturas nativas foram restritas a `armeabi-v7a` e `arm64-v8a`, as únicas distribuídas pelo ViroReact, o que eliminou o encerramento ao iniciar em

dispositivos x86_64 e reduziu o AAB de 75,84 MB para 55,16 MB (−27 %) e o APK universal de 116,82 MB para 83,30 MB (−29 %). O tráfego sem criptografia foi proibido, exceto para endereços locais de teste, e os artefatos foram assinados com uma chave de *upload* dedicada, mantida fora do controle de versão. Após a integração com o servidor real, o AAB e o APK foram recompilados e verificados, com os resultados a seguir.

Tabela 9. Verificações no AAB e no APK *release* finais do CERISE AR 1.1.0

Requisito	Resultado medido
Nível de API	targetSdkVersion 36, compileSdk 36, minSdk 24
Páginas de memória de 16 KB [23]	30/30 bibliotecas de 64 bits alinhadas a 16 KB no AAB e no APK; zipalign -c -P 16 bem-sucedido
Permissões no APK	CAMERA, INTERNET, ACCESS_NETWORK_STATE e permissão interna do AndroidX
Componentes exportados	apenas a atividade principal e um receptor do AndroidX protegido por permissão
Assinatura	APK e AAB assinados com o certificado de *upload* do CERISE/EMC/UFG
Tamanho	AAB de 55,16 MB; *download* estimado de 35,65 MB (arm64-v8a) e 34,09 MB (armeabi-v7a)

Na parte normativa, foram preparados a política de privacidade em português, conforme a LGPD [24], com o uso da câmera e das fotos, o envio ao servidor apenas para gerar o modelo, a exclusão em até 7 dias (ou imediata quando o usuário exclui o modelo) e a ausência de anúncios e de rastreamento; as respostas do formulário de segurança dos dados, da classificação etária IARC e do público-alvo (13 anos ou mais); a remoção, na versão de loja, de modelos de personagens de franquias, em atenção à política de propriedade intelectual; a restrição das atualizações remotas ao código JavaScript; a ficha da loja, com ícone 512×512, recurso gráfico 1024×500 e sete capturas de tela, três delas do teste com o servidor real; e um guia de publicação com as etapas de teste interno, teste fechado com 12 testadores por 14 dias (exigido de contas pessoais novas) e produção.



Figura 11. Pacote gráfico da Play Store: recurso gráfico 1024×500, ícone 512×512 e sete capturas de tela do emulador, das quais as de números 2, 3 e 7 provêm do teste com o servidor 3D real

4.6.2. Situação e pendências

O pacote está pronto e verificado, mas o aplicativo não foi publicado, razão pela qual a atividade é considerada parcialmente concluída. Permanecem pendentes a criação do aplicativo no Play Console, o envio do AAB ao teste interno, o teste fechado e os formulários; a publicação da política de privacidade no endereço público previsto, que respondia com erro 404 em 26/10/2026; e a exposição pública do servidor de geração 3D, sem a qual a revisão de produção não deve ser solicitada. Também devem ser verificadas a disponibilidade do nome de pacote e a cópia de segurança da chave de *upload*. No iOS, a compilação assinada e o envio ao TestFlight foram documentados, mas não executados.

Código e evidências do aplicativo: repositório cerise-ar, branch feat/2026-gen3d-playstore (privado)

4.7. Geração e documentação do novo dataset do braço robótico

Esta etapa corresponde à Atividade 7 do Objetivo 3: gerar, com um braço robótico virtual, um conjunto de dados estruturado e documentado para o modelo de pose (Atividades 8 e 9) e para o gêmeo digital (Atividades 11 e 12). Optou-se pelo braço virtual servo6dof, cujo pacote ROS 2 (descrição, controladores, configuração do MoveIt 2 e câmeras) foi fornecido pelo Prof. Dr. Alisson Assis Cardoso e adaptado para um estúdio fotográfico no simulador. O braço possui cinco juntas de revolução (`joint_1` a `joint_5`) e uma junta de garra (`joint_6`).

4.7.1. Metodologia de geração

As poses seguem um plano fixo e reprodutível (semente 20260909) com 5.000 configurações sorteadas uniformemente em ±90° para as juntas do braço e em [−40°; 30°] para a garra, descontada uma folga de 5 % em cada extremo. A simulação usou ROS 2 Humble [25] e Ignition Gazebo Fortress [26] em contêineres na estação DGX Spark, com renderização por GPU sem interface gráfica. Para cada pose, o script comanda a trajetória, aguarda o assentamento, descarta quadros antigos e grava, das câmeras frontal e diagonal, imagens PNG de 1280×720 pixels de RGB, profundidade (em milímetros) e segmentação (braço, chão e fundo), além de um JSON com as juntas comandadas e as efetivamente alcançadas (`juntas_reais`, lidas de `/joint_states` no instante da captura). A geração foi paralelizada em oito instâncias do Gazebo, com retomada automática. A documentação segue o formato de *datasheet* de Gebru et al. [27] (`docs/DATASET.md`), com uma amostra de 8 poses versionada no repositório; o *datasheet* recomenda uma divisão determinística de 4.000, 500 e 500 poses, e as avaliações seguintes adotaram, nas mesmas proporções, a divisão aleatória por pose definida na etapa de estimação de pose.

4.7.2. Resultados

A geração foi concluída com as 5.000 poses do plano, totalizando 30.000 imagens e 2,51 GB, conforme a composição a seguir.

Tabela 10. Composição do dataset por modalidade e câmera (5.000 poses; tamanhos medidos em disco)

Modalidade	Câmera frontal	Câmera diagonal	Total
RGB (PNG 8 bits)	0,60 GB (116 KB/img.)	0,50 GB (98 KB/img.)	1,10 GB
Profundidade (PNG inteiro, mm)	0,24 GB (47 KB/img.)	1,05 GB (206 KB/img.)	1,30 GB
Segmentação (rótulos 0/1/2)	0,05 GB (10 KB/img.)	0,05 GB (11 KB/img.)	0,11 GB

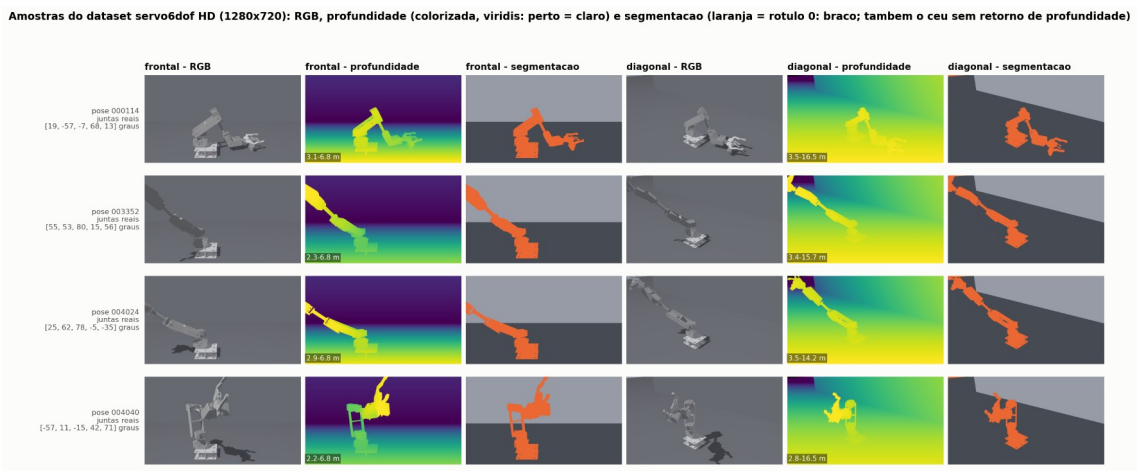


Figura 12. Amostras do dataset nas duas câmeras: imagem RGB, profundidade colorizada e mapa de segmentação, com o rótulo do braço destacado em laranja

A verificação completa decodificou as 30.000 imagens sem nenhum problema: todas têm 1280×720 pixels e o modo de cor esperado, as 10.000 máscaras contêm exatamente os rótulos {0, 1, 2}, não há arquivos órfãos nem poses faltantes e não há quadro RGB duplicado entre poses distintas, o que indica ausência de quadros desatualizados. Os principais indicadores estão reunidos na tabela a seguir.

Tabela 11. Indicadores de integridade, distribuição e geração do dataset

Indicador	Valor medido
Imagens verificadas sem problema	30.000 de 30.000
PNGs órfãos / faltantes / RGB duplicados	0 / 0 / 0
Área média ocupada pelo braço (frontal / diagonal)	8,8 % / 6,0 % da imagem
Desvio-padrão dos ângulos comandados (juntas 1 a 5)	45,9° a 46,7° (uniforme: 46,8°)
Cobertura dos pares de juntas (grade 20×20)	100 % nos 10 pares
Maior correlação de Pearson entre juntas (em módulo)	0,024
Poses com juntas reais diferentes das comandadas (> 0,01 rad)	34,1 %
Duração da geração (8 instâncias) e vazão média	3,64 h; 0,38 pose/s

A distância média ao vizinho mais próximo no espaço de juntas normalizado (0,1265) é praticamente igual à de uma amostra uniforme de mesmo tamanho (0,1262), o que confirma a cobertura homogênea ilustrada abaixo.

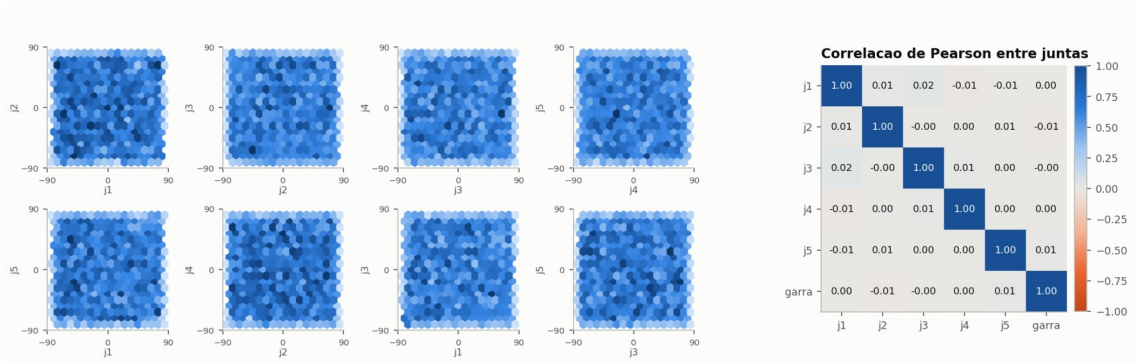


Figura 13. Cobertura do espaço de juntas em oito dos dez pares de juntas e matriz de correlação de Pearson entre as juntas

A divergência entre juntas reais e comandadas concentra-se em `joint_1` a `joint_3` (19,8 %, 26,9 % e 21,9 % das poses). A cinemática direta do modelo digital explica a maior parte dos casos: 19,7 % das configurações comandadas levariam algum elo abaixo do chão, e todas elas divergem, pois o braço é barrado pelo chão e o JSON registra a posição em que parou. As imagens confirmam que o rótulo correto é a junta real: em 670 imagens de poses divergentes, o IoU médio entre a silhueta do modelo e a máscara do simulador foi de 0,930 com as juntas reais e de 0,547 com as comandadas.



Figura 14. Divergência entre juntas reais e comandadas por junta e em função da altura mínima do braço na configuração comandada

4.7.3. Discussão e limitações

O dataset é íntegro, uniforme e documentado, mas o campo `juntas_reais` deve ser adotado como rótulo, e cerca de 2 % das imagens (estimativa sobre 1.000 poses) têm silhueta incompatível com as juntas registradas e devem ser filtradas por IoU. Na segmentação, o braço recebe o rótulo 0, compartilhado com regiões sem objeto, e sua máscara deve ser obtida como rótulo 0 com profundidade válida. O Gazebo Fortress não aplica as juntas *mimic* da garra, cuja geometria nas imagens difere parcialmente do modelo cinemático. O domínio é sintético e fixo (duas câmeras, iluminação, fundo e materiais constantes), e a transferência para o braço físico exigirá randomização de domínio [28] ou ajuste fino com imagens reais. Os resultados sobre o braço referem-se à versão do dataset com o braço em cinza, preservada integralmente na estação. Ao final do período foi gerada uma segunda versão, com as mesmas 5.000 poses (mesmos identificadores e ângulos) e o braço renderizado na cor laranja, para aumentar o contraste com o fundo (30.000 imagens, 2,4 GB); essa versão ainda não foi usada em treino ou avaliação e seus resultados não fazem parte deste relatório.

4.8. Aplicação do modelo de estimação de pose de 2025 ao novo dataset

Esta etapa corresponde à Atividade 8 do Objetivo 3: obter resultados quantitativos do modelo de detecção de pose do ano anterior sobre o novo conjunto de dados, descrito na subseção anterior. O modelo de 2025 estima a pose do braço a partir de uma única imagem RGB, sem acesso aos estados das juntas, e segue a arquitetura de estimação holística de pose de robôs de Ban et al. [29] (*Real-time Holistic Robot Pose Estimation with Unknown States*), referida nos artigos de 2026 como HRPE (grafada HPRE no relatório de 2025), acrescida de um ramo de profundidade monocular baseado no Depth Anything V2 [30]. No relatório de dezembro de 2025, esse modelo atingiu 75,4 % de acurácia ADD com limiar de 60 mm no robô Panda, no conjunto real DREAM com câmera RealSense [31]; esse resultado foi publicado em 2026 nos dois artigos descritos na subseção de produção científica.

4.8.1. Metodologia e implementação

Os 75,4 % correspondem à época 355 do experimento final de 2025 (`MyNet/experiments/panda/realsense_full`), com AUC-ADD de 0,547; como o script de 2025 guardava apenas os pesos de melhor AUC e os do último estado, os pesos dessa época não existem; dos dois arquivos salvos, a transferência partiu do estado final do mesmo treino, por ser o mais treinado (111,2 milhões de parâmetros). A arquitetura combina uma rede de profundidade da raiz (DepthNet, HRNet-W32 [32]) fundida com a profundidade relativa do Depth Anything V2, um ramo integral com mapas de calor 3D (ResNet-50 [33]), a regressão iterativa dos ângulos e da rotação da raiz e uma cinemática direta diferenciável.

O código, escrito para o Google Colab, foi portado sem reimplementação para um contêiner na estação DGX Spark (aarch64), com compatibilidade de pesos. Foram corrigidos dois defeitos do código de 2025: a profundidade da raiz era calculada depois da camada integral, o que explica as métricas zeradas desse ramo nos registros de 2025, e o laço de treino chamava `optimizer.step()` duas vezes por iteração. A adaptação ao servo6dof envolveu a cinemática direta da cadeia, obtida do URDF; 7 pontos-chave (base, origens das cinco juntas e centro da garra), o mesmo número do Panda, o que permitiu reaproveitar a camada final dos mapas de calor; cinco juntas estimadas, sem a abertura da garra; e a ampliação da faixa de profundidade dos mapas de calor de 1,3 m para 3,0 m, pois o braço tem cerca de 2,0 m de alcance, aproximadamente 2,4 vezes o do Panda. A geometria das câmeras foi verificada projetando sobre as imagens os pontos-chave calculados com os ângulos reais: numa amostra de 300 poses, 99,9 % (frontal) e 99,6 % (diagonal) dos pontos-chave dentro do quadro recaem sobre a máscara do robô, conforme ilustrado abaixo.

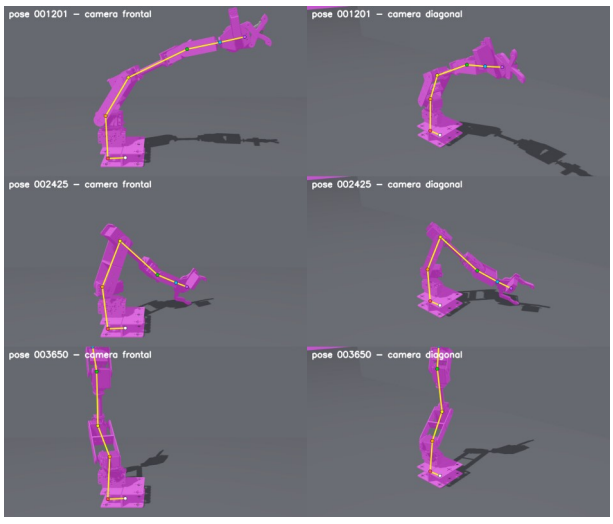


Figura 15. Verificação da geometria: pontos-chave e esqueleto obtidos por cinemática direta do URDF com os ângulos reais, projetados sobre imagens do dataset; em magenta, a máscara de segmentação do robô

A divisão treino/validação/teste foi feita por pose, de forma aleatória (semente 2026), na proporção 80/10/10 (8.000/1.000/1.000 imagens), mantendo as duas câmeras de uma pose no mesmo subconjunto. Em vez do recorte pela caixa dos pontos-chave verdadeiros do protocolo DREAM de 2025, adotou-se um recorte fixo por câmera, calculado só com o treino, de modo que teste e inferência não dependem do gabarito.

Foram aplicadas as etapas da metodologia de 2025 compatíveis com dados sintéticos. Na etapa 1, a DepthNet foi treinada por 5 épocas sobre as 3.151 poses de treino disponíveis naquele momento, atingindo erro de profundidade da raiz de 71 mm na validação. Na etapa 2, o modelo completo foi treinado com taxa de aprendizado de 1×10^{-4}, decaimento de 0,9 por época, lote 32 e o aumento de dados fotométrico de 2025. A etapa 3 de 2025, autossupervisionada com imagens reais, não se aplica, pois ainda não há imagens reais do braço. Compararam-se duas inicializações: a **transferência**, em que 98,1 % dos parâmetros vieram do modelo de 2025, e uma **linha de base** com redes pré-treinadas no ImageNet. A transferência recebeu 20 épocas (cerca de 62 min); a linha de base, executada isoladamente depois que uma tentativa em paralelo foi abortada por falta de memória na estação compartilhada, parou pelo limite de tempo após 12 épocas. Na época 12, a transferência tinha erro 3D de validação de 65,0 mm, contra 83,7 mm da linha de base, e acurácia@60 mm de 59,1 % contra 34,9 %, com MAE das juntas semelhante (13,1° contra 12,8°), conforme as curvas a seguir.

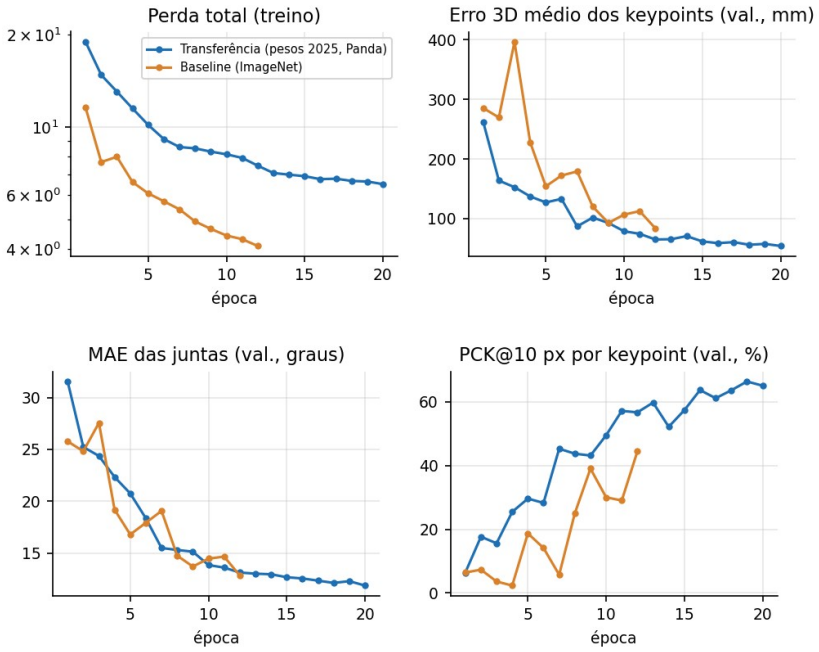


Figura 16. Etapa 2 do treino: perda total e métricas de validação por época (erro 3D, MAE das juntas e PCK a 10 px) para a transferência dos pesos de 2025 e para a linha de base inicializada no ImageNet

4.8.2. Resultados no conjunto de teste

A tabela a seguir resume as métricas no conjunto de teste (500 poses × 2 câmeras), não usado no treino nem na escolha do estado salvo, feita pela validação.

Tabela 12. Resultados no conjunto de teste do servo6dof (1.000 imagens)

Métrica	Transferência (pesos 2025)	Linha de base (ImageNet)
MAE médio das 5 juntas (°)	12,68	13,51
Erro 3D médio dos pontos-chave (mm)	58,7	88,3
Erro 3D mediano dos pontos-chave (mm)	41,1	65,7
AUC-ADD 0–100 mm	0,471	0,265
Acurácia ADD@60 mm (%)	68,0	32,8
Acurácia ADD@100 mm (%)	91,2	69,7
Acurácia ADD@150 mm (%)	96,7	90,2
Erro 2D médio (px, imagem 1280×720)	12,0	17,7
PCK@10 px por ponto-chave (%)	61,7	43,6
Épocas de treino	20	12

O modelo adaptado atingiu MAE de 12,68° nas cinco juntas, erro 3D médio de 58,7 mm (mediana de 41,1 mm) e acurácia@60 mm de 68,0 %. Em relação à linha de base, o erro 3D médio caiu cerca de 34 % e a acurácia@60 mm mais que dobrou, enquanto o MAE melhorou menos (13,51° para 12,68°); como a linha de base teve menos épocas, a comparação de mesmo orçamento é a da validação na época 12, que confirma a vantagem da transferência na precisão 3D.

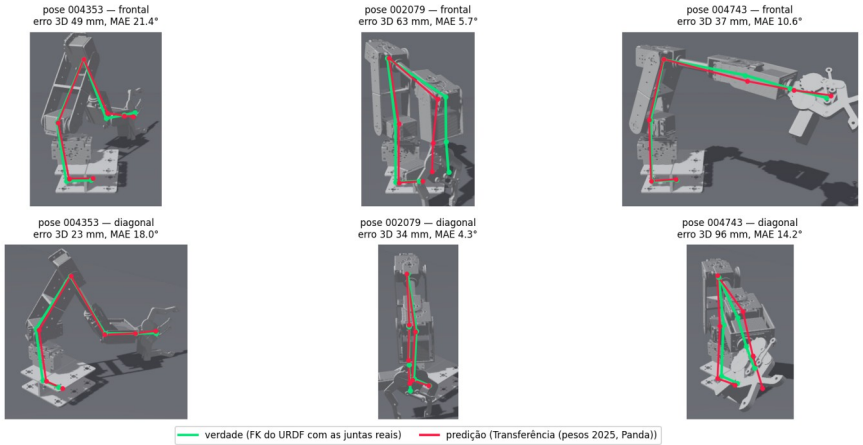


Figura 17. Predições do modelo de transferência (vermelho) e verdade obtida por cinemática direta com as juntas reais (verde) sobre imagens do teste nas duas câmeras, para poses nos percentis 25, 50 e 75 do erro 3D

A tabela seguinte coloca lado a lado os resultados de 2025 e de 2026, que **não são diretamente comparáveis**: robôs, escala, câmeras e dados diferem. Como o servo6dof tem cerca de 2,4 vezes o alcance do Panda, 60 mm correspondem a aproximadamente 3 % do alcance, contra cerca de 7 % no Panda; além disso, os dados de 2026 são sintéticos, o recorte não usa o gabarito

e não houve etapa autossupervisionada. Com limiar proporcional ao tamanho do robô (cerca de 140 mm), a acurácia@150 mm é de 96,7 %.

Tabela 13. Acurácia@60 mm em 2025 (Panda) e 2026 (servo6dof); robôs, escalas, câmeras e dados diferentes

Modelo	Robô / dados	ADD@60 mm (%)	AUC-ADD	MAE (°)
RoboPose [34]	Panda / DREAM real	74,3	—	—
HRPE [29]	Panda / DREAM real	75,2	—	—
Modelo de 2025 (HRPE + DA-V2)	Panda / DREAM real	75,4	0,547	11,58
Modelo de 2025 com transferência	servo6dof / sintético	68,0	0,471	12,68
Mesma arquitetura, ImageNet	servo6dof / sintético	32,8	0,265	13,51

Os valores de RoboPose e HRPE foram transcritos do relatório de 2025 e não puderam ser reconferidos em arquivo de resultados; os do modelo de 2025 foram conferidos no arquivo de resultados do treino de 2025. As principais limitações desta etapa são os dados exclusivamente sintéticos, o uso do estado final do treino de 2025 em vez dos pesos da época de 75,4 %, a linha de base com menos épocas, a etapa 1 treinada num subconjunto e o orçamento de treino de cerca de uma hora em GPU compartilhada.

4.9. Avaliação de precisão e robustez do modelo de pose

Esta etapa corresponde à Atividade 9 do Objetivo 3: avaliar o modelo de detecção de pose quanto à precisão e à robustez no novo cenário. Avaliou-se o modelo adaptado por transferência, comparado, quando pertinente, à linha de base ImageNet, sobre o conjunto de teste (1.000 imagens sintéticas), nunca usado no treino nem na seleção do estado salvo. A avaliação compreendeu o erro absoluto médio (MAE) por junta; o erro 3D dos pontos-chave e a acurácia ADD, isto é, a fração de imagens cujo erro 3D médio dos 7 pontos-chave não excede um limiar, com AUC-ADD entre 0 e 100 mm; o PCK 2D a 5, 10 e 20 px; a generalização entre câmeras; a robustez a sete perturbações da imagem, em três ou quatro severidades, num subconjunto fixo de 400 imagens, com a profundidade do Depth Anything V2 recalculada sobre a imagem perturbada, conforme os exemplos a seguir; e a latência com lote 1 na GPU compartilhada.

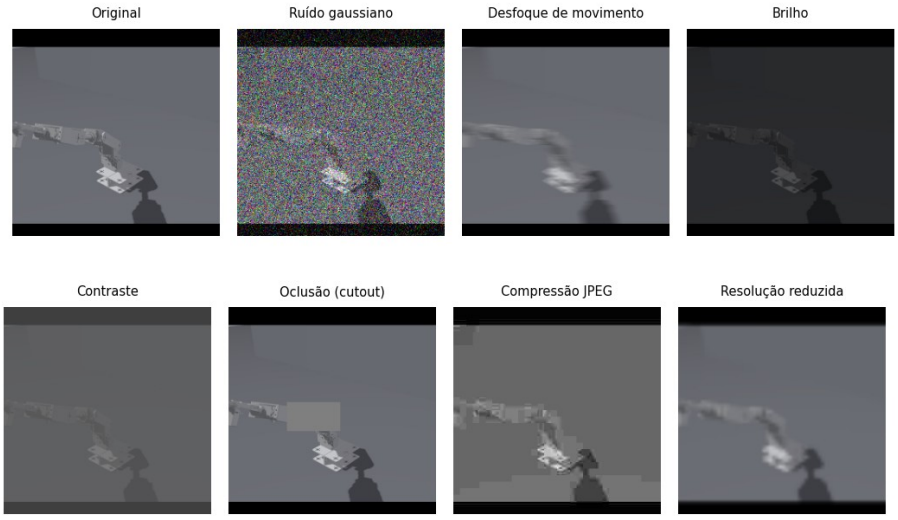


Figura 18. Exemplos das perturbações aplicadas ao recorte de entrada de 256×256 px nos testes de robustez: ruído gaussiano, desfoque de movimento, brilho, contraste, oclusão, compressão JPEG e resolução reduzida

4.9.1. Precisão por junta, por ponto-chave e por câmera

Tabela 14. Erro por junta no conjunto de teste

Junta	Transferência: MAE (°)	Transferência: mediana (°)	Linha de base: MAE (°)
joint_1 (base)	4,19	2,94	7,20
joint_2 (ombro)	3,04	1,89	3,87
joint_3 (cotovelo)	3,91	2,54	5,41
joint_4 (punho)	10,52	6,64	9,25
joint_5 (rotação da garra)	41,73	42,49	41,83

As juntas 1 a 3 são estimadas com MAE entre 3,0° e 4,2° e a junta 4 (punho), com 10,5°. A junta 5, rotação da garra em torno do eixo do antebraço, tem MAE de cerca de 42° nos dois modelos (41,7° e 41,8°), valor compatível com um preditor que não extrai informação dessa junta: o ponto-chave da garra está praticamente sobre o eixo de rotação, de modo que sua variação quase não altera os pontos-chave e as perdas geométricas não a supervisionam. Excluída a junta 5, o MAE médio das juntas 1 a 4 é de 5,4°. O erro 3D por ponto-chave é mínimo no ponto de referência (cotovelo, 27 mm) e cresce em direção à extremidade da cadeia, atingindo 110 mm na garra. No plano da imagem, 30,1 %, 61,7 % e 87,1 % dos pontos-chave ficam a menos de 5, 10 e 20 px da posição verdadeira, conforme as curvas a seguir. As duas câmeras têm desempenho equivalente: erro 3D de 57,4 mm e acurácia@60 mm de 69,0 % na frontal, contra 60,0 mm e 67,0 % na diagonal.

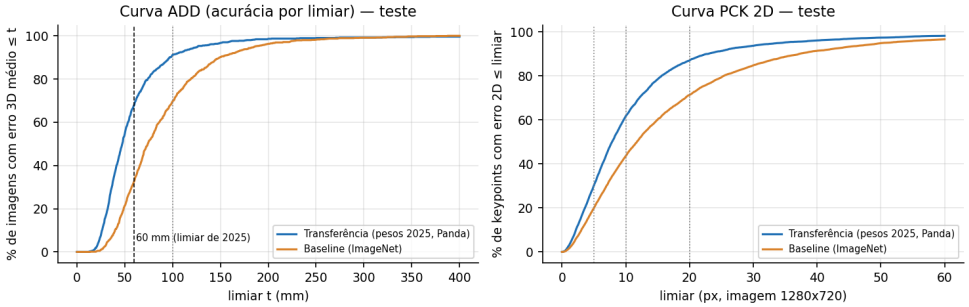


Figura 19. Curvas ADD (porcentagem de imagens com erro 3D médio até o limiar, com o limiar de 60 mm de 2025 destacado) e PCK 2D (porcentagem de pontos-chave com erro até o limiar, em pixels da imagem 1280×720) no conjunto de teste

4.9.2. Generalização entre câmeras

O modelo treinado apenas com a câmera frontal (experimento reduzido, com 3.151 poses de treino e 10 épocas) degrada fortemente na câmera não vista: o erro 3D passa de 140,4 mm na frontal para 418,5 mm na diagonal, a acurácia@60 mm cai de 8,6 % para 0,0 % e o MAE sobe de 23,1° para 32,6°. Como esse experimento usou menos dados e épocas, a comparação absoluta com o modelo final é apenas indicativa; a evidência relevante é a degradação de cerca de três vezes no erro 3D. O modelo aprende implicitamente a relação entre câmera e robô da vista de treino, e generalizar para câmeras novas exige variar a pose da câmera nos dados de treino ou fornecer os parâmetros extrínsecos como entrada.

4.9.3. Robustez a perturbações da imagem

A tabela a seguir apresenta uma seleção das condições avaliadas; as curvas completas são mostradas na figura subsequente.

Tabela 15. Robustez em subconjunto fixo de 400 imagens do teste, com profundidade recalculada sobre a imagem perturbada

Perturbação	Severidade	Transf.: erro 3D (mm)	Transf.: ADD@60 (%)	Transf.: MAE (°)	Linha de base: erro 3D (mm)
Sem perturbação	—	57,4	68,0	12,41	88,2
Ruído gaussiano	σ = 0,05	99,0	24,5	15,64	109,7
Ruído gaussiano	σ = 0,10	213,2	2,0	23,57	169,7
Desfoque de movimento	7 px	96,0	23,5	15,21	113,0
Desfoque de movimento	15 px	206,6	1,0	20,44	162,1
Brilho	×0,4	60,5	64,2	12,68	93,7
Contraste	×0,3	67,2	52,0	13,24	93,1
Oclusão	30 % da caixa	94,3	33,2	15,59	114,5
JPEG	qualidade 30	83,5	37,0	14,35	132,0
JPEG	qualidade 15	113,8	14,2	16,19	214,6
Resolução reduzida	escala 0,25	109,2	14,8	15,49	130,0

O modelo mostrou-se robusto a brilho e contraste (erro 3D de no máximo 68 mm mesmo com brilho ×0,4 ou contraste ×0,3), o que é coerente com o aumento de dados fotométrico herdado de 2025, mas sensível a perturbações de alta frequência: ruído gaussiano com σ = 0,05 eleva o erro 3D de 57 para 99 mm, o desfoque de 7 px para 96 mm e a compressão JPEG com qualidade 30 para 84 mm; a oclusão de 30 % da caixa leva a 94 mm e a redução da resolução para 25 %, a 109 mm. O MAE das juntas degrada mais lentamente que o erro 3D, que acumula também os erros de profundidade e de rotação da raiz. A linha de base parte de um erro maior, mas degrada menos sob ruído e desfoque fortes (σ = 0,10: 170 mm contra 213 mm; desfoque de 15 px: 162 mm contra 207 mm) e mais sob compressão JPEG (qualidade 15: 215 mm contra 114 mm); sob brilho, contraste, oclusão, JPEG e resolução reduzida, a transferência permanece melhor em todas as severidades.

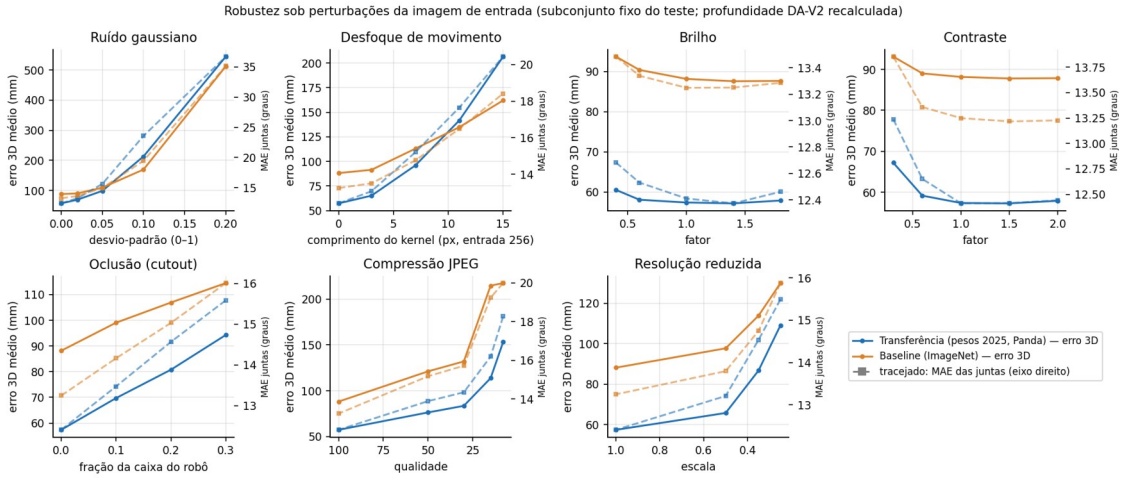


Figura 20. Robustez: erro 3D médio (linha cheia) e MAE das juntas (tracejado) em função da severidade de cada perturbação, para a transferência e para a linha de base ImageNet, em subconjunto fixo de 400 imagens do teste

4.9.4. Latência e limitações

O modelo final foi empacotado no módulo de inferência `PosePredictor`, usado pelo gêmeo digital, que recebe uma imagem RGB 1280×720 e devolve os ângulos das cinco juntas e os pontos-chave 2D e 3D. Em 100 chamadas com lote 1, a latência ponta a ponta foi de 162,5 ms em média (mediana de 170,1 ms e p95 de 174,1 ms), ou 6,15 quadros por segundo, com pico de 2,94 GB de memória de GPU, valores pessimistas por causa da GPU compartilhada. Numa medição separada, o Depth Anything V2 (ViT-L) respondeu por cerca de 65 % do tempo total (138,9 ms de 213,8 ms), enquanto a rede HRPE isolada levou 74,9 ms; versões menores do Depth Anything V2 ou a otimização com TensorRT são as alternativas para tempo real.

As principais limitações da avaliação são as medições apenas em dados sintéticos e com duas câmeras fixas, sem imagens reais do braço físico; a junta

5 não observável com os pontos-chave escolhidos e a abertura da garra não estimada; e os experimentos de generalização e de robustez feitos em subconjuntos. Como próximos passos, propõem-se adicionar pontos-chave nas pontas das pinças, randomizar pose de câmera, iluminação e texturas na geração de dados, incluir ruído, desfoque e compressão no aumento de dados e capturar imagens reais para aplicar a etapa autossupervisionada de 2025.

Código e evidências da estimação de pose: repositório cerise-2026-pose-servo6dof (privado)

4.10. Modelo digital do braço robótico

Esta etapa corresponde à Atividade 11 do Objetivo 4: modelar o braço robótico em ambiente virtual, com suas características físicas e cinemáticas. O modelo digital é a base comum do dataset, do planejamento de movimento e do gêmeo digital, e por isso foi caracterizado e validado de forma independente.

4.10.1. Metodologia

O URDF foi gerado a partir do xacro do pacote de descrição, com os mesmos parâmetros usados pelo Gazebo no dataset. Foi implementada uma cinemática direta própria em Python e NumPy (`modelo_digital/cinematica.py`), que resolve as juntas *mimic* e opera em lotes vetorizados, validada contra o TF publicado pelo `robot_state_publisher` do ROS 2 em 300 configurações aleatórias e 11 elos. O espaço de trabalho foi estimado por Monte Carlo com 100.000 configurações. Também foi implementado o modelo *pinhole* das câmeras do estúdio, que permite projetar as malhas STL posicionadas pela cinemática sobre as imagens do dataset e medir o IoU entre a silhueta do modelo e a máscara do simulador. Por fim, o MoveIt 2 [35] foi executado sem interface gráfica, com o planejador OMPL [36] RRTConnect [37], cinemática inversa KDL e `ros2_control` com hardware simulado, para cinco objetivos com dez pedidos de planejamento cada.

4.10.2. Resultados

O URDF possui 13 elos e 12 juntas: 11 de revolução, das quais 6 são ativas (`joint_1` a `joint_6`) e 5 são *mimic* de `joint_6`, e uma junta fixa entre o mundo e a base. O braço tem 5 graus de liberdade e a garra, 1. As massas somam 1,51 kg, e todas as inércias têm valor de preenchimento (1 kg·m²), de modo que o modelo é cinematicamente completo, mas não dinamicamente calibrado.

Tabela 16. Juntas ativas do modelo digital extraídas do URDF

Junta	Elo pai → elo filho	Origem xyz (m)	Eixo	Limites (°)
joint_1	base_link → primeiro_elo	(0; −0,23; −0,01)	z	−90 a 90
joint_2	primeiro_elo → segundo_elo	(−0,05; −0,03; 0,57)	x	−90 a 90
joint_3	segundo_elo → terceiro_elo	(0,06; 0; 0,63)	x	−90 a 90
joint_4	terceiro_elo → quarto_elo	(0; 0,78; 0)	x	−90 a 90
joint_5	quarto_elo → base_garra	(0,02; 0,36; 0)	y	−90 a 90
joint_6 (garra)	base_garra → garra_elo1	(−0,072; 0,12; 0,01)	z	−40 a 30

Tabela 17. Verificações do modelo digital

Verificação	Amostra	Resultado medido
Cinemática direta própria × TF do ROS 2	300 configurações × 11 elos	erro máx. $1{,}9\times10^{-15}$ m e $4{,}2\times10^{-8}$ rad
Cinemática do visualizador (three.js) × própria	capturas do visualizador	até $7{,}6\times10^{-7}$ m
Espaço de trabalho da ponta da garra	100.000 configurações	alcance horizontal 2,05 m; z de −0,82 a 2,58 m
Silhueta do modelo × máscara (câmera frontal)	1.000 imagens	IoU médio 0,950 (mediana 0,965)
Silhueta do modelo × máscara (câmera diagonal)	1.000 imagens	IoU médio 0,938 (mediana 0,951)
MoveIt 2 sem interface gráfica	5 objetivos × 10 pedidos	50/50 planos; 12,9 a 17,0 ms em média

A cinemática própria coincidiu com o TF do ROS 2 no nível do arredondamento numérico, inclusive nos elos *mimic*, e a do visualizador web, usada no gêmeo, coincidiu com ela dentro da precisão de 32 bits. O fecho convexo do espaço de trabalho tem 27,8 m³ e o volume efetivamente ocupado, medido em voxels de 5 cm, 7,9 m³; cerca de 20 % das configurações dentro dos limites levariam a ponta da garra abaixo do chão, o que explica parte das divergências entre juntas comandadas e reais do dataset.

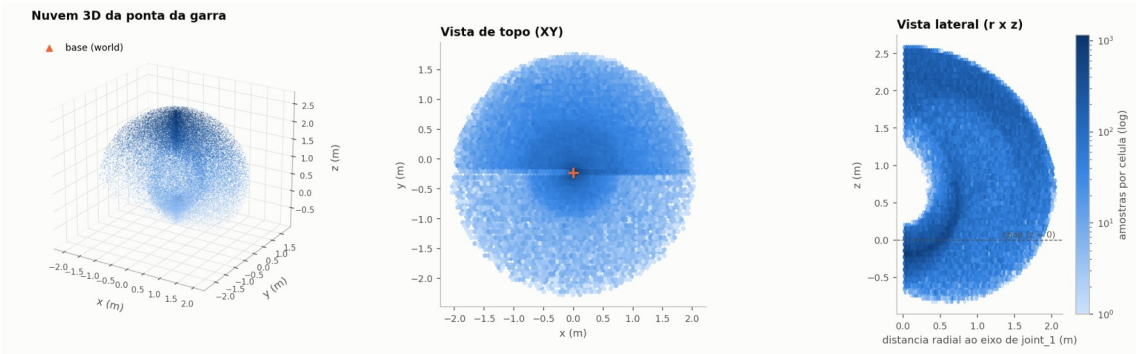


Figura 21. Espaço de trabalho da ponta da garra estimado com 100.000 configurações: nuvem 3D, vista de topo e vista lateral radial com o plano do chão

A projeção das malhas sobre as imagens do simulador coincide visualmente com o braço renderizado, o que valida simultaneamente o modelo cinemático, o modelo de câmera e os rótulos do dataset; em 2,05 % das imagens avaliadas (41 de 2.000), o IoU ficou abaixo de 0,8, casos em que o braço aparece

numa configuração diferente da registrada. No MoveIt 2, os cinco objetivos foram planejados e executados com sucesso, com trajetórias de 17 a 28 pontos e duração de 1,56 a 2,63 s; no alvo cartesiano, a posição final ficou a 1,6 cm do alvo, dentro da tolerância de 2 cm especificada.

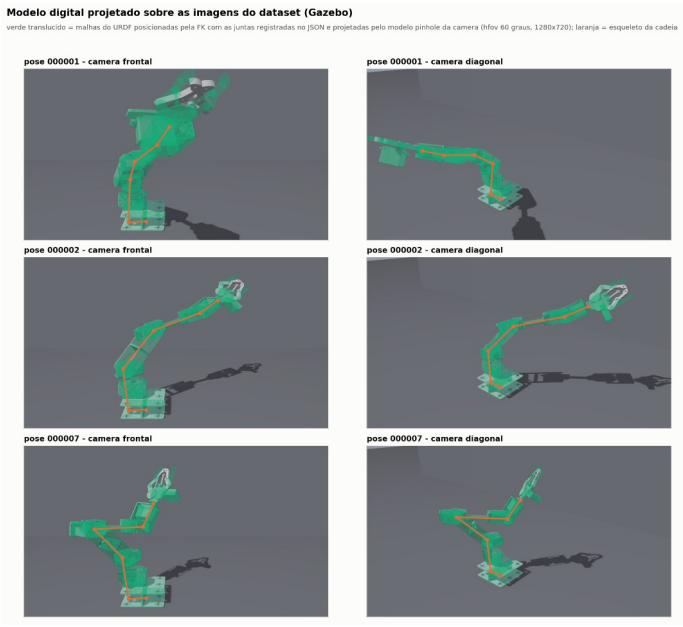


Figura 22. Malhas do modelo digital (verde translúcido) e cadeia cinemática (laranja) projetadas sobre imagens do dataset nas câmeras frontal e diagonal

4.10.3. Discussão e limitações

O modelo digital é cinematicamente consistente com o simulador e com o ROS 2, o que o qualifica como referência para o gêmeo digital. As características físicas, porém, não foram representadas de forma fiel: as massas são simbólicas e as inércias são valores de preenchimento, inadequados para simulação dinâmica, e a calibração a partir do CAD ou de medições do braço físico permanece como trabalho futuro; por isso, a atividade é considerada parcialmente concluída. Além disso, o Gazebo Fortress não implementa as juntas *mimic*, o que responde por parte do erro residual de silhueta na garra; o planejamento não inclui o chão como objeto de colisão; e o MoveIt 2 foi exercitado apenas com hardware simulado.

4.11. Gêmeo digital do braço robótico

Esta etapa corresponde à Atividade 12 do Objetivo 4: integrar o modelo de detecção de pose ao ambiente virtual do braço, com o resultado esperado de um gêmeo digital capaz de reproduzir o comportamento do braço a partir dos dados capturados [38]. No sistema desenvolvido, o braço de referência segue a verdade-base do simulador (`juntas_reais`), enquanto o gêmeo segue apenas as juntas estimadas pelo modelo de pose a partir de uma única imagem RGB; a comparação quadro a quadro entre os dois mede a fidelidade do gêmeo.

4.11.1. Arquitetura e implementação

O servidor do gêmeo (FastAPI com WebSocket) executa em contêiner na estação DGX Spark, acessível apenas pela interface local, por túnel SSH. A estimação é feita por um componente plugável (`Predictor`), cuja implementação principal adapta o módulo de inferência da frente de pose (Depth Anything V2 [30] e estimador de pose) e devolve os ângulos `joint_1` a `joint_5`. O *front-end*, em three.js r160 com urdf-loader 0.13.1, carrega o mesmo URDF e as malhas duas vezes (referência sólida e gêmeo verde translúcido) e exibe a imagem com o esqueleto 2D previsto, os ângulos por junta e o gráfico de erro. A avaliação usa o conjunto de teste da frente de pose (1.000 imagens). Para medir a latência, o WebSocket opera em malha fechada: cada quadro só é enviado após o *front-end* confirmar que o anterior foi desenhado, em Chromium sem interface gráfica com WebGL acelerado pela GPU. No ROS 2, o nó `gemeo_twin` publica `/twin/joint_states` a partir das imagens, e um segundo `robot_state_publisher` alimenta o RViz 2 em tela virtual.

4.11.2. Resultados

A média das cinco juntas no gêmeo (12,68°) é igual à da avaliação da frente de pose nas mesmas imagens, o que indica que a integração não acrescenta erro (as medianas das juntas diferem em até 0,02° por pequenas diferenças numéricas da nova execução da inferência na GPU); o viés médio é inferior a 1° em módulo nas juntas 1 a 4.

Tabela 18. Erro absoluto por junta no gêmeo digital, em graus (conjunto de teste, apenas imagem)

Junta	Frontal (média)	Diagonal (média)	Total (média)	Total (mediana)	Total (p95)
joint_1 (base)	4,08	4,30	4,19	2,92	11,9
joint_2 (ombro)	3,12	2,96	3,04	1,89	9,0
joint_3 (cotovelo)	4,09	3,73	3,91	2,54	11,6
joint_4 (punho)	10,46	10,57	10,52	6,63	36,1
joint_5 (rotação da garra)	41,75	41,71	41,73	42,49	78,0
Média das juntas 1 a 4	5,44	5,39	5,41	3,83	15,4

O erro de posição do efetuador teve mediana de 76,4 mm, média de 107,1 mm, p95 de 275,0 mm e máximo de 991,4 mm; 64,5 % dos quadros ficaram abaixo de 100 mm e 27,2 % abaixo de 50 mm. Como a ponta da garra alcança até 2,09 m do ombro, 1° de erro no ombro desloca a garra em até cerca de 37 mm.

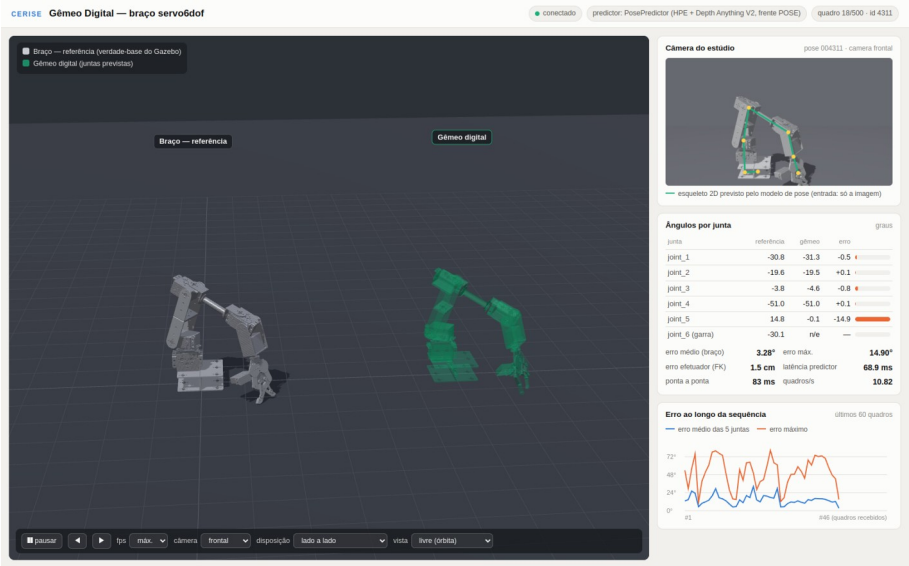


Figura 23. Gêmeo digital com o modelo de pose em um quadro de baixo erro: braço de referência e gêmeo lado a lado, esqueleto 2D previsto sobre a imagem, ângulos por junta e erro do efetuador de 1,5 cm

A latência entre a leitura da imagem e o gêmeo desenhado está decomposta na tabela seguinte; o modelo de pose domina o tempo, e a mediana ponta a ponta supera a do modelo em cerca de 22 ms. A mediana do modelo neste ensaio (77,7 ms) é menor que a da medição isolada do `PosePredictor` (170,1 ms); as duas foram feitas em momentos diferentes e com cargas diferentes na GPU compartilhada com os serviços de produção, e por isso os valores absolutos não são comparáveis entre si.

Tabela 19. Latência por etapa, em milissegundos (1.000 quadros, malha fechada, WebGL na GPU)

Etapa	Mediana	Média	p95
Leitura do PNG 1280×720	8,4	10,0	18,2
Modelo de pose (recorte, profundidade e pose)	77,7	103,1	180,4
Cinemática direta e JPEG da câmera	3,6	4,4	8,6
Serialização e WebSocket	1,3	1,4	2,2
Atualização no three.js até o *canvas*	14,1	15,0	18,2
Ponta a ponta medido	99,6	134,5	216,9

Em malha fechada, a sessão da câmera frontal sustentou 9,76 quadros/s e a da diagonal, 5,93 quadros/s; com taxas-alvo fixas, foram medidos 2,00, 5,00 e 9,97 quadros/s para alvos de 2, 5 e 10. A diferença entre câmeras decorre da carga da GPU compartilhada: num controle com 120 chamadas alternando as câmeras, a latência do modelo foi de 65,2 ms (frontal) e 64,6 ms (diagonal). Com rasterização por software, o navegador caiu para 0,42 quadro/s, o que mostra a necessidade de WebGL acelerado.

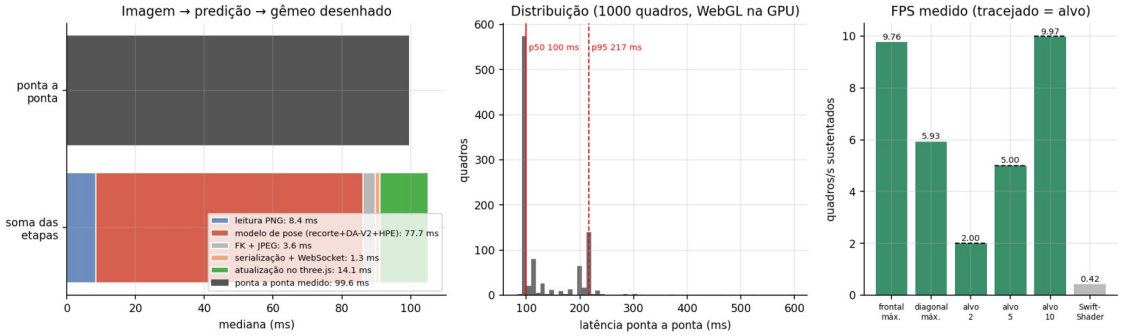


Figura 24. Latência imagem → predição → gêmeo desenhado por etapa, distribuição da latência em 1.000 quadros e taxa de quadros sustentada por sessão

O principal modo de falha foi o braço cortado pelo enquadramento: numa amostra de 200 quadros, isso ocorreu em 22,5 % deles, com erro mediano do efetuador de 117 mm contra 70 mm nos demais, e esteve presente em 45 % dos 20 piores quadros. O pior caso (991 mm) é uma ambiguidade do tipo “cotovelo invertido”, com erros de +111° no ombro e −118° no cotovelo e silhueta 2D semelhante à verdadeira. O erro também cresce com o giro da base (correlação de Spearman de 0,36 com o módulo de `joint_1`).

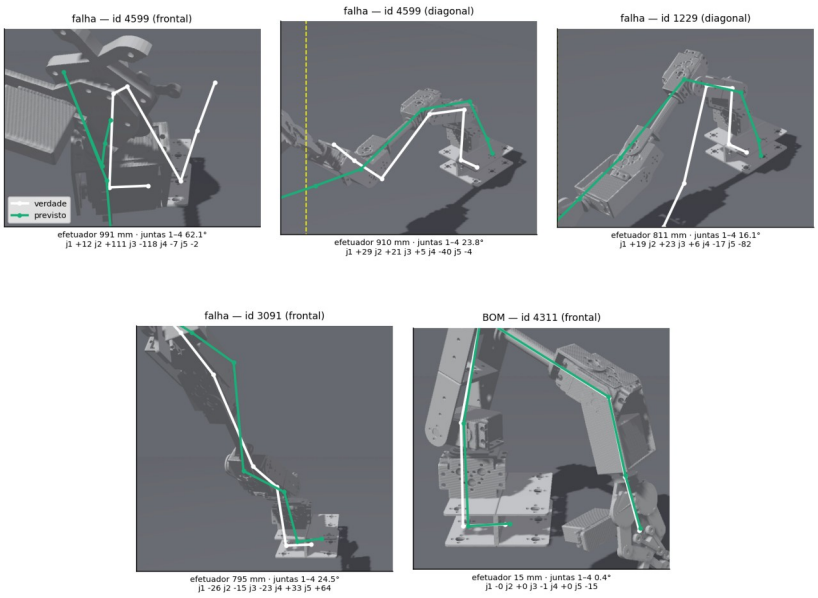


Figura 25. Piores quadros do gêmeo pelo erro do efetuador e um quadro de baixo erro, com o esqueleto verdadeiro em branco e o previsto em verde

No teste em ROS 2 (40 s, imagens da câmera frontal publicadas a 2 Hz), o tópico `/twin/joint_states` foi publicado a 2,00 Hz, com 79 pares casados com a referência; o atraso entre a imagem e a mensagem do gêmeo teve mediana de 255 ms (p95 de 282 ms), dos quais 87 ms correspondem ao modelo via HTTP e o restante ao transporte da imagem bruta, e o erro mediano do efetuador, de 77 mm, foi coerente com a avaliação pelo navegador.

4.11.3. Discussão e limitações

O gêmeo reproduz o comportamento do braço nas juntas que o modelo observa bem: nas juntas 1 a 4, o erro mediano é de 3,8°, e 64,5 % dos quadros têm o efetuador a menos de 10 cm da posição real, com 5,9 a 9,8 quadros/s em malha fechada (conforme a câmera e a carga da GPU) e latência mediana de 0,1 s, adequados ao acompanhamento visual. Como limitações, a validação foi feita apenas em simulação, com imagens sintéticas e as mesmas duas câmeras do treino; a rotação da garra não é estimada de forma útil e a abertura da garra não é estimada; a latência foi medida com a GPU compartilhada, sem garantia de tempo real; e cada quadro é tratado de forma independente, sem filtro temporal. O visualizador foi mantido acessível apenas pela rede interna, por túnel SSH, e sua exposição pública ficou para uma etapa posterior. Propõem-se a avaliação com o braço físico e câmera calibrada, a fusão das duas câmeras com filtragem temporal e a inclusão de pontos-chave nas pinças.

Código e evidências do dataset, do modelo digital e do gêmeo digital: repositório cerise-2026-servo6dof-gemeo-digital (privado)

4.12. Comunicação LoRa: protótipo e projeto do módulo

Esta etapa corresponde à Atividade 10 do Objetivo 3, que prevê a aquisição de módulos LoRa e microcontroladores e o desenvolvimento de um protótipo capaz de enviar e receber dados entre dois dispositivos, e aos itens 3 e 4 do Objetivo 5, que preveem a construção dos módulos de comunicação definitivos e a adequação dos drones para recebê-los. O protótipo foi concebido como o enlace entre um nó embarcado no drone (papel *node*) e uma estação de solo (papel *gateway*). Como os módulos de rádio ainda não foram adquiridos, a gravação nas placas, o ensaio físico e a fabricação dos módulos não foram realizados: a verificação foi feita sem hardware de rádio, por compilação, testes, validação cruzada e simulação, e o módulo dos drones ficou em nível de projeto.

4.12.1. Protocolo e firmware

O protocolo `cerise_proto` v1 foi implementado em C99 como biblioteca compartilhada pelo firmware e pelos testes. Cada quadro tem cabeçalho de 8 bytes, *payload* de layout fixo por tipo e CRC-16/CCITT-FALSE, conforme ilustrado abaixo. Os tipos de mensagem foram alinhados com a frente de drones: `HEARTBEAT`, `POS` (posição, bateria e contagem), `AREA_ASSIGN` (área atribuída pela estação de solo), `COVERAGE_DONE`, `COUNT_REPORT` e `ACK`, que devolve ao nó o RSSI e o SNR medidos no *gateway*; três tipos propostos por aquela frente (`AREA_POLY`, `TASK_BID` e `TASK_AWARD`) foram incorporados como extensão aditiva. O *payload* do `POS` tem 14 bytes, para que o quadro completo, de 24 B, fique abaixo de 400 ms no ar até SF10.

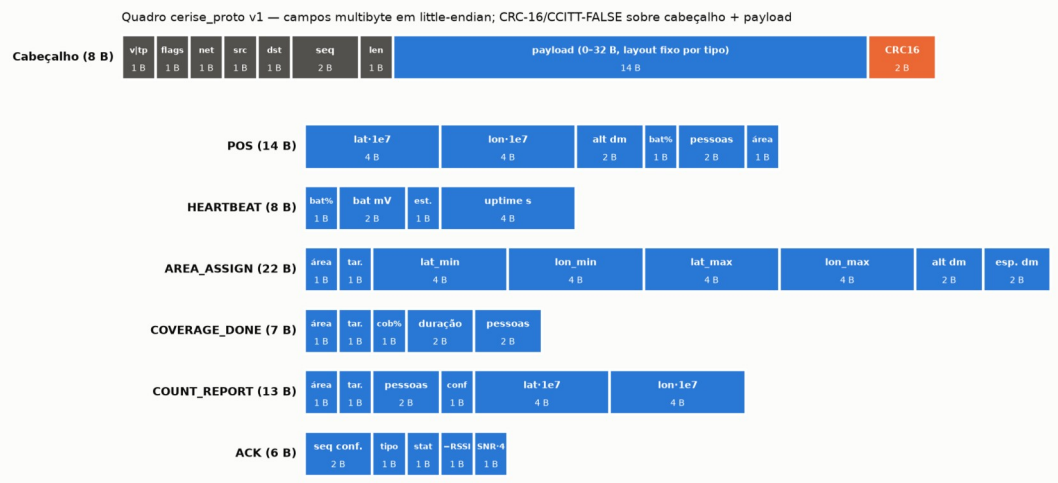


Figura 26. Formato do quadro cerise_proto v1 e layout do payload de cada tipo de mensagem

O firmware foi escrito com PlatformIO (framework Arduino), RadioLib 7.8.1 [39] e U8g2, tendo a Heltec WiFi LoRa 32 V3 (ESP32-S3 e SX1262) como placa principal. O *node* gera um `POS` a cada 5 s com variação aleatória de ±10 %, executa detecção de atividade no canal (CAD) antes de transmitir, aguarda o ACK e repete a transmissão até três vezes com recuo aleatório exponencial; o *gateway* valida os CRCs, descarta duplicatas, responde com ACK, estima a taxa de entrega de pacotes (PDR) por nó, mantém uma fila de *downlink* e emite uma linha JSON por evento. Os parâmetros de rádio seguiram o enquadramento brasileiro: plano AU915-928 [40], [41], canal único de 916,8 MHz para a bancada, 14 dBm e limite de 400 ms no ar por quadro, compatível com o item 10.2.5 do Ato ANATEL nº 14.448/2017 [42].

4.12.2. Verificação e simulação

A verificação compreendeu 30 testes unitários nativos (Unity) e 21 testes em Python, todos aprovados, que cobrem, entre outros casos, a detecção de todos os erros de 1 e de 2 bits e de rajadas de até 16 bits num quadro de 24 B; um espelho do protocolo em Python, com resultados idênticos aos da implementação em C em 3.000 de 3.000 quadros válidos, linhas JSON e quadros corrompidos; uma calculadora de tempo no ar baseada na formulação da Semtech [43], conferida em 9.360 de 9.360 combinações; e a compilação, sem avisos, dos cinco ambientes de firmware. O tempo no ar do `POS` resultou em 61,7 ms, 205,8 ms e 370,7 ms em SF7, SF9 e SF10; o quadro de `AREA_ASSIGN` (32 B, com *payload* de 22 B) ocupa 452,6 ms em SF10 e, portanto, deve ser enviado com SF ≤ 9.

Uma simulação de eventos discretos de N drones compartilhando o canal, validada contra a expressão analítica do ALOHA puro [44] (erro absoluto médio de 0,56 p.p. e máximo de 1,47 p.p. entre N = 2 e N = 20), totalizou 1.130

execuções (5 sementes de 1.800 s simulados por configuração). A tabela e a figura a seguir apresentam a PDR média em SF9, com POS a cada 5 s.

Tabela 20. PDR simulada (%) em função do número de drones N, em SF9, com POS de 24 B a cada 5 s

N	ALOHA puro	ALOHA + ACK	CSMA/CAD + ACK	Slotted ALOHA	TDMA + ACK
2	91,1	99,7	100,0	93,0	100,0
5	71,4	67,6	100,0	83,7	100,0
8	54,5	26,6	97,9	72,8	100,0
10	46,0	15,1	62,6	67,2	100,0
12	38,4	8,1	44,0	61,0	100,0
15	30,1	3,2	26,5	54,2	83,5
20	19,2	0,7	11,2	43,7	62,6

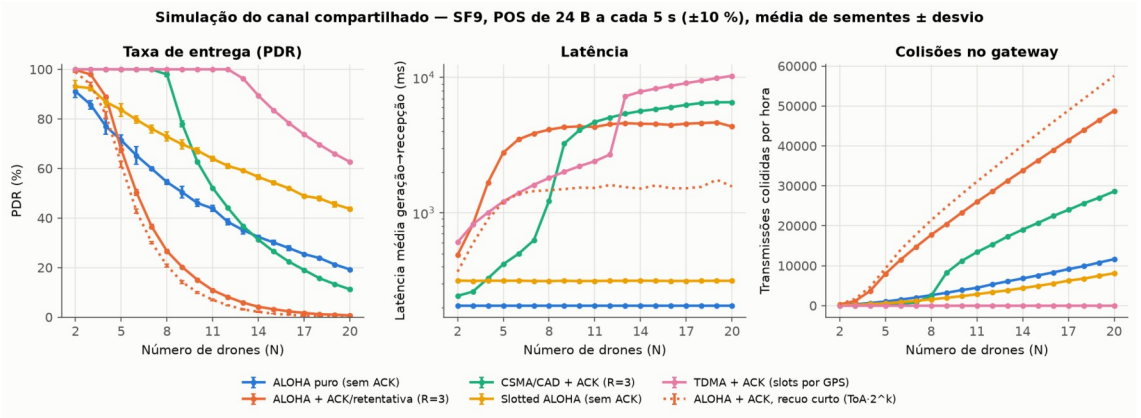


Figura 27. Resultados da simulação de eventos discretos em SF9: PDR, latência média e colisões no gateway em função do número de drones para cinco esquemas de acesso ao canal; a curva pontilhada é o ALOHA + ACK com o recuo curto (ToA·2^k) da versão inicial do firmware

Três conclusões orientam o projeto. O ACK com retransmissão no mesmo canal piora o desempenho sob carga: com 10 drones, a ocupação somada do canal chega a 145 % e a PDR cai para 15,1 %, contra 46,0 % sem ACK, o que motivou a alteração do recuo de retransmissão no firmware. A escuta prévia do canal (CAD) mantém PDR ≥ 97,9 % até 8 drones, mas também satura acima disso. O TDMA com *slots* sincronizados por GPS (*slot* de 401 ms em SF9) entrega 100 % até 12 drones, ao custo de maior latência (2.214 ms em média com 10 drones, contra 206 ms no ALOHA puro).

O leitor serial do *gateway* (*dashboard*) também foi exercitado com 600 s simulados de três drones a 1,0 km, 4,6 km e 6,4 km da estação, sobre um canal sintético: foram lidas 593 linhas sem nenhuma inválida, e a PDR calculada coincidiu com a reportada pelo *gateway* simulado: 98,6 % (137/139), 97,8 % (135/138) e 94,9 % (129/136).

4.12.3. Projeto do módulo de comunicação dos drones

O módulo definitivo foi especificado como um *modem* LoRa transparente para o computador de bordo do drone, o mesmo que executa o planejamento de cobertura e a contagem de pessoas: o computador envia comandos de linha pela UART (3,3 V, 115200 bps) e recebe o registro em linhas JSON, inclusive os *downlinks* `AREA_ASSIGN`, de modo que o rádio não depende da controladora de voo. A alimentação de 5 V vem de um BEC dedicado, e a antena dipolo de 915 MHz fica na vertical, voltada para baixo, a pelo menos 10 cm do receptor GNSS e dos cabos dos ESCs, conforme o diagrama a seguir. O gabinete foi modelado de forma paramétrica em Python, com dimensões externas de 55,0 × 44,3 × 17,2 mm e massa estimada de 12,9 g em PETG com preenchimento total, valor tomado como limite superior.

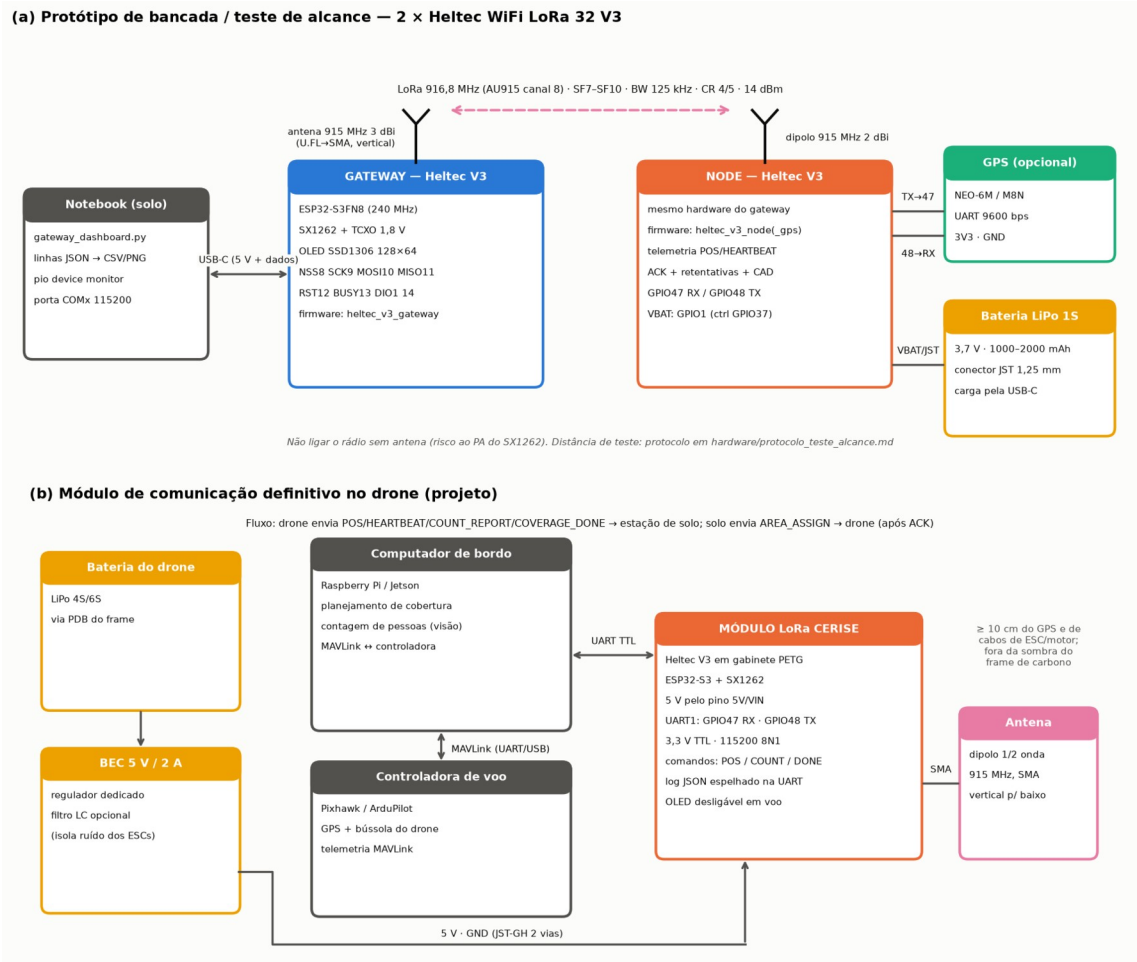


Figura 28. Diagrama de ligação: (a) protótipo de bancada com duas placas Heltec V3 e notebook de solo; (b) módulo definitivo no drone, com BEC de 5 V, UART para o computador de bordo e antena dipolo

Foram documentados quinze requisitos (R1 a R15), dos quais os principais estão resumidos a seguir com a natureza de cada valor.

Tabela 21. Principais requisitos do módulo de comunicação definitivo e natureza dos valores de projeto

Requisito	Valor de projeto	Natureza
Ocupação por quadro	≤ 400 ms (POS até SF10; AREA_ASSIGN até SF9)	calculado
Alcance	≥ 3 km com margem de 10 dB, drone a 60 m	estimativa
Acesso múltiplo	até 12 drones por canal em SF9 (TDMA)	simulação
Massa	≤ 40 g; gabinete de 12,9 g	meta; estimativa
Alimentação	5 V; 0,23 a 0,34 W médios	estimativa
Interface	UART 3,3 V, 115200 8N1, JST-GH 4 vias	especificação
Regulamentação	salto de frequência (≥ 35 canais) e homologação	pendente
Segurança	autenticação e cifragem AES-128-CCM	não implementado

O orçamento de enlace considerou 14 dBm, antenas de 2 dBi (drone) e 3 dBi (solo), 1,5 dB de perdas, margem de 10 dB, sensibilidade teórica do receptor e antena de solo a 2 m, com os modelos log-distância e Okumura–Hata [45].

Tabela 22. Alcance estimado ar–solo por fator de espalhamento (14 dBm, drone a 60 m, margem de 10 dB)

SF	Sensibilidade (dBm)	Perda máx. (dB)	Log-dist. n = 2,7 (km)	Hata suburbano (km)	Hata urbano (km)
7	−124,5	132,0	5,21	4,24	2,12
8	−127,0	134,5	6,44	5,04	2,52
9	−129,5	137,0	7,97	5,99	3,00
10	−132,0	139,5	9,87	7,12	3,57

No modelo suburbano em SF9, o alcance estimado passa de 4,13 km com o drone a 30 m para 5,99 km a 60 m e 9,11 km a 120 m. O consumo, estimado com correntes típicas de *datasheet* do SX1262 e a hipótese de 40 mA para o ESP32-S3, ficou entre 0,23 W e 0,34 W na entrada de 5 V, ou 0,11 % a 0,17 % de uma potência média de voo de referência de 200 W. Para a validação, foi redigido um protocolo de ensaio de alcance e PDR com 200 quadros por ponto, distâncias de 50 m a 3 km e SF7, SF9 e SF10.

4.12.4. Discussão e limitações

O software do protótipo está completo e verificado, mas a Atividade 10 permanece parcial: nenhum ensaio físico foi realizado, de modo que RSSI, SNR, PDR real, alcance e temporização do ACK não foram medidos, e os resultados do *dashboard* validam o software de registro, não o enlace. Nenhum módulo foi fabricado nem instalado nos drones, e os valores de alcance, massa e consumo são estimativas. O protótipo opera em canal único: a conformidade plena ao Ato nº 14.448/2017 requer salto de frequência com no mínimo 35 canais ou o enquadramento como modulação digital, além da homologação, e o protocolo ainda não tem cifragem nem autenticação. O próximo passo é adquirir duas

placas Heltec V3 e antenas, gravar o firmware e executar o ensaio de alcance e PDR.

Código e evidências: repositório cerise-2026-lora-prototipo (privado)

4.13. Segmentação da área de atuação entre drones

Esta etapa corresponde à Atividade 13 do Objetivo 5: desenvolver um software funcional de segmentação da área de atuação entre os drones disponíveis. O problema consiste em dividir uma área poligonal georreferenciada entre uma frota possivelmente heterogênea, de modo que as aeronaves concluam a varredura em tempos próximos, e em gerar, para cada drone, uma missão de cobertura executável por um piloto automático. A validação desta etapa foi algorítmica e em planta (2D), com um modelo cinemático simplificado e o polígono real de uma área do Campus Samambaia; rotas geradas pelo mesmo método para um recorte de 11,27 ha dessa área foram depois percorridas na simulação 3D no Isaac Sim descrita adiante, sem SITL nem voos reais.

4.13.1. Metodologia e implementação

Foi desenvolvido o pacote Python `cerise_drones`, que recebe o polígono da área em GeoJSON, zonas proibidas com margem de segurança e a descrição da frota (velocidade, autonomia, reserva de bateria de 20 %, campo de visão da câmera, altitude de 40 m e base). O espaçamento entre linhas de varredura corresponde a 80 % da largura da pegada da câmera em nadir. Foram definidos três tipos de multirrotor com parâmetros representativos, sem correspondência com modelos comerciais: A (10 m/s, 20 min), B (6 m/s, 30 min) e C (14 m/s, 25 min). Foram comparados quatro métodos de partição:

- **faixas ponderadas**: faixas paralelas à direção de varredura, com área proporcional ao peso de cada drone;
- **k-means + Voronoi**: células de Voronoi dos centroides de um k-means, atribuídas aos drones pelo método húngaro [46], como linha de base clássica, sem ponderação;
- **DARP em grade** [47]: atribuição das células de uma grade por distância geodésica a sementes, com balanceamento iterativo; além da ponderação multiplicativa original, foi implementada uma ponderação aditiva que gera regiões conexas, com retificação das fronteiras pelo algoritmo de Douglas–Peucker [48];
- **DARP + refinamento por tempo**, proposto neste trabalho: sementes nos centros de um k-means ponderado e quatro iterações de reajuste dos pesos `w ← w·(T̄/T)^0,8`, em que `T` é o tempo de missão estimado

de cada drone e $\bar{T}$, a média desses tempos na frota, mantendo-se a solução de menor *makespan* (maior tempo de missão da frota).

Cada subárea recebe uma rota em boustrophedon [49] com a orientação de menor tempo estimado e trânsito por grafo de visibilidade em torno das zonas proibidas, com retorno à base para troca de bateria quando necessário. Para cada drone, são exportados um plano `.plan` do QGroundControl, arquivos KML e GeoJSON e um mapa interativo; os planos foram aprovados por um validador estrutural, mas ainda não foram abertos no QGroundControl nem executados em SITL. A avaliação utilizou o campo aberto do Campus Samambaia (polígono côncavo de 34,23 ha, obtido do OpenStreetMap [50]) e três cenários sintéticos (convexo e côncavos em L e em U, com zonas proibidas), com N = 2 a 6 drones e frotas homogênea e heterogênea, num total de 160 execuções, além de 70 de ablação. A suíte de 16 testes automatizados foi aprovada integralmente.

4.13.2. Resultados

A tabela a seguir resume a partição do campo aberto do Campus Samambaia entre quatro drones heterogêneos (dois do tipo A, um B e um C).

Tabela 23. Partição do campo aberto do Campus Samambaia (34,2 ha) entre 4 drones heterogêneos pelos quatro métodos

Método	*Makespan* (min)	CV do tempo	Distância (km)	Curvas	Cobertura (%)
Faixas ponderadas	8,63	0,120	13,36	56	100,0
k-means + Voronoi	10,63	0,385	10,65	70	100,0
DARP (sementes nas bases)	8,18	0,152	11,66	77	100,0
DARP + refinamento	6,57	0,002	11,37	73	100,0

No plano final (DARP + refinamento), o drone do tipo C recebeu 13,46 ha, os do tipo A, 7,07 ha e 9,20 ha, e o do tipo B, 4,50 ha; os quatro concluem a missão entre 6,54 e 6,57 min, com uso de 22 % a 33 % da bateria.

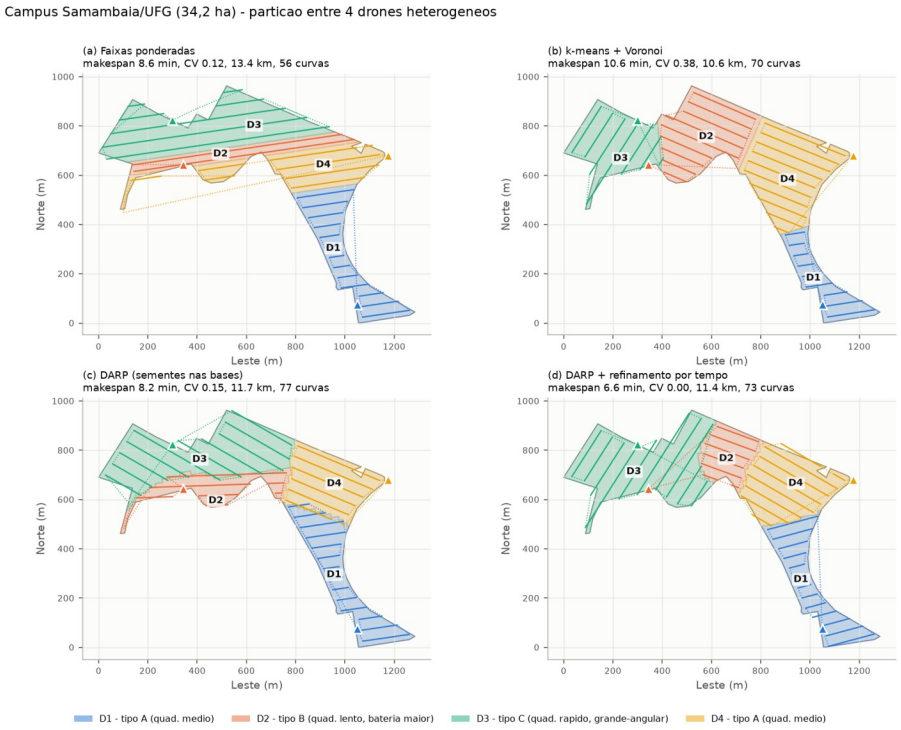


Figura 29. Partição do campo aberto do Campus Samambaia entre 4 drones heterogêneos pelos quatro métodos, com as rotas em boustrophedon (linhas cheias: câmera ligada; pontilhadas: trânsito e curvas) e as bases indicadas por triângulos

Tabela 24. Médias sobre os 4 cenários e N = 2 a 6 (20 casos por linha; vitórias = casos com o menor makespan)

Frota	Método	*Makespan* médio (min)	*Makespan* / melhor do caso	CV do tempo	Vitórias
Homogênea	Faixas	11,66	1,153	0,097	2
Homogênea	k-means	11,47	1,100	0,117	2
Homogênea	DARP	11,37	1,107	0,055	1
Homogênea	DARP + ref.	10,64	1,008	0,023	15
Heterogênea	Faixas	12,68	1,189	0,123	3
Heterogênea	k-means	15,90	1,504	0,308	0
Heterogênea	DARP	12,04	1,119	0,075	1
Heterogênea	DARP + ref.	10,96	1,007	0,028	16

O DARP com refinamento por tempo obteve o menor *makespan* em 31 dos 40 casos, ficou em média a 0,7–0,8 % do melhor resultado de cada caso e reduziu o coeficiente de variação (CV) dos tempos de missão para 0,023–0,028, contra 0,055–0,308 dos demais métodos. Com frota heterogênea, o método k-means/Voronoi, por não ser ponderado, ficou em média 50 % acima do melhor *makespan*. A cobertura rasterizada ficou entre 99,65 % e 100 % nas 160 execuções, e as medianas de tempo de execução foram de 0,4 a 1,2 s para os métodos de passo único e de 5,8 a 7,1 s para o DARP + refinamento. A ablação mostrou que a ponderação multiplicativa original não convergiu no polígono real da UFG (erro máximo de área de 24,9 %, contra 0,6 % a 1,2 % da ponderação aditiva) e que o refinamento por tempo foi o fator de maior ganho.

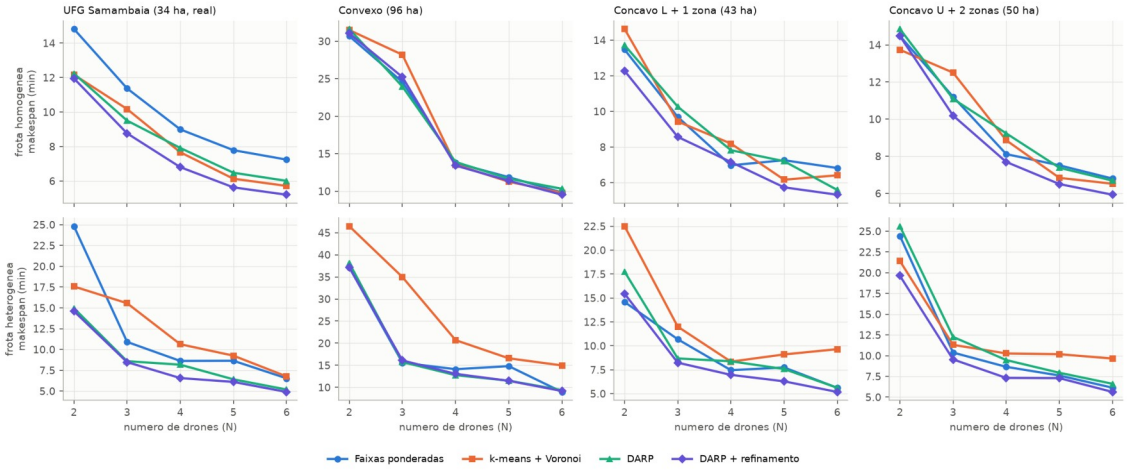


Figura 30. Makespan em função do número de drones para os quatro métodos, por cenário (colunas) e tipo de frota (linhas)

Em frotas heterogêneas, a ponderação pela capacidade de cobertura e, sobretudo, o ajuste dos pesos pelo tempo estimado de missão são determinantes para o balanceamento. As principais limitações são os tempos de missão derivados de um modelo cinemático simples (velocidade constante, penalidade fixa por curva e ausência de vento), não calibrado com voos; a validação em planta e na simulação 3D descrita adiante, sem SITL nem voo real; e o uso do k-means/Voronoi sem ponderação.

4.14. Colaboração entre drones e realocação dinâmica

Esta etapa trata dos algoritmos de colaboração entre drones previstos no segundo eixo do plano de trabalho e complementa a Atividade 13. O objetivo foi tratar o abandono da missão por um drone, por falha ou por bateria baixa, redistribuindo a área ainda não varrida entre os sobreviventes, e quantificar o custo de comunicação dessa coordenação no enlace LoRa. Toda a avaliação foi feita por simulação 2D, com o mesmo modelo cinemático da segmentação.

4.14.1. Metodologia e implementação

O cenário de referência é o plano final da UFG com quatro drones heterogêneos, cujo *makespan* sem falha é de 394 s (6,57 min); a falha é detectada pela ausência de mensagens HEARTBEAT, com atraso de 3 s. Foram simuladas 12 falhas (cada drone abandonando a missão a 25 %, 50 % e 75 % do seu voo) e comparadas três estratégias: **sem realocação**, em que a parte não varrida da subárea do drone perdido é abandonada; **repartição**, em que a área ainda não coberta é repartida entre os sobreviventes, a partir de suas posições atuais, pelo DARP com refinamento pelo instante de conclusão, com as novas subáreas difundidas por mensagens AREA_ASSIGN e AREA_POLY; e **leilão**, em que a área órfã é dividida em tarefas distribuídas por leilão

sequencial de item único com objetivo minimax [51], por mensagens TASK_BID e TASK_AWARD.

As mensagens seguem a biblioteca `cerise_proto` v1 do protótipo LoRa, codificadas byte a byte pelo módulo `cerise_drones.protocolo`. Como o AREA_ASSIGN v1 descreve apenas um retângulo envolvente, insuficiente para as subáreas poligonais do DARP, foi proposta a extensão com os tipos AREA_POLY (até 7 vértices por quadro, com fragmentação), TASK_BID e TASK_AWARD, incorporada à biblioteca do protótipo, mas ainda não exercitada com rádios. Os tempos no ar foram calculados pela formulação da Semtech [43].

4.14.2. Resultados

Tabela 25. Efeito do abandono de um drone no plano da UFG com 4 drones heterogêneos: área perdida sem realocação, aumento do makespan por estratégia e custo LoRa sem confirmações (4 falhas por instante)

Instante da falha	Perda sem realocação (%)	*Makespan*, repartição (%)	*Makespan*, leilão (%)	Quadros LoRa (rep./leilão)	Tempo no ar em SF9 (s) (rep./leilão)
25 % do voo	16,7	+33,3	+49,2	20,0 / 59,8	5,5 / 10,5
50 % do voo	9,5	+18,3	+32,2	15,0 / 38,5	4,0 / 6,8
75 % do voo	1,4	+17,8	+12,2	8,0 / 12,8	2,0 / 2,3
Média (12 casos)	9,2 (máx. 26,4)	+23,1	+31,2	14,3 / 37,0	3,8 / 6,5

Sem realocação, em média 9,2 % da área permanece descoberta, chegando a 26,4 % quando o drone rápido do tipo C abandona a missão a 25 % do voo. As duas estratégias colaborativas recuperaram pelo menos 99,7 % de cobertura em todos os casos. A repartição teve menor aumento de *makespan* em 9 dos 12 casos; o leilão foi melhor em três, dois deles com falha a 75 % do voo, quando a área órfã é pequena, e um praticamente empatado (+75,8 % nas duas estratégias). Uma política híbrida que escolhesse a melhor das duas teria aumento médio de 20,6 %. O replanejamento levou em média 1,95 s na repartição e 0,08 s no leilão.

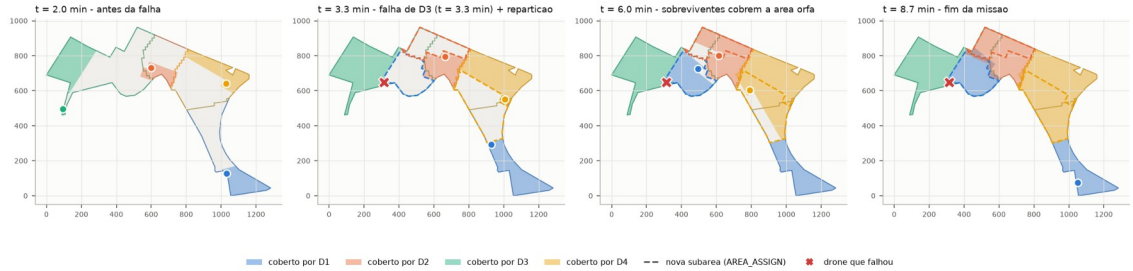


Figura 31. Linha do tempo da realocação por repartição quando o drone D3 abandona a missão a 50 % do voo: áreas já cobertas coloridas por drone e novas subáreas tracejadas; makespan de 6,6 para 8,7 min, com 100 % de cobertura

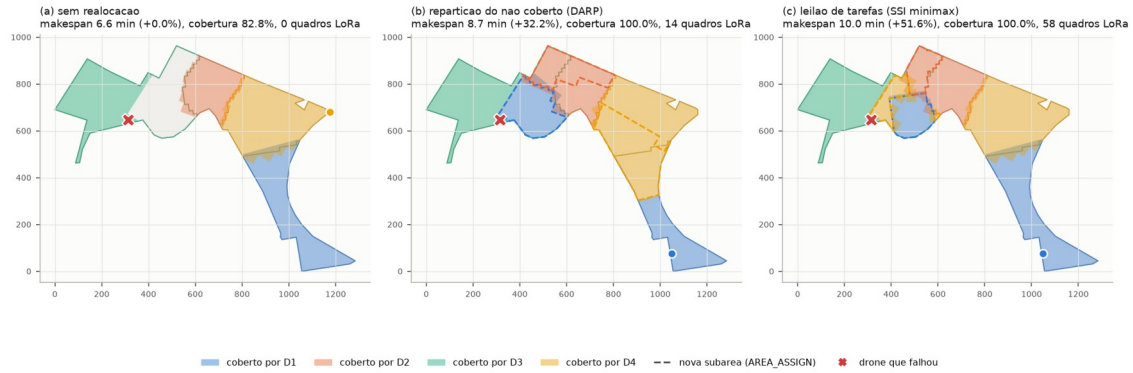


Figura 32. Estado final das três estratégias para a falha de D3 a 50 % do voo: sem realocação (82,8 % de cobertura), repartição (+32,2 % de makespan, 14 quadros LoRa) e leilão (+51,6 %, 58 quadros)

Os tempos no ar dos tipos do `cerise_proto` v1 foram apresentados na seção 4.12; a tabela a seguir traz os da extensão proposta.

Tabela 26. Tamanho e tempo no ar das mensagens da extensão proposta (BW 125 kHz, CR 4/5)

Mensagem	Quadro (B)	SF7 (ms)	SF9 (ms)	SF10 (ms)
AREA_POLY (7 vértices)	42	87,3	287,7	534,5
TASK_BID	14	46,3	164,9	288,8
TASK_AWARD	13	46,3	164,9	288,8

Em SF9, configuração padrão do firmware, todos os quadros de coordenação ficam abaixo do limite de 400 ms; em SF10, AREA_ASSIGN e AREA_POLY o ultrapassam. Em média, uma repartição custou 14,3 quadros (537 B e 3,8 s no ar em SF9) e um leilão, 37,0 quadros (608 B e 6,5 s), sem contar confirmações. A repartição é mais vantajosa quando a falha ocorre cedo e a área órfã é grande; o leilão é competitivo para áreas órfãs pequenas, o que sugere uma política híbrida, ainda não implementada. A simulação assume detecção da falha em 3 s e canal sem perdas, e a realocação não foi testada com os módulos de rádio nem em voo.

4.15. Detecção e contagem de pessoas em imagens aéreas

Esta etapa corresponde ao item 1 do Objetivo 5: planejamento e adequação de um projeto com drones para detecção e contagem de pessoas. Foram validados os pipelines de detecção e de contagem em imagens aéreas reais do conjunto público VisDrone2019-DET [52], desenvolvido o método de agregação das contagens entre drones e levantados os requisitos de adequação das aeronaves. Não foram coletadas imagens próprias do Campus Samambaia, e a adequação física dos drones permanece pendente.

4.15.1. Dados, treinamento e resultados

As categorias *pedestrian* e *people* do VisDrone2019-DET foram unidas numa classe *pessoa*: o treino tem 6.471 imagens e 106.396 pessoas; a validação, 548 imagens e 13.969 pessoas; e o test-dev, 1.610 imagens e 27.382 pessoas. O treinamento e a avaliação foram executados na estação DGX Spark, com a biblioteca Ultralytics 8.4.189 [53] e GPU compartilhada, com uso de memória limitado a 7,5 GB (pico de 6,5 GB). No GB10, o *autocast* em FP16 produziu valores NaN, razão pela qual treino e inferência foram feitos em FP32. O ajuste fino partiu do YOLO11s com pesos COCO, com imagens de 960 px, lote 4 e limite de tempo de 0,8 h (o treino conclui a época em curso), o que resultou em 5 épocas em 48,7 min.

Tabela 27. Classe pessoa no VisDrone2019-DET (validador Ultralytics, FP32, conf 0,001, IoU 0,7)

Modelo	Resolução (px)	Conjunto	mAP50	mAP50-95	Revocação
YOLO11n COCO	960	val	0,274	0,111	0,285
YOLO11m COCO	960	val	0,433	0,194	0,401
YOLO11s COCO	1280	val	0,443	0,201	0,417
YOLO11s COCO	960	val	0,384	0,163	0,363
YOLO11s ajustado	960	val	0,528	0,220	0,473
YOLO11s COCO	960	test-dev	0,183	0,077	0,192
YOLO11s ajustado	960	test-dev	0,316	0,121	0,312

As linhas do YOLO11s a 960 px foram avaliadas com até 1.000 detecções por imagem, e as demais, com o padrão de 300. O ajuste fino elevou o mAP50 de pessoa de 0,384 para 0,528 na validação (+37 %) e de 0,183 para 0,316 no test-dev (+73 %), superando o YOLO11m COCO a 960 px e o YOLO11s COCO a 1280 px. Para a contagem, considera-se o número de detecções acima de um limiar de confiança escolhido na validação (mínimo erro absoluto médio, MAE) e aplicado ao test-dev; o erro percentual absoluto médio (MAPE) foi calculado apenas sobre as imagens com pelo menos uma pessoa.

Tabela 28. Erro de contagem de pessoas por imagem, com limiar escolhido na validação e aplicado ao test-dev

Modelo	Limiar	MAE val	MAPE val (%)	MAE test-dev	MAPE test-dev (%)
YOLO11s COCO	0,05	9,91	44,8	11,44	81,7
YOLO11s ajustado	0,175	6,91	31,3	8,76	61,2

O modelo ajustado reduziu o MAE em cerca de 30 % na validação e 23 % no test-dev, com viés de subcontagem: o erro do total de pessoas foi de −13,8 % na validação e de −42,8 % no test-dev, que concentra pessoas muito pequenas e imagens noturnas. Com lote 1 e FP32, o YOLO11s ajustado atingiu 25,5 imagens/s de ponta a ponta a 960 px; como os quatro drones do plano da UFG somam cerca de 3,3 fotos/s, a vazão medida, cerca de 8 vezes superior, permite processar as imagens de vários drones numa única estação de solo.

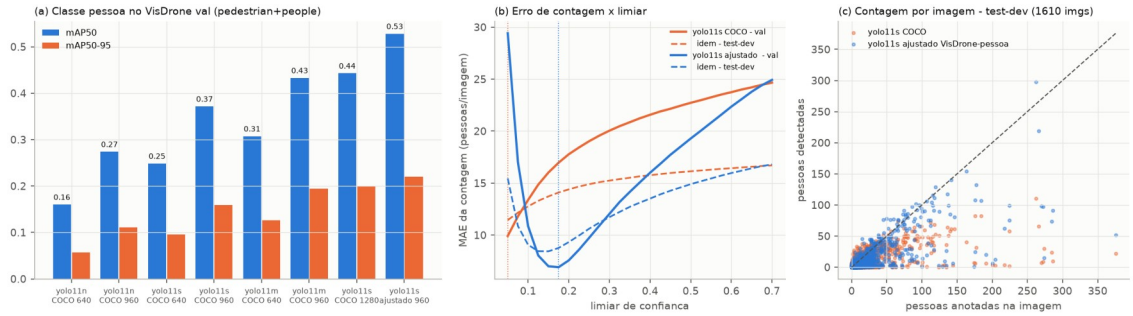


Figura 33. Métricas de detecção e contagem: mAP de pessoa na validação para os modelos COCO (com o limite padrão de 300 detecções) e o ajustado; MAE da contagem em função do limiar; e pessoas detectadas versus anotadas por imagem no test-dev



Figura 34. Exemplos da validação do VisDrone com as detecções do modelo ajustado (limiar 0,175, exibido como “0.17” nos rótulos) e as contagens real e detectada por imagem

4.15.2. Agregação das contagens entre drones

Como as subáreas são disjuntas, a contagem total é a soma das contagens de cada subárea, desde que cada pessoa seja contada uma única vez. O método implementado (`cerise_drones.agregacao`) georreferencia cada detecção pela posição, altitude e rumo do drone (câmera em nadir e terreno plano); atribui a detecção apenas ao drone dono da subárea em que o ponto cai (regra de posse); agrupa detecções a menos de um raio `r` para eliminar repetições entre fotos sobrepostas; e, numa faixa de fronteira, faz o agrupamento entre drones, que trocariam as detecções de borda por mensagens COUNT_REPORT.

A lógica foi validada apenas por simulação, com 348 pessoas sintéticas estáticas no plano da UFG, 619 fotos e ruído de georreferenciamento σ. Com detector ideal, σ = 0,8 m e r = 3 m, a soma ingênua superestimou a contagem em 300,5 %, a deduplicação sem regra de posse em 6,8 ± 3,4 %, e o método proposto errou −1,1 ± 1,8 %. A regra de posse deslocou a estimativa para baixo em 7,4 a 19,9 pontos percentuais nas configurações simuladas, o que reduz o erro quando há sobrecontagem, mas acentua a subcontagem quando o raio é grande em relação a σ (com σ = 0,3 m e r = 2 m, de +4,0 % para −6,8 %). Com a revocação e a precisão medidas (0,473 e 0,638) e falsos positivos independentes entre fotos, hipótese pessimista, o erro foi de +102,7 %, dominado pelos falsos positivos.

4.15.3. Requisitos, discussão e limitações

A adequação física dos drones não foi executada por depender de hardware. Além do módulo LoRa, foram levantados como requisitos: câmera em nadir com pelo menos 12 MP (resolução no solo inferior a 1,5 cm/pixel a 40 m) e disparo por distância; processamento das imagens na estação de solo após o pouso, cenário validado nesta etapa, ou em computador embarcado, o que exige revisar massa e autonomia; e GNSS com correção RTK ou PPK, que permite raios de deduplicação de 1 a 2 m. O ajuste fino foi curto (5 épocas), e o VisDrone contém majoritariamente imagens oblíquas de cenas urbanas, distintas das imagens em nadir a 40 m previstas para o campus; a agregação foi validada apenas com pessoas sintéticas, e o resultado com o detector medido indica a necessidade de confirmação multivista antes do uso operacional. Os próximos passos incluem coletar e anotar imagens próprias do campus e treinar por mais épocas.

Código e evidências da segmentação, da colaboração e da detecção: repositório cerise-2026-drones-segmentacao (privado)

4.16. Validação da contagem em vídeos reais de drone

A etapa anterior mediu o detector YOLO11s ajustado em imagens estáticas. A missão de contagem no Campus Samambaia, porém, produz vídeo, e nele a mesma pessoa aparece em dezenas de quadros. Por isso, esta atividade mediu em vídeo aéreo real, com gabarito de identidades: o erro da contagem por quadro; o erro da contagem de pessoas distintas obtida pelos identificadores (IDs) de um rastreador multiobjeto; as métricas padronizadas de rastreamento da classe pessoa; e a vazão do pipeline na estação NVIDIA DGX Spark (GB10).

4.16.1. Dados e metodologia

Foram usadas as partições val (7 sequências, 2.846 quadros) e test-dev (17 sequências, 6.635 quadros, gabarito público) do VisDrone2019-MOT [52], obtidas dos links oficiais e conferidas por SHA-256. A classe pessoa reúne `pedestrian` e `people`, e as regiões marcadas como ignoradas são excluídas, como no protocolo do conjunto. O test-dev contém 1.012 pessoas distintas em cenas majoritariamente oblíquas e urbanas, com resoluções de 680×382 a 2720×1530 e média de 1,1 a 109,3 pessoas por quadro nas 16 sequências com pessoas (até 148 em um único quadro); a sequência restante não contém pessoas.

Foram avaliados quatro detectores em FP32: o YOLO11s ajustado no VisDrone-DET, a 960 e a 1280 px, e o YOLO11s com pesos COCO nas mesmas resoluções, como referência [53]. As detecções foram calculadas uma vez e guardadas em cache, com confiança mínima de 0,01. Sobre elas rodaram, sem modificação de código, os rastreadores do Ultralytics: ByteTrack [54] e BoT-SORT [55], este com compensação de movimento de câmera (GMC, fluxo óptico esparso) e sem reidentificação por aparência.

Todas as escolhas foram feitas apenas no val e aplicadas sem alteração ao test-dev:

- a grade de limiares do rastreador (1.904 execuções), pelo maior HOTA;
- o limiar da contagem por quadro, pelo menor erro absoluto médio (MAE);
- o comprimento mínimo de trilha L da contagem de IDs, pelo menor erro relativo.

Para o detector ajustado a 960 px com BoT-SORT, a configuração escolhida foi `track_high_thresh` = `new_track_thresh` = 0,40, `track_low_thresh` = 0,05 e `track_buffer` de 30 quadros. As métricas HOTA [56], MOTA [57] e IDF1 [58] foram reimplementadas seguindo o TrackEval [59]. A implementação foi

conferida com casos sintéticos e com o py-motmetrics em 72 pares sequência × combinação: o IDF1 coincidiu e a diferença máxima de MOTA foi de 0,0026.

4.16.2. Resultados

Tabela 29. Rastreamento e contagem de pessoas no VisDrone2019-MOT test-dev (16 sequências com pessoas; 1.012 pessoas distintas), com as configurações escolhidas no val; IDs brutos = trilhas com L = 1

Combinação	HOTA	IDF1	MOTA	Trocas de ID	MAE por quadro (trilhas)	IDs brutos	IDs com ≥ L quadros
Ajustado 960 + ByteTrack	0,286	0,339	0,095	1.639	8,46	2.684	500 (L = 60)
Ajustado 960 + BoT-SORT	0,328	0,399	0,092	974	7,46	2.095	584 (L = 60)
Ajustado 960 + BoT-SORT sem GMC	0,289	0,343	0,094	1.589	—	2.639	—
COCO 1280 + BoT-SORT	0,328	0,406	0,214	444	11,14	1.173	599 (L = 30)

As duas melhores combinações empataram em HOTA (0,328), por caminhos opostos. O detector ajustado recupera mais pessoas pequenas (revocação de detecção de 0,358 contra 0,278), mas tem menor precisão (0,458 contra 0,574), o que derruba o MOTA. O BoT-SORT superou o ByteTrack em todas as combinações (+0,035 a +0,042 de HOTA). A ablação atribui essa vantagem à GMC: sem ela, o HOTA do ajustado cai de 0,328 para 0,289 e as trocas de ID sobem de 974 para 1.589. Na contagem por quadro, com média de 20,8 pessoas anotadas por quadro, as trilhas do ajustado com BoT-SORT tiveram MAE de 7,46 e viés de −4,52. Todos os estimadores subcontam, sobretudo em multidões.

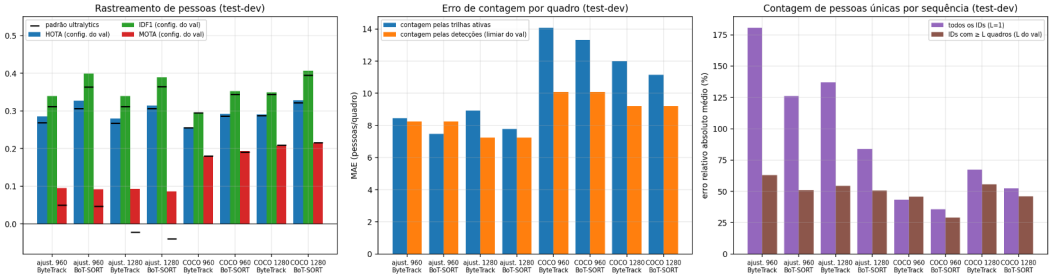


Figura 35. HOTA, IDF1 e MOTA, MAE da contagem por quadro e erro da contagem de pessoas únicas no VisDrone2019-MOT test-dev para as oito combinações detector × rastreador

O resultado central para a missão é que contar IDs do rastreador não é contar pessoas. Com o ajustado e o BoT-SORT, os 2.095 IDs brutos (+107 % em relação às 1.012 pessoas) decompõem-se em:

- 861 IDs falsos, isto é, trilhas curtas sobre falsos positivos;

- 1.234 IDs associados a 816 pessoas, o que corresponde a 418 IDs extras por troca ou fragmentação.

Outras 196 pessoas nunca foram rastreadas. O filtro por comprimento mínimo troca inflação por subcontagem: com L = 60, o total cai para 584 (−42 %). Entre as configurações com L escolhido apenas no val, isto é, sem uso do test-dev, a melhor, do COCO a 960 px com L = 10, ainda tem 29 % de erro relativo médio por sequência.

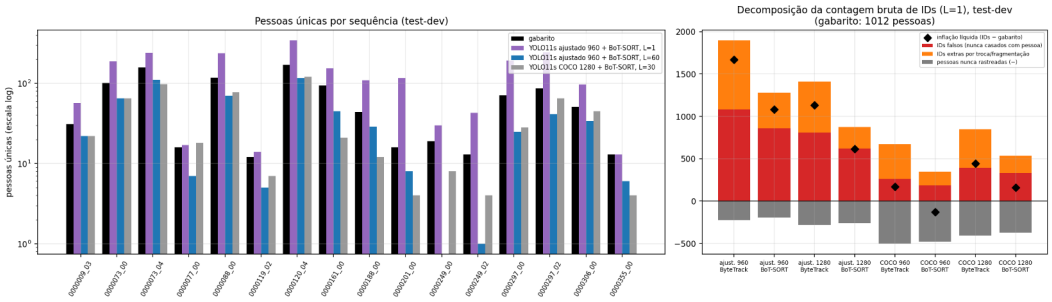


Figura 36. Pessoas únicas por sequência do test-dev (gabarito, IDs brutos e IDs com ≥ L quadros) e decomposição dos IDs brutos em IDs falsos, IDs extras por troca ou fragmentação e pessoas nunca rastreadas

A escala da pessoa na imagem, que funciona como indicador da altitude, foi o fator mais associado ao desempenho, com correlação de Spearman ρ = 0,62 (p = 0,011) entre a altura mediana e o HOTA. A revocação do ajustado foi de 12 % abaixo de 20 px de altura, 30 % entre 20 e 40 px, 56 % entre 40 e 80 px e 66 % acima de 80 px; para o COCO, os valores foram 2 %, 15 %, 43 % e 75 %. O ajuste no VisDrone compensa justamente a faixa de pessoas pequenas de uma missão em nadir a 40 m. Quanto à vazão, o pipeline com BoT-SORT rodou a 23,0 quadros/s em 4K e a 68,9 quadros/s em 1344×756, com no máximo 0,33 GB de GPU.

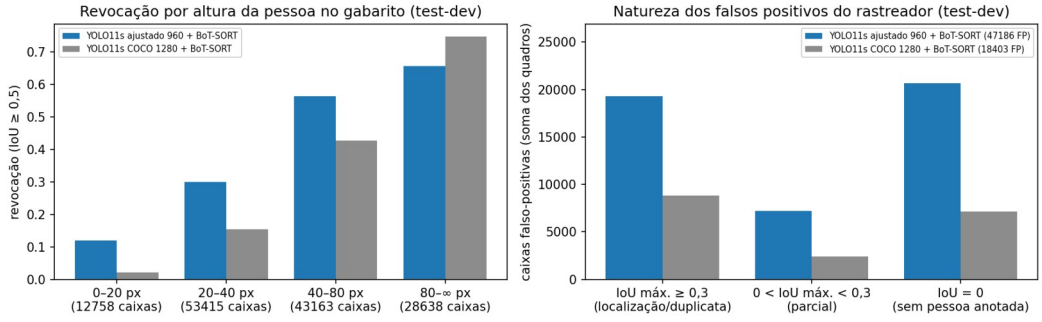


Figura 37. Revocação por faixa de altura da pessoa e natureza dos falsos positivos do rastreador no test-dev (ajustado 960 × COCO 1280, com BoT-SORT)

Três decisões foram levadas para a simulação: usar o detector ajustado com BoT-SORT e GMC; tratar as trilhas apenas como agregadoras de uma mesma passagem; e fazer a contagem única pela posição projetada no solo, não pelos IDs. Como limitações, o VisDrone2019-MOT é oblíquo, urbano e filmado a

baixa altura, ao passo que a missão da UFG é em nadir sobre um campus. Além disso, o val tem só 7 sequências e o TrackEval original não foi executado.

Código e evidências: cerise-2026-drones-segmentacao (privado)

4.17. Simulação 3D de drones contando pessoas

Para fechar o ciclo entre a partição da área, o detector e a validação em vídeo, foi construída uma simulação fotorrealista no NVIDIA Isaac Sim 5.1 [60], executada sem tela (headless) no DGX Spark. A cena reproduz um recorte de 11,27 ha da área aberta da UFG, com terreno, caminhos, 14 edifícios esquemáticos e 78 árvores gerados a partir do polígono do OpenStreetMap. Quatro drones percorrem as rotas DARP + refinamento [47] da frota heterogênea, em nadir a 40 m (1280×960 e campo de visão horizontal de 70°; no D3, grande-angular, 1280×720 e 82°). Na cena caminham **80 pessoas animadas**: caminhantes, grupos andando ou parados e pessoas que cruzam as fronteiras entre subáreas.

As pessoas usam personagens e comportamentos do `omni.anim.people` e seguem os comandos do plano sobre uma navmesh. O Replicator grava, a 5 fps, por câmera: RGB, caixas 2D de gabarito e a pose; o gabarito de posição é a posição real de cada personagem. A licença foi aceita pelo bolsista, sem telemetria, e os ativos oficiais (0,55 GB) foram espelhados localmente. Nada foi exposto à rede externa.

Os testes de fumaça revelaram quatro divergências de API, todas corrigidas; entre elas:

- o esqueleto `biped_demo` ausente impedia o grafo de animação;
- o volume da navmesh ficava restrito a ±4,5 m da origem, o que fazia as pessoas andarem em linha reta.

Depois das correções, as caixas do Replicator coincidiram com a projeção do gabarito pela pose, com erro mediano de 2,8 px. O voo a 40 m (866 instantes × 5 câmeras, incluindo a visão geral) levou 21,0 min, ou 1,46 s por instante. Ele ocupou de 18 a 21 GB de memória unificada e manteve sempre mais de 24 GB livres no Spark. Foi feito também um voo a 30 m (1.001 instantes). O maior lado mediano das caixas de gabarito foi de 21 px a 40 m e de 31 px a 30 m.

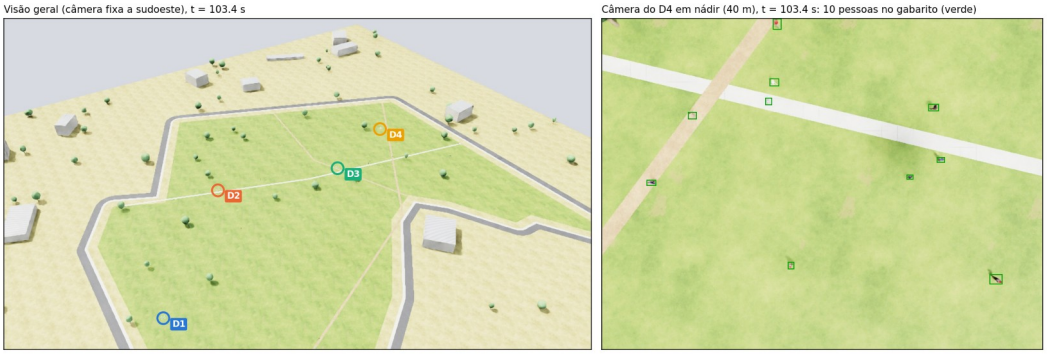


Figura 38. Cena renderizada no Isaac Sim: visão geral com os quatro drones e câmera do D4 em nadir a 40 m com as caixas de gabarito do Replicator

Como a visão geral e a câmera a 40 m não mostram o realismo da cena, foi renderizado um trecho de 14 s (24 quadros/s, 1920×1080) com uma câmera baixa e oblíqua, a 5,2 m de altura e a 9,5–15 m de um grupo de três pessoas animadas, enquanto um quadricóptero de 0,9 m passa sobre elas a 8,5 m. A câmera fica 24° para baixo e, na passagem, recua e levanta o enquadramento para incluir o drone; a cena usa céu HDRI, texturas fotográficas no chão e sombras do RTX em tempo real. Os 337 quadros, com os da câmera do drone, levaram 9,5 min no Spark (1,66 s por quadro gravado, descontado o avanço inicial da simulação), com pico de 13,3 GB no contêiner e no mínimo 25,0 GB livres no Spark, e o clipe (sim_closeup.mp4, 12,1 MB) está no repositório cerise-2026-drones-segmentacao. Na câmera do drone, o YOLO11s encontra 80 % das pessoas a 18–30 m em visada oblíqua, mas quase nenhuma abaixo de 10 m, quando elas são vistas de cima.

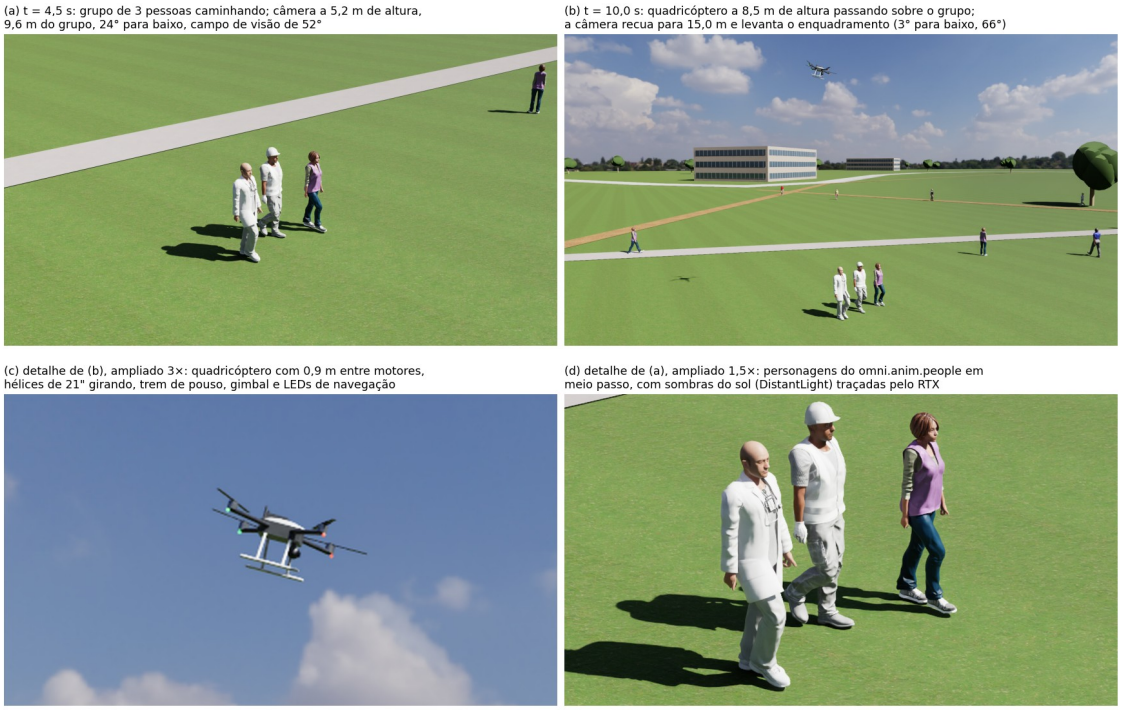


Figura 39. Close-up no Isaac Sim: (a) grupo de três pessoas animadas visto de 5,2 m de altura, 24° para baixo; (b) quadricóptero a 8,5 m passando sobre o grupo, com a sombra no gramado; (c) detalhe do drone; (d) detalhe das pessoas e das sombras

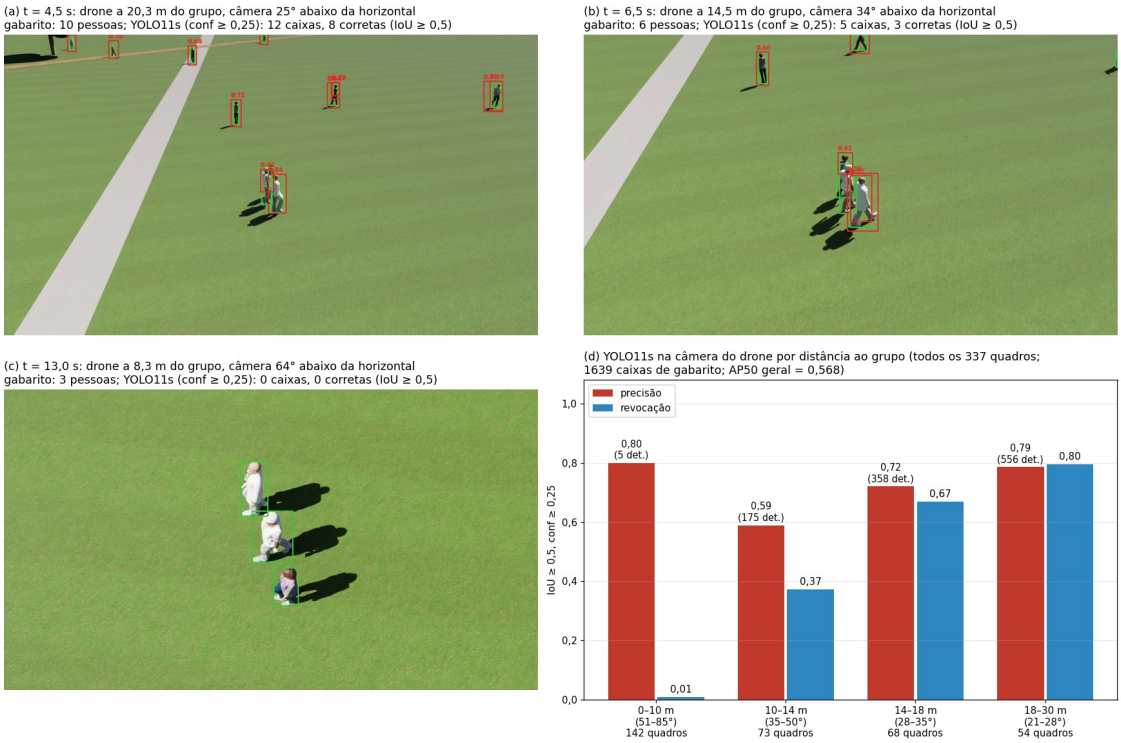


Figura 40. Câmera do drone no close-up a 20,3 m, 14,5 m e 8,3 m do grupo, com as caixas do gabarito (verde) e do YOLO11s (vermelho), e precisão e revocação do detector por faixa de distância

A contagem usa o YOLO11s ajustado (FP32, 1280 px) e projeta cada trilha no solo com a pose da câmera. Em seguida, aplica a posse da subárea, com faixa de fronteira de 6 m, e a deduplicação espaço-temporal pela posição. O detector localiza bem as pessoas simuladas: AP de 0,702 a IoU 0,5 a 40 m e de 0,652 a 30 m. As pontuações, porém, ficaram baixas no domínio simulado: no limiar 0,40 a precisão é de 0,998 e a revocação, de 0,230. O detector roda a 82 a 89 FPS, ou 56 a 60 FPS com leitura e GMC.

Tabela 30. Contagem única global na simulação contra as pessoas distintas vistas no gabarito (79 a 40 m; 73 a 30 m)

Voo	Rastreador + deduplicação no solo	Contagem	Erro	Corretos	Duplicatas	Perdidas
40 m	BoT-SORT recomendado pelos vídeos reais (pré-registrado)	34	−57,0 %	30	4	49
40 m	Rastreador próprio no solo (pré-registrado)	88	+11,4 %	68	20	11
40 m	BoT-SORT sem `fuse_score`, limiar 0,15 (escolhido neste voo)	78	−1,3 %	65	13	14
30 m	BoT-SORT recomendado	29	−60,3 %	29	0	44
30 m	Rastreador próprio no solo	94	+28,8 %	61	33	12
30 m	BoT-SORT sem `fuse_score`, limiar 0,15 (teste)	70	−4,1 %	54	16	19

Os resultados se resumem em três pontos:

- **Deduplicação.** A soma ingênua das detecções dá 1.306 para 79 pessoas, o que confirma que a deduplicação no solo é indispensável. Nenhuma configuração produziu clusters falsos.
- **Transferência da configuração dos vídeos reais.** A configuração recomendada subconta 57 % a 60 % porque o `fuse_score` padrão multiplica a IoU pela pontuação da detecção: com pontuações baixas, as trilhas novas não se confirmam. Sem GMC, nada é rastreado, pois a imagem se desloca de 28 a 71 px por quadro.
- **Configuração calibrada.** Escolhida a posteriori no voo a 40 m, ela errou −4,1 % no voo a 30 m. Esse total próximo do correto, contudo, vem da compensação entre 16 duplicatas e 19 perdidas.

As duplicatas vêm sobretudo do mesmo drone, por pessoas andando que são revistas na faixa seguinte da varredura. A posse e a faixa de fronteira limitaram as duplicatas entre drones a 2 ou 3 por voo.

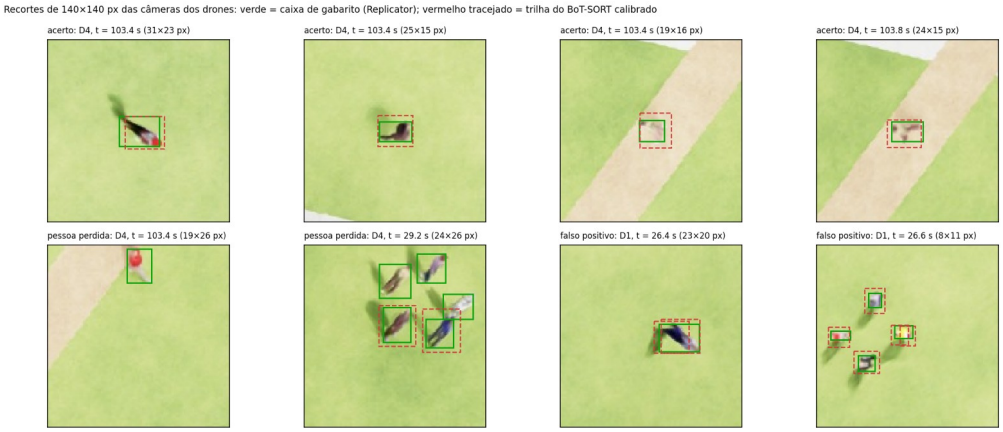


Figura 41. Recortes das câmeras dos drones a 40 m com acertos, pessoas perdidas e falsos positivos; verde = gabarito do Replicator, vermelho = trilha

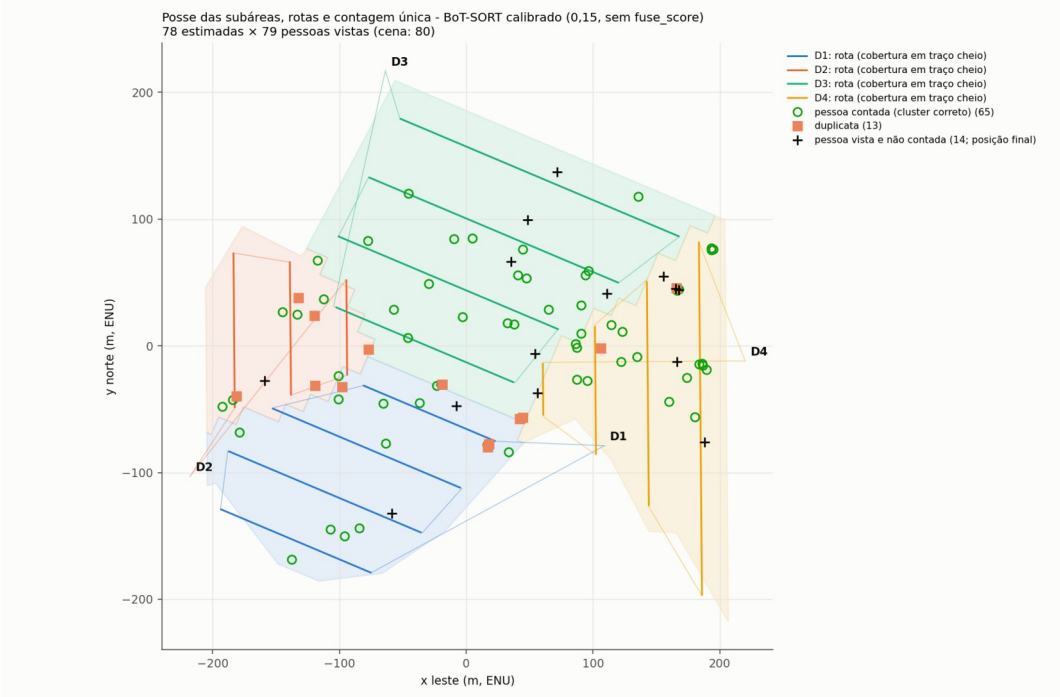


Figura 42. Mapa da contagem única no voo a 40 m: subáreas de posse, rotas, clusters corretos, duplicatas e pessoas não contadas

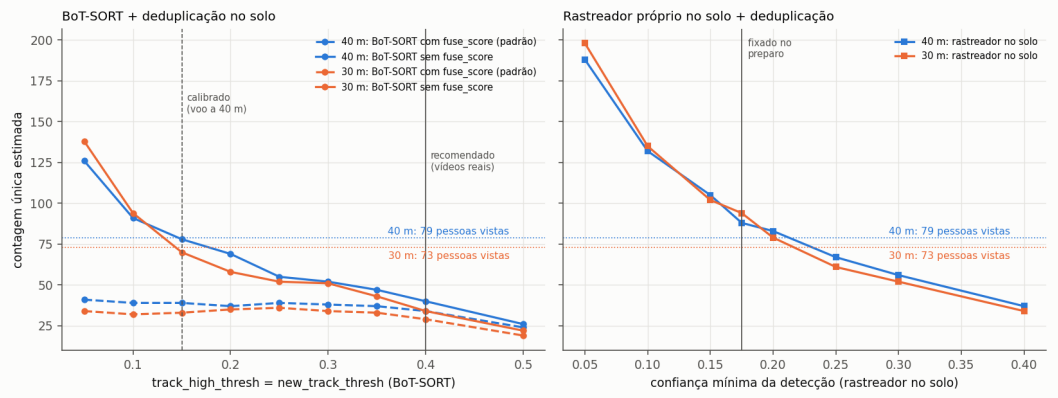


Figura 43. Sensibilidade da contagem única ao limiar do BoT-SORT (com e sem fuse_score) e à confiança mínima do rastreador no solo, a 40 m e a 30 m

O vídeo sim_video_run01.mp4 (H.264, 57,7 s), guardado no repositório cerise-2026-drones-segmentacao, mostra a visão geral com os drones em voo e as pessoas caminhando, além das câmeras dos drones com caixas, contagem atual e contagem acumulada.

O cenário completo (34,2 ha, 120 pessoas) não foi renderizado. A geração (*bake*) da navmesh de 1.330 × 998 m consumiu mais de 34 GB em ~22 s; o vigia de memória chegou a interromper o contêiner, mas não a tempo de evitar o esgotamento da memória do sistema (OOM, *out of memory*). O OOM reiniciou o worker do servidor 3D, que voltou sozinho; os serviços de produção não foram afetados. Os scripts passaram a impor limite de memória ao contêiner e um vigia mais rápido.

Ficam como limitações o terreno procedural, a atitude ideal dos drones (sem vento, borrão ou erro de pose), a velocidade fixa de caminhada, a diferença de domínio das pontuações do detector, apenas dois voos e a altitude confundida com o plano de voo. O plano B com Gazebo e PX4 SITL não foi necessário. Os próximos passos são:

- calibrar ou ajustar o detector com imagens sintéticas;
- repetir o cenário completo com a navmesh por subárea;
- validar em voo real no campus.

Código e evidências: cerise-2026-drones-segmentacao (privado)

4.18. Produção científica: artigos publicados e pôster

Esta subseção corresponde à parte de divulgação científica da Atividade 14, que prevê a escrita de artigo e sua submissão a congressos, simpósios ou periódicos com os resultados do projeto. Em 2026, o modelo de estimação de pose de braço robótico desenvolvido no projeto, o mesmo que serviu de ponto de partida para as Atividades 8 e 9 deste relatório, foi publicado em dois eventos, ambos sob o título *Estimated Position of the Moving Axes of a Mechanical Arm*

from an Unknown State: no International Symposium on Mechanism Design for Robotics (MEDER 2026) [61] e no Congresso Brasileiro de Automática (CBA 2026), realizado em São Paulo em edição especial de 50 anos [62], no qual o trabalho também foi apresentado em pôster.

Tabela 31. Artigos e pôster publicados em 2026 com os resultados do projeto

Produção	Autores	Evento	Formato
Artigo completo (8 páginas)	E. M. C. Caixeta; L. J. B. Gonçalves; A. A. Cardoso	MEDER 2026	Artigo
Artigo completo (6 páginas)	E. M. C. Caixeta; A. A. Cardoso; R. F. Almeida	CBA 2026 (São Paulo – SP)	Artigo e pôster

Os trabalhos propõem estimar a pose 3D de um manipulador articulado de 7 graus de liberdade (Franka Emika Panda) a partir de uma única imagem RGB, sem conhecimento prévio dos estados das juntas. A abordagem parte da arquitetura holística HRPE de Ban et al. [29] e acrescenta características de profundidade extraídas pelo Depth Anything V2 [30] (mapas de profundidade e de calor), que são concatenadas, comprimidas e fundidas ao ramo HRNet-W32 sem alterar a estrutura original da rede. O treinamento ocorre em duas fases, a sintética e a autossupervisionada, divididas em três etapas — pré-treino da DepthNet com dados sintéticos, treino do modelo completo com dados sintéticos e refinamento autossupervisionado com dados reais, guiado por consistência cinemática e de segmentação com renderização diferenciável —, a mesma metodologia relatada em dezembro de 2025.

No conjunto real do benchmark DREAM (câmera RealSense), o método atingiu 75,4 % de acurácia ADD com limiar de 60 mm (ADD@60 mm), ligeiramente acima do RoboPose (74,3 %) e da HRPE (75,2 %), usando apenas 8.000 amostras sintéticas de treino, contra mais de 26.000 na HRPE — uma redução de cerca de 70 % dos dados sintéticos e do custo de treino por época. Os dois artigos agradecem à FUNAPE pelo apoio financeiro por meio da bolsa do projeto "Implantação do CERISE – Centro de Excelência em Redes Inteligentes Sem Fio e Serviços Avançados". A figura a seguir mostra a primeira página dos dois artigos, e a seguinte, o pôster apresentado no CBA 2026.



(a)

Estimated Position of the Moving Axes of a Mechanical Arm from an Unknown State

Eduardo M. de C. Caixeta[1], Lucas de J. B. Gonçalves[2] and Alisson Assis Cardoso[3]

Federal University of Goiás (UFG), Goiânia, Goiás, Brazil
eduardocaixeta@discente.ufg.com[1]
lucasdejesus@discente.ufg.com[2]
alsnac@ufg.com[3]

Abstract. This work presents an improved methodology for monocular robot pose estimation by enhancing a holistic pose estimation framework through the integration of depth-aware feature extraction and self-supervised learning. The proposed approach incorporates a depth estimation network to extract complementary geometric information from RGB images, generating enriched feature representations through depth maps and heatmaps. These features are concatenated and compressed before being used in the original pose estimation architecture, preserving the network structure while improving representation quality. The training process consists of two stages. First, the model is trained with synthetic data to learn geometric relationships between robot joints, camera parameters, and 3D keypoints. Then, the model is refined using self-supervised learning with real data, guided by kinematic and segmentation consistency constraints instead of ground-truth labels. The self-supervised strategy uses forward kinematics reconstruction and differentiable rendering to maintain geometric consistency between predicted robot pose and image observations. Experimental results show that the proposed method achieves competitive performance in robot pose estimation tasks, slightly outperforming previous methods while maintaining stable training and strong generalization to real-world scenarios. These results indicate that combining synthetic pretraining, depth-enhanced features, and self-supervised adaptation is effective for improving monocular robot pose estimation.

Keywords: Robot Pose Estimation, Monocular Depth Estimation, Computer Vision, Deep Learning, Convolutional Neural Networks, Keypoint Detection Depth-Aware Feature Fusion.

1 Introduction

In recent years, Artificial Intelligence (AI) has become an essential tool in daily activities across society and industry (Pratti, 2025). Among its research areas, computer vision has stood out as a key driver of technological advances, particularly in healthcare, automation, and robotics (Barbosa, 2023).

In robotics, several recent studies focus on estimating robotic manipulator pose using RGB images combined with joint state information. Methods such as CtRNet-X and

(b)

Estimated Position of the Moving Axes of a Mechanical Arm from an Unknown State

Eduardo M. de C. Caixeta*. Alisson Assis Cardoso.**
Renato França de Almeida***

Federal University of Goiás (UFG), Goiânia, Goiás, Brazil
**(e-mail: eduardocaixeta@discente.ufg.br).*
*** (e-mail: alsnac@ufg.br)*
**** (e-mail: renatofrv@gmail.com)*

Abstract: This work presents an improved methodology for monocular robot pose estimation by enhancing a holistic pose estimation framework through the integration of depth-aware feature extraction and self-supervised learning. The proposed approach incorporates a depth estimation network to extract complementary geometric information from RGB images, generating enriched feature representations through depth maps and heatmaps. These features are concatenated and compressed before being used in the original pose estimation architecture, preserving the network structure while improving representation quality. The training process consists of two stages. First, the model is trained with synthetic data to learn geometric relationships between robot joints, camera parameters, and 3D keypoints. Then, the model is refined using self-supervised learning with real data, guided by kinematic and segmentation consistency constraints instead of ground-truth labels. The self-supervised strategy uses forward kinematics reconstruction and differentiable rendering to maintain geometric consistency between predicted robot pose and image observations. Experimental results show that the proposed method achieves competitive performance in robot pose estimation tasks, slightly outperforming previous methods while maintaining stable training and strong generalization to real-world scenarios. These results indicate that combining synthetic pretraining, depth-enhanced features, and self-supervised adaptation is effective for improving monocular robot pose estimation.

Resumo: O presente estudo propõe uma metodologia avançada voltada para a estimativa de pose de robôs monoculares. O método aprimora uma estrutura holística já existente por meio da integração de aprendizado autossupervisionado e extração de características conscientes de profundidade. Para isso, uma rede de estimativa de profundidade é incorporada ao sistema com o intuito de obter informações geométricas complementares das imagens RGB, gerando representações ricas por meio de mapas de calor e de profundidade. Essas características são compactadas e concatenadas, o que melhora a qualidade dos dados sem alterar a arquitetura original da rede. O treinamento é estruturado em duas etapas principais. Na primeira fase, o modelo utiliza dados sintéticos para compreender as relações geométricas entre os parâmetros da câmera, as articulações do robô e os pontos-chave 3D. Na segunda fase, ocorre o refinamento com dados reais por meio de aprendizado autossupervisionado. Esse processo utiliza a renderização diferenciável e a reconstrução por cinemática direta para manter a consistência entre as observações da imagem e a pose prevista, dispensando o uso de rótulos de ground-truth. Os resultados experimentais revelam que a abordagem atinge um desempenho competitivo, superando métodos anteriores. Além disso, destaca-se a forte capacidade de generalização para cenários reais e a estabilidade durante o treinamento. Tais resultados evidenciam que a união entre o pré-treinamento sintético, a adaptação autossupervisionada e as características aprimoradas por profundidade é uma estratégia eficaz para a estimativa de pose de robôs.

Keywords: Robot Pose Estimation; Monocular Depth Estimation; Computer Vision; Deep Learning; Convolutional Neural Networks; Keypoint Detection Depth-Aware Feature Fusion.

1. INTRODUCTION

In recent years, Artificial Intelligence (AI) has become an essential tool in daily activities across society and industry (Pratti, 2025). Among its research areas, computer vision has stood out as a key driver of technological advances, particularly in healthcare, automation, and robotics (Barbosa, 2023).

In robotics, several recent studies focus on estimating robotic manipulator pose using RGB images combined with joint state information. Methods such as CtRNet-X and RoboPose have demonstrated high accuracy in pose estimation tasks (Labbé, 2021; Lu, 2024). However, the need for prior knowledge of joint angles limits these approaches, reducing their applicability in scenarios where such data is unavailable or unreliable (Lu, 2023). As a result, there is an increasing

Figura 44. Primeira página dos artigos publicados: (a) MEDER 2026; (b) CBA 2026

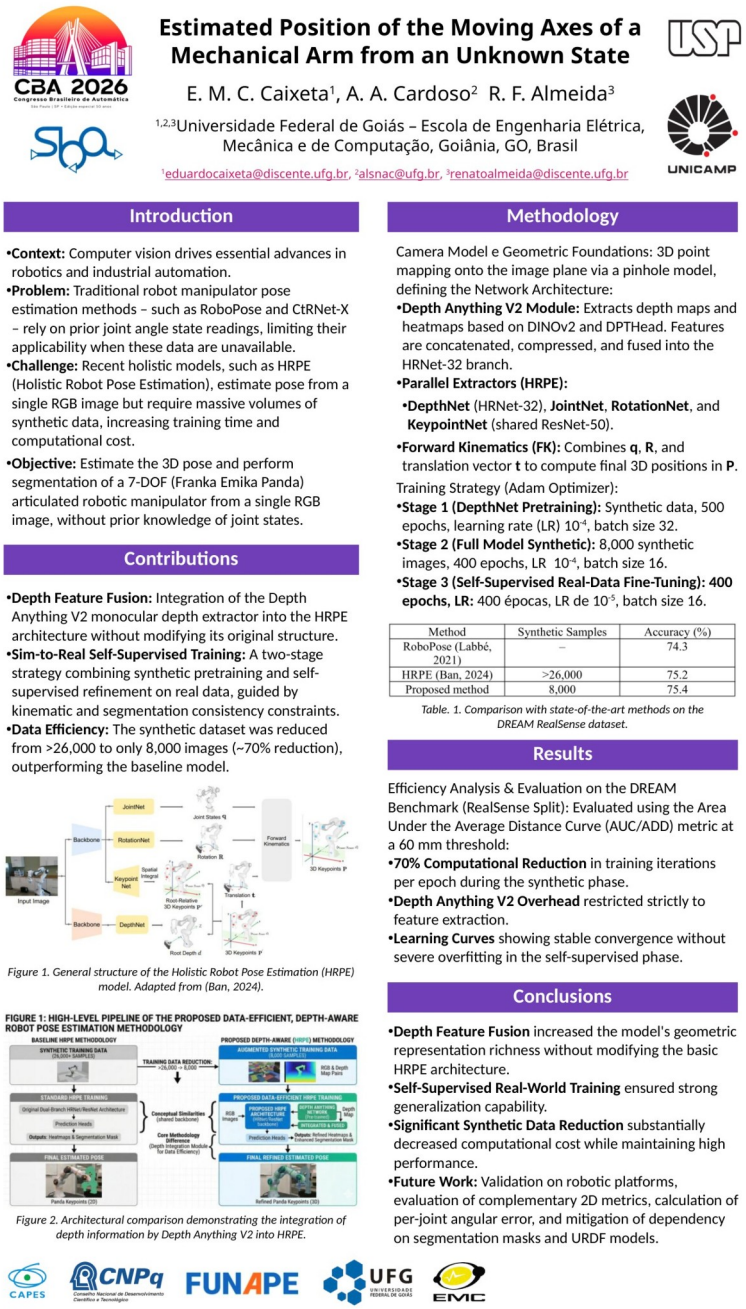


Figura 45. Pôster apresentado no CBA 2026, com a metodologia (fusão de profundidade do Depth Anything V2 à HRPE), a comparação com o estado da arte e as conclusões

Os artigos tratam do robô Panda e dos dados do DREAM; os resultados de 2026 com o braço servo6dof, o novo dataset e o gêmeo digital (Atividades 7 a 9, 11 e 12) ainda não foram publicados e constituem o próximo trabalho de divulgação.

4.19. Participação no CERISE Summit 2026

O bolsista participou do CERISE Summit 2026, evento de extensão organizado pelo CERISE UFG na Escola de Engenharia Elétrica, Mecânica e de Computação, cadastrado na PROEX UFG sob o código EV308-2026 e realizado em 18 de setembro de 2026. A participação ocorreu em duas funções, cada uma com carga horária de 10 horas: como expositor nas apresentações de projetos e de protótipos e como monitor do evento, conforme os certificados reproduzidos adiante.

Na sessão de apresentações, os projetos e protótipos do centro foram expostos em estandes ao público visitante, com demonstrações de robótica móvel e de realidade virtual e aumentada, além de painéis dos trabalhos, como ilustra a figura a seguir. Para o evento foi preparado, no aplicativo CERISE AR, o modo de jogo "Caça CERISE", uma caça ao tesouro em realidade aumentada com personagens animados e classificação por tempo, distribuído aos aparelhos por atualização remota (OTA).



Figura 46. Registros do CERISE Summit 2026: (a)–(e) estandes de projetos e protótipos, com demonstrações de robótica móvel, realidade virtual e aumentada; (f) equipe e visitantes do evento



Figura 47. Certificados do CERISE Summit 2026: (a) expositor das apresentações de projetos e protótipos; (b) monitor do evento

5. COMPARAÇÃO COM O PLANO DE TRABALHO

A tabela a seguir compara as 14 atividades do plano de trabalho, e os itens do Objetivo 5 e do segundo eixo do plano que não correspondem a uma atividade numerada, com o que foi efetivamente entregue. A situação foi atribuída a partir dos resultados medidos e das evidências guardadas nos repositórios: considera-se concluída a atividade cuja entrega prevista foi produzida e verificada, ainda que com ressalvas; parcial, aquela em que parte essencial da entrega não pôde ser realizada; e pendente, aquela que ficou apenas em projeto.

Tabela 32. Situação das atividades do plano de trabalho 2026

Atividade	Entrega prevista	Situação	Observação
1. Análise comparativa de modelos de geração 3D	Comparação de redes de geração 3D com textura e seleção do modelo	Concluída	10 modelos levantados e 6 medidos; seleção concluída; SF3D, SPAR3D e TRELLIS.2 aguardam aceite de licença no Hugging Face; TRELLIS sem pacote aarch64
2. Aplicativo com geração 3D	Versão funcional do aplicativo com objetos 3D texturizados	Concluída	Validada no emulador Android contra o servidor real, apenas na rede interna; RA dos modelos gerados não validada (exige aparelho físico); uso externo depende da exposição do servidor
3. Aplicativo otimizado e relatório de desempenho	Otimização e relatório de desempenho	Parcial	Tempos medidos; tamanhos de GLB a remedir após a correção da textura
4. Validação	Relatório de validação e resultados experimentais	Concluída	58/58 testes; sem fotos tiradas pela câmera do celular
5. Integração e servidor	Análise de gargalos e servidor para múltiplas requisições	Concluída	Erro de 90,9 % para 0 %; servidor não exposto publicamente

Atividade	Entrega prevista	Situação	Observação
6. Play Store	Publicação do aplicativo	Parcial	Pacote pronto e verificado; publicação depende das etapas no Play Console e de decisões pendentes
7. Dataset	Dataset estruturado e documentado	Concluída	Braço virtual; 5.000 poses e 30.000 imagens
8. Resultados do modelo	Resultados do modelo de pose de 2025 no novo dataset	Concluída	ADD@60 mm de 68,0 %; dados sintéticos
9. Avaliação	Avaliação de precisão e robustez	Concluída	Somente em simulação, sem imagens reais
10. Protótipo LoRa	Aquisição dos módulos e envio e recebimento entre dois dispositivos	Parcial	Software verificado sem rádio; módulos não adquiridos; ensaio físico pendente
11. Modelo digital	Modelo físico e cinemático do braço	Parcial	Cinemática validada contra o ROS 2 e as imagens; massas simbólicas e inércias de preenchimento (1 kg·m²); calibração depende do CAD ou do braço físico
12. Gêmeo digital	Gêmeo com o modelo de pose integrado	Concluída	Validado somente em simulação
13. Segmentação de áreas	Software funcional de segmentação entre drones	Concluída	Validação algorítmica e simulação 3D no Isaac Sim (4 drones, 80 pessoas); sem voo real
14. Relatório e artigo	Relatório técnico-científico e artigo submetido	Concluída	Relatório concluído; artigos publicados no MEDER 2026 e no CBA 2026, com pôster; resultados de 2026 com o servo6dof ainda não publicados
Obj. 5, item 1. Detecção e contagem de pessoas	Planejamento e adequação	Parcial	Pipeline validado no VisDrone (imagens e vídeos) e numa simulação 3D no Isaac Sim com 4 drones e 80 pessoas; adequação física dos drones pendente (hardware)
Obj. 5, itens 3 e 4. Módulos de comunicação e adequação dos drones	Construção e instalação	Pendente (somente projeto)	Placas LoRa não adquiridas
Algoritmos de colaboração entre drones (segundo eixo)	Realocação dinâmica e mensagens de coordenação	Concluída	Simulação 2D; sem rádios nem voo

Das 14 atividades numeradas, 10 foram concluídas e 4 ficaram parciais. Dos itens adicionais, os algoritmos de colaboração foram concluídos em simulação, o item 1 do Objetivo 5 ficou parcial e os itens 3 e 4 do Objetivo 5 ficaram pendentes, por falta de hardware. Todas as conclusões, porém, têm ressalvas que precisam ser explicitadas. As atividades com robótica foram realizadas inteiramente em simulação, com o braço virtual servo6dof, e a validação com o braço físico e com imagens reais não foi feita. A segmentação de áreas e a contagem de pessoas foram validadas em nível algorítmico e numa

simulação 3D fotorrealista dos voos no Isaac Sim, e a colaboração entre drones apenas em nível algorítmico; nenhum voo real foi realizado. Os tempos de geração 3D, de servidor e de inferência foram medidos numa estação compartilhada com serviços de IA em produção e devem ser lidos como limites superiores.

As pendências decorrem de cinco causas principais. A primeira é a falta de hardware: as placas LoRa e as antenas não foram adquiridas, de modo que o protótipo LoRa (Atividade 10) não passou pelo ensaio físico entre dois dispositivos e, no Objetivo 5, a construção dos módulos de comunicação definitivos e a adequação dos drones (itens 3 e 4) ficaram apenas em projeto, e a adequação física dos drones para a detecção e a contagem de pessoas (item 1) também ficou pendente; também a validação da RA dos modelos gerados depende de um aparelho físico. A segunda são as ações em plataformas externas ainda não executadas: a publicação na Play Store (Atividade 6) depende do Play Console, e a ampliação do *benchmark* de geração 3D aos modelos de acesso restrito (Stable Fast 3D, SPAR3D e TRELLIS.2), não necessária para a seleção já realizada, depende do aceite de licenças no Hugging Face. A terceira é a decisão, ainda não tomada, sobre a exposição pública do servidor de geração 3D e do visualizador do gêmeo digital: nada foi exposto à internet, e o aplicativo só funcionará fora da rede interna depois dessa exposição; a publicação da política de privacidade no endereço público previsto também aguarda decisão. A quarta é um defeito encontrado no fim do período: a recodificação da textura em JPEG não ocorria no pipeline empacotado, o que invalidou os tamanhos de arquivo medidos na rodada de *presets* (Atividade 3); o defeito foi corrigido no código do servidor e no repositório, mas a imagem Docker ainda precisa ser reconstruída e a rodada, refeita. A quinta é a calibração física do modelo digital do braço (Atividade 11), que depende do CAD ou de medições do braço físico. Quanto à Atividade 14, além deste relatório, os resultados de estimação de pose do projeto foram publicados em dois artigos (MEDER 2026 e CBA 2026, com pôster), e o bolsista participou do CERISE Summit 2026 como expositor e monitor.

6. PRÓXIMOS PASSOS

Os próximos passos foram priorizados conforme o quanto contribuem para concluir as entregas pendentes:

- **Decisões e ações do bolsista**: decidir sobre a exposição pública do servidor de geração 3D e do visualizador do gêmeo digital; publicar a política de privacidade no endereço previsto; criar o aplicativo no Play Console e conduzir o teste interno, o teste fechado e o pedido de produção; aceitar no Hugging Face as licenças do Stable Fast 3D, do

SPAR3D e do DINOv3 (codificador exigido pelo TRELLIS.2) para ampliar o *benchmark*; e definir como os avaliadores terão acesso aos repositórios privados.

- **Aquisição de hardware e ensaios de rádio**: adquirir as placas Heltec WiFi LoRa 32 V3 e as antenas da lista de materiais, gravar o firmware, executar o ensaio de alcance e de taxa de entrega e medir o consumo e a massa reais do módulo; em seguida, implementar o TDMA sincronizado por GPS, o salto de frequência e a cifragem com autenticação, encaminhar a homologação, testar a realocação entre drones com os rádios e instalar os módulos nos drones.
- **Remedições e consolidação**: reconstruir a imagem Docker do pipeline com a correção da textura e refazer, numa janela ociosa da GPU, a rodada de *presets*, os tamanhos de GLB e a suíte de carga do servidor; exigir textura JPEG nos testes automatizados; recalibrar a estimativa de tempo do *preset* alto; e ligar o monitoramento das métricas do servidor.
- **Validação em condições reais**: testar o aplicativo em aparelhos físicos Android e iOS, com fotografias da câmera, rede móvel e RA dos modelos gerados; calibrar massas e inércias do modelo digital a partir do CAD ou de medições do braço físico; capturar imagens do braço servo6dof físico para aplicar a etapa autossupervisionada de 2025 e validar o modelo de pose e o gêmeo digital fora da simulação, incluindo pontos-chave nas pinças para estimar a rotação e a abertura da garra; repetir a simulação no Isaac Sim no cenário completo (navmesh por subárea) e calibrar o detector com imagens sintéticas; executar as missões de drones no QGroundControl e em SITL antes de voos no Campus Samambaia; e coletar imagens próprias em nadir para ajustar o detector de pessoas.
- **Divulgação científica**: dar continuidade aos artigos do MEDER 2026 e do CBA 2026 com um trabalho sobre os resultados de 2026 — adaptação do modelo de pose ao braço servo6dof com o novo dataset e o gêmeo digital, e a geração 3D integrada ao aplicativo — para submissão a congresso ou periódico da área.

7. ANEXOS

O código-fonte, os resultados medidos (arquivos JSON e CSV), os registros de execução e as evidências citadas neste relatório estão nos repositórios listados a seguir, reunidos na organização cerise-ufg do GitHub. Os repositórios são privados, e o acesso pode ser concedido mediante solicitação ao bolsista. Os arquivos volumosos — o dataset completo do braço robótico nas

versões cinza e laranja, os pesos dos modelos treinados, as saídas brutas das simulações, os resultados da geração 3D e os pacotes do aplicativo — estão anexados às Releases de cada repositório, com somas SHA-256 e instruções de extração no arquivo RELEASES.md; dados e pesos de terceiros não são redistribuídos e têm a fonte documentada em cada README. Algumas figuras deste relatório foram recortadas, rearranjadas ou redesenhadas a partir das evidências originais, sem alteração dos dados, para melhorar a legibilidade.

O roteiro de implementação (Roteiro de Implementação CERISE 2026, em .docx e .pdf) está na pasta roteiro do repositório cerise-2026-relatorio e na pasta Roteiros do CERISE no Google Drive. Cada repositório de código traz a parte correspondente no arquivo ROTEIRO.md.

Tabela 33. Repositórios com o código, os resultados medidos e as evidências das atividades

Repositório	Conteúdo
cerise-2026-gen3d	Análise comparativa, otimização e validação da geração 3D; servidor de geração 3D (pasta server)
cerise-ar, branch feat/2026-gen3d-playstore	Aplicativo CERISE AR 1.1.0: integração com o servidor e pacote da Play Store (branch feat/2026-gen3d-playstore)
cerise-2026-servo6dof-gemeo-digital	Geração do dataset do braço (datasheet e amostra), modelo digital e gêmeo digital
cerise-2026-pose-servo6dof	Porte, treino e avaliação do modelo de estimação de pose e módulo de inferência
cerise-2026-lora-prototipo	Protocolo, firmware, testes, simulação e projeto do módulo de comunicação LoRa
cerise-2026-drones-segmentacao	Segmentação de áreas, colaboração entre drones, detecção e contagem de pessoas, validação em vídeos reais e simulação 3D no Isaac Sim
cerise-2026-relatorio	Este relatório (texto-fonte, .docx e .pdf), roteiro de implementação (pasta roteiro, .docx e .pdf), gerador, figuras e anexos (artigos, pôster, certificados e fotos do CERISE Summit 2026)

8. REFERÊNCIAS BIBLIOGRÁFICAS

[1] AZUMA, R. T. A survey of augmented reality. Presence: Teleoperators and Virtual Environments, v. 6, n. 4, 1997.

[2] BILLINGHURST, M.; CLARK, A.; LEE, G. A survey of augmented reality. Foundations and Trends in Human-Computer Interaction, v. 8, n. 2-3, 2015.

[3] CHEN, P.; LIU, X.; CHENG, W.; HUANG, R. A review of using augmented reality in education from 2011 to 2016. In: Innovations in Smart Learning. Singapore: Springer, 2017.

[4] BARSOM, E. Z.; GRAAFLAND, M.; SCHIJVEN, M. P. Systematic review on the effectiveness of augmented reality applications in medical training. Surgical Endoscopy, v. 30, n. 10, 2016.

[5] DOWNS, L. et al. Google Scanned Objects: a high-quality dataset of 3D scanned household items. In: IEEE INTERNATIONAL CONFERENCE ON ROBOTICS

AND AUTOMATION (ICRA), 2022, Filadélfia. Proceedings [...]. IEEE, 2022. arXiv:2204.11918.

[6] LAINE, S. et al. Modular primitives for high-performance differentiable rendering. ACM Transactions on Graphics, v. 39, n. 6, 2020.

[7] UMEYAMA, S. Least-squares estimation of transformation parameters between two point patterns. IEEE Transactions on Pattern Analysis and Machine Intelligence, v. 13, n. 4, p. 376-380, 1991.

[8] RADFORD, A. et al. Learning transferable visual models from natural language supervision. In: INTERNATIONAL CONFERENCE ON MACHINE LEARNING (ICML), 38., 2021. Proceedings [...]. PMLR, 2021. arXiv:2103.00020.

[9] ZHAO, Z. et al. Hunyuan3D 2.0: scaling diffusion models for high resolution textured 3D assets generation. arXiv:2501.12202, 2025.

[10] LAI, Z. et al. Unleashing vecset diffusion model for fast shape generation. arXiv:2503.16302, 2025.

[11] HUNYUAN3D TEAM. Hunyuan3D 2.1: from images to high-fidelity 3D assets with production-ready PBR material. arXiv:2506.15442, 2025.

[12] XIANG, J. et al. Structured 3D latents for scalable and versatile 3D generation. arXiv:2412.01506, 2024.

[13] TOCHILKIN, D. et al. TripoSR: fast 3D object reconstruction from a single image. arXiv:2403.02151, 2024.

[14] BOSS, M. et al. SF3D: stable fast 3D mesh reconstruction with UV-unwrapping and illumination disentanglement. arXiv:2408.00653, 2024.

[15] HUANG, Z. et al. SPAR3D: stable point-aware reconstruction of 3D objects from single images. arXiv:2501.04689, 2025.

[16] XU, J. et al. InstantMesh: efficient 3D mesh generation from a single image with sparse-view large reconstruction models. arXiv:2404.07191, 2024.

[17] RAMÍREZ, S. FastAPI. Disponível em: https://fastapi.tiangolo.com. Acesso em: 26 out. 2026.

[18] SQLITE. Write-Ahead Logging. Disponível em: https://www.sqlite.org/wal.html. Acesso em: 26 out. 2026.

[19] EXPO. Expo documentation. Disponível em: https://docs.expo.dev. Acesso em: 26 out. 2026.

[20] GOOGLE. model-viewer: easily display interactive 3D models on the web and in AR. Disponível em: https://modelviewer.dev. Acesso em: 26 out. 2026.

[21] REACTVISION. ViroReact: AR and VR for React Native. Disponível em: https://github.com/ReactVision/viro. Acesso em: 26 out. 2026.

[22] KHRONOS GROUP. glTF 2.0 Specification. Beaverton: Khronos Group, 2021. Disponível em: https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html. Acesso em: 26 out. 2026.

[23] GOOGLE. Support 16 KB page sizes. Android Developers. Disponível em: https://developer.android.com/guide/practices/page-sizes. Acesso em: 26 out. 2026.

[24] BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União, Brasília, DF, 15 ago. 2018.

[25] MACENSKI, S. et al. Robot Operating System 2: design, architecture, and uses in the wild. Science Robotics, v. 7, n. 66, eabm6074, 2022.

[26] KOENIG, N.; HOWARD, A. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In: IEEE/RSJ INTERNATIONAL CONFERENCE ON

INTELLIGENT ROBOTS AND SYSTEMS (IROS), 2004, Sendai. Proceedings [...]. IEEE, 2004. p. 2149-2154.

[27] GEBRU, T. et al. Datasheets for datasets. Communications of the ACM, v. 64, n. 12, p. 86-92, 2021.

[28] TOBIN, J. et al. Domain randomization for transferring deep neural networks from simulation to the real world. In: IEEE/RSJ INTERNATIONAL CONFERENCE ON INTELLIGENT ROBOTS AND SYSTEMS (IROS), 2017, Vancouver. Proceedings [...]. IEEE, 2017. p. 23-30.

[29] BAN, S.; FAN, J.; MA, X.; ZHU, W.; QIAO, Y.; WANG, Y. Real-time holistic robot pose estimation with unknown states. In: EUROPEAN CONFERENCE ON COMPUTER VISION (ECCV), 2024. Proceedings [...]. Cham: Springer, 2024.

[30] YANG, L. et al. Depth Anything V2. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NeurIPS), 37., 2024. Proceedings [...]. 2024.

[31] LEE, T. E.; TREMBLAY, J.; TO, T.; CHENG, J.; MOSIER, T.; KROEMER, O.; FOX, D.; BIRCHFIELD, S. Camera-to-robot pose estimation from a single image. In: IEEE INTERNATIONAL CONFERENCE ON ROBOTICS AND AUTOMATION (ICRA), 2020. Proceedings [...]. IEEE, 2020. p. 9426-9432.

[32] SUN, K.; XIAO, B.; LIU, D.; WANG, J. Deep high-resolution representation learning for human pose estimation. In: IEEE/CVF CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR), 2019. Proceedings [...]. IEEE, 2019. p. 5693-5703.

[33] HE, K.; ZHANG, X.; REN, S.; SUN, J. Deep residual learning for image recognition. In: IEEE CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR), 2016. Proceedings [...]. IEEE, 2016. p. 770-778.

[34] LABBÉ, Y.; CARPENTIER, J.; AUBRY, M.; SIVIC, J. Single-view robot pose and joint angle estimation via render & compare. In: IEEE/CVF CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR), 2021. Proceedings [...]. IEEE, 2021. p. 1654-1663.

[35] COLEMAN, D. et al. Reducing the barrier to entry of complex robotic software: a MoveIt! case study. Journal of Software Engineering for Robotics, v. 5, n. 1, p. 3-16, 2014.

[36] ŞUCAN, I. A.; MOLL, M.; KAVRAKI, L. E. The Open Motion Planning Library. IEEE Robotics & Automation Magazine, v. 19, n. 4, p. 72-82, 2012.

[37] KUFFNER, J. J.; LAVALLE, S. M. RRT-Connect: an efficient approach to single-query path planning. In: IEEE INTERNATIONAL CONFERENCE ON ROBOTICS AND AUTOMATION (ICRA), 2000, San Francisco. Proceedings [...]. IEEE, 2000. p. 995-1001.

[38] GRIEVES, M.; VICKERS, J. Digital twin: mitigating unpredictable, undesirable emergent behavior in complex systems. In: KAHLEN, F.-J.; FLUMERFELT, S.; ALVES, A. (ed.). Transdisciplinary perspectives on complex systems. Cham: Springer, 2017. p. 85-113.

[39] GROMEŠ, J. RadioLib: universal wireless communication library for embedded devices. Versão 7.8.1. Disponível em: https://github.com/jgromes/RadioLib. Acesso em: 26 out. 2026.

[40] THE THINGS NETWORK. Frequency plans by country. Disponível em: https://www.thethingsnetwork.org/docs/lorawan/frequencies-by-country/. Acesso em: 26 out. 2026.

[41] AGÊNCIA NACIONAL DE TELECOMUNICAÇÕES (ANATEL). Resolução nº 680, de 27 de junho de 2017. Aprova o Regulamento sobre Equipamentos de Radiocomunicação de Radiação Restrita. Brasília: Anatel, 2017.

[42] AGÊNCIA NACIONAL DE TELECOMUNICAÇÕES (ANATEL). Ato nº 14.448, de 4 de dezembro de 2017. Aprova os requisitos técnicos para a avaliação da conformidade de equipamentos de radiocomunicação de radiação restrita. Brasília: Anatel, 2017.

[43] SEMTECH CORPORATION. AN1200.13: LoRa Modem Designer's Guide. Camarillo: Semtech, 2013.

[44] ABRAMSON, N. The ALOHA System: another alternative for computer communications. In: FALL JOINT COMPUTER CONFERENCE, 1970, Houston. Proceedings [...]. New York: ACM, 1970. p. 281-285.

[45] HATA, M. Empirical formula for propagation loss in land mobile radio services. IEEE Transactions on Vehicular Technology, v. 29, n. 3, p. 317-325, 1980.

[46] KUHN, H. W. The Hungarian method for the assignment problem. Naval Research Logistics Quarterly, v. 2, n. 1-2, p. 83-97, 1955.

[47] KAPOUTSIS, A. C.; CHATZICHRISTOFIS, S. A.; KOSMATOPOULOS, E. B. DARP: divide areas algorithm for optimal multi-robot coverage path planning. Journal of Intelligent & Robotic Systems, v. 86, n. 3-4, p. 663-680, 2017.

[48] DOUGLAS, D. H.; PEUCKER, T. K. Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. Cartographica, v. 10, n. 2, p. 112-122, 1973.

[49] CHOSET, H. Coverage of known spaces: the boustrophedon cellular decomposition. Autonomous Robots, v. 9, n. 3, p. 247-253, 2000.

[50] OPENSTREETMAP CONTRIBUTORS. OpenStreetMap. Dados disponibilizados sob a licença Open Database License (ODbL). Disponível em: https://www.openstreetmap.org. Acesso em: 26 out. 2026.

[51] KOENIG, S. et al. The power of sequential single-item auctions for agent coordination. In: NATIONAL CONFERENCE ON ARTIFICIAL INTELLIGENCE (AAAI-06), 21., 2006, Boston. Proceedings [...]. Menlo Park: AAAI Press, 2006. p. 1625-1629.

[52] ZHU, P. et al. Detection and tracking meet drones challenge. IEEE Transactions on Pattern Analysis and Machine Intelligence, v. 44, n. 11, p. 7380-7399, 2022.

[53] JOCHER, G.; QIU, J. Ultralytics YOLO11. Versão 11.0.0, 2024 (biblioteca ultralytics 8.4.189 usada neste trabalho). Disponível em: https://github.com/ultralytics/ultralytics. Acesso em: 26 out. 2026.

[54] ZHANG, Y. et al. ByteTrack: multi-object tracking by associating every detection box. In: EUROPEAN CONFERENCE ON COMPUTER VISION (ECCV), 17., 2022, Tel Aviv. Proceedings [...]. Cham: Springer, 2022. p. 1-21. (Lecture Notes in Computer Science, v. 13682).

[55] AHARON, N.; ORFAIG, R.; BOBROVSKY, B.-Z. BoT-SORT: robust associations multi-pedestrian tracking. arXiv:2206.14651, 2022. Disponível em: https://arxiv.org/abs/2206.14651.

[56] LUITEN, J. et al. HOTA: a higher order metric for evaluating multi-object tracking. International Journal of Computer Vision, v. 129, n. 2, p. 548-578, 2021.

[57] BERNARDIN, K.; STIEFELHAGEN, R. Evaluating multiple object tracking performance: the CLEAR MOT metrics. EURASIP Journal on Image and Video Processing, v. 2008, art. 246309, 2008.

[58] RISTANI, E. et al. Performance measures and a data set for multi-target, multi-camera tracking. In: EUROPEAN CONFERENCE ON COMPUTER VISION (ECCV) WORKSHOPS, 2016, Amsterdam. Proceedings [...]. Cham: Springer, 2016. p. 17-35.

[59] LUITEN, J.; HOFFHUES, A. TrackEval. 2020. Disponível em: https://github.com/JonathonLuiten/TrackEval. Acesso em: 26 out. 2026.

[60] NVIDIA CORPORATION. NVIDIA Isaac Sim. Versão 5.1.0. Santa Clara: NVIDIA, 2025. Plataforma de simulação robótica baseada no Omniverse, com as extensões omni.anim.people e omni.replicator. Disponível em: https://developer.nvidia.com/isaac/sim. Acesso em: 26 out. 2026.

[61] CAIXETA, E. M. C.; GONÇALVES, L. J. B.; CARDOSO, A. A. Estimated position of the moving axes of a mechanical arm from an unknown state. In: INTERNATIONAL SYMPOSIUM ON MECHANISM DESIGN FOR ROBOTICS (MEDER 2026). 2026.

[62] CAIXETA, E. M. C.; CARDOSO, A. A.; ALMEIDA, R. F. Estimated position of the moving axes of a mechanical arm from an unknown state. In: CONGRESSO BRASILEIRO DE AUTOMÁTICA (CBA 2026), São Paulo, 2026. Artigo e pôster.